FishBot v0.6: Breaking the Exact Tie, Repairing the Optimiser, and Guarding a Determinized Search

Why the exact posterior loses to a policy-aware approximation, what an unguarded search argmax costs, and a measurement protocol that can tell the difference

Dylan Nguyen
FishLab Research Project

Technical report — August 23, 2026
Repository: `https://github.com/dylann29/fish-optimization`
Experiments `E0-E15`, the follow-ups `F0-F10` and the exploitability probe `X1` were produced from the FishBot v0.6 working tree whose baseline is commit `bd812fe` ("v0.5"); the FishBot v0.6 sources themselves were uncommitted when the battery ran, so the artifacts rather than a commit are the record. Digests of all of them are in `research/v06/results/MANIFEST.json`.

Abstract

FishBot v0.5 removed a failure mode and gained nothing in win rate, and said so. This report asks where the remaining value in six-player Canadian Fish actually is. The answer turns out to sit in one place, and it is a place three studies had written off.

At 54.74% of FishBot v0.5's ask decisions two or more candidates score *bit-for-bit identically* and the winner is whichever the enumerator emitted first. 93.77% of those ties are two cards of one half-suit at one target, and the *exact* posterior — not merely the approximation the policy deploys — assigns them identical probability in 0.00% of cases. Every rule that ranks them by that probability therefore realises the same hit rate to three significant figures, and so does enumeration order. The obvious conclusion, which an earlier draft of this work drew, is that the ties are irreducible.

They are not. Resolving the tie group uniformly at random, seeded from the public event stream, is worth *exactly* nothing: 50% [47.1756, 52.8244], mean half-suits 4.500 to 4.500. Resolving it with one sampled deal is worth nothing either: 49.5833%. Resolving it with an ensemble of twelve sampled deals — each a full determinization of the hidden state, played forward by six players reconstructed at their own information sets rather than made clairvoyant — is worth **52.64%** against FishBot v0.5 over 2,160 games and 3 banks, none below 52.08%. The separating information is real and it lives in the *joint* posterior, which every marginal integrates away.

Two negative controls make that a claim about *information* rather than about a deterministic policy being readable, and they are the reason the result is worth stating. Where the gain sits within the search — inside the tie group, outside it, or both — is *not* resolved at this budget: the restricted variants land between 49.8611% and 50.88% and their intervals all overlap the unrestricted search's.

Getting there required correcting the statistic first. An unguarded argmax over the same rollout ensemble is **35.69 points worse** than the blueprint it searches from (13.61% against a 49.31% control at identical budget, seed and sample size), because one determinization's return is the final half-suit differential and genuine differences between candidates are an order of magnitude smaller than its spread. A paired lower-confidence-bound deviation rule recovers all of it.

Two further results. The exact observer-conditioned posterior is a *worse* predictor of card location than the approximation the policy deploys — 47.94% argmax hit rate at 1.42246 nats against 51.49% at 1.38218 — so the matched-budget refit under exact inference that three studies listed

as outstanding is not worth running. And FishBot v0.5's forty-generation refit is indistinguishable from sampling noise, so its shipped vector is v0.4's; five mechanisms in this study cleared a 95% paired interval at the per-cell budget this project has always used and returned exactly zero at three times it.

What ships is two configurations. FishBot v0.6 is the deployed policy: the same mechanisms as FishBot v0.5 with a vector found by a repaired optimiser, and 303.4 games per second against FishBot v0.5's 276.3. It beats FishBot v0.5 head to head by $50.89\% - 50$ over 126,000 games with every one of 7 banks above parity $[50.61, 51.16]$, and FishBot v0.4 by $51.11\% - 50$ over 36,000 games; it is ahead on the paired panel comparison at all 3 banks; its minimax regret over a thirteen-style set is 3.06 against FishBot v0.5's 8.60; and a fitted best response reaches parity against FishBot v0.4 and FishBot v0.5 and fails to reach it against FishBot v0.6 (48.36% $[46.75, 49.97]$). Every one of those margins is about a point. FishBot v0.6-Search adds the tie-resolving search and is the strongest configuration measured, at a cost of three orders of magnitude in throughput.

1 Introduction

Six-player Canadian Fish is a team game of pure public information flow. Fifty-four cards are dealt nine each into two alternating teams of three; a player asks a named opponent for a named card, must already hold another card of that card's six-card half-suit, must not hold the card itself, and keeps the turn on a hit and loses it on a miss; a team scores a half-suit by naming, correctly and in full, which of its members holds each of the six cards. Every ask, every hit, every miss and every declaration is seen by all six players. That single property makes the entire hidden state the *initial deal*, and it makes the posterior over that deal exactly computable — which is why this project has been able to treat inference as a solved problem and spend three studies on policy.

The three prior FishBots are, in order: v0.3, a fitted linear policy in a TypeScript simulator; v0.4, a rules-exact C++ engine with an exact reference posterior, a fast deployed approximation, a twenty-feature ask score and a value-based declaration rule; and v0.5, which diagnosed and removed a failure mode in v0.4 — a two-question deadlock in mirror play that ended in misdeclaration — and reported, in terms, that removing it was worth almost nothing in win rate.

That report is the starting point here. A project whose last two releases produced no measurable strength gain is either at the ceiling of its policy class or is measuring badly, and the two possibilities call for opposite responses. This study answers the question by building the mechanisms the accumulated design register ranked highest, measuring each at a resolution the project had not previously used, and then measuring the measurement.

Contributions.

1. **Exchangeable ties are irreducible** (§6.3). More than half of v0.5's ask decisions end in a bit-for-bit tie; the tied candidates are two cards of one half-suit at one target; the *exact* posterior separates 0.00% of them; and every tie-break rule realises the same hit rate. The largest item on the inherited register is a property of the information set, not a defect.
2. **Exactness is not the same as informativeness** (§5.4). The exact posterior is a worse predictor of card location than the deployed approximation, because the approximation carries a soft policy prior and the exact object is exact under a uniform prior over deals. This closes an experiment three studies have listed as outstanding.
3. **The optimizer's curse in determinized search** (§8.3). A search that determinizes the deal but reconstructs all six continuation players at their own information sets — not a double-dummy rollout — is 35.69 points worse than its own blueprint when the action is chosen by an unguarded argmax, and a paired deviation rule recovers all of it. Guarded, it is the one mechanism in this study that beats a static rule: 52.08% against FishBot v0.6 itself over 2,880 games with every cell above parity.

4. **A fitting harness that moves the policy** (§9). Four measured defects in the previous optimiser, a repair, and a recovery test the previous one provably fails. The repaired optimiser finds a vector that trades nothing head to head for a large, replicated gain against the deception archetype.
5. **An evidence standard** (§10.3). Five mechanisms in this study cleared a 95% paired interval at the per-cell budget this project has used since v0.3 and returned exactly zero at three times it, on two disjoint banks.
6. **Engine corrections** (§4.4, §12), including an exact-inference object that returned another observer's tables whenever two observers held one, and a declaration pre-gate audit that had been dead code since v0.5.

What this paper does not claim. FishBot v0.6 beats FishBot v0.5 head to head by 50.89% − 50, which is *small*: about nine tenths of a point, resolved only at 126,000 games. It is not a large win-rate improvement, and at the per-cell budget this project has always used it reads as a null. The substantive claims are robustness across playstyles, lower measured exploitability, correctness, and a substantially narrowed design space.

2 The game and the rules dialect studied

Canadian Fish, also called Literature, is played by six players in two teams of three, seated so that the teams alternate. The 54-card deck is partitioned into nine *half-suits* of six cards, each player is dealt nine, and the team that takes at least five half-suits wins. On their turn a player asks one opponent for one named card; the ask is legal only if the named card lies in a live half-suit, the asker holds at least one other card of that half-suit and not the named card, and the target is an opponent who still holds cards. A hit transfers the card and retains the turn, a miss passes the turn to the target, and every ask, answer, transfer and card count is public. A half-suit is claimed by a *declaration*: an exact allocation naming, for each of the six cards, which member of the declarer's own team holds it, awarded to the declaring team if every assignment is correct and to the opposing team otherwise.

That is the whole game, and it is the same game the two preceding reports studied. The dialect is unchanged from FishBot v0.4 and FishBot v0.5, so this section states the selection and points at the treatment rather than repeating it: §A gives the clause table and the clauses this study's mechanisms depend on, and the v0.4 report's Appendix A gives the dialect at the length required to re-implement it [2].

2.1 The dialect studied

Published descriptions of the game document mutually incompatible variants [4], so the dialect below is one explicit selection among them and not a canonical rule set. It matches the strategy corpus this line of work draws on [5] and the rules of record supplied by the project owner, and every contested choice is an engine switch, so its effect is measurable rather than assumed. The consequential choices are: a 54-card deck in nine half-suits, nine cards per player; declaration legal at any moment, in or out of turn, by a seat holding cards or not; a misdeclaration awarding the half-suit to the opposing team; lowest seat index first among simultaneous declarers; an eight-level willingness ladder in the forced endgame that arises when one whole team is cardless; and a 400-ask action cap whose residue is adjudicated neutrally by physical majority. No prearranged signalling conventions are available to any policy, and team communication is limited to that willingness bit and to the successor chosen by a cardless turn-holder. Table 16 in §A lists each choice against the switch that changes it. §11.11 reports the primary result under 4 dialect perturbations; unless a result says otherwise, every number in this report was produced under the settings above, and the action cap was reached in 0.00% of games.

Two of those choices are places where the engine is *more restrictive* than the rules of record allow, and both are coordination channels a policy could in principle use. The v0.5 study measured both, and this report inherits its conclusions rather than re-opening them.

Turn transfer. When the turn reaches a cardless seat whose teammates both still hold cards, the rules of record permit the team to negotiate openly over who receives it, sharing nothing but willingness. The engine instead has the cardless seat choose unilaterally from its own belief, and solicits no willingness bit. The channel is empty: a genuine multi-candidate transfer decision arises 0.148 times per game, and replacing the unilateral rule with a *ground-truth* chooser on one team, paired on common deals, is worth -0.0007 ± 0.0024 sets per game over 6,000 paired games, with the two arms diverging at all in 0.48% of games (`research/v05/results/P8-coordination.md`, §1.4). A legal mechanism cannot beat the clairvoyant one, so that interval bounds the whole channel, and no version of this policy has ever been built to exploit it.

Declaration arbitration. The rules do not say who acts first when two seats offer a declaration at the same instant, so the order is a modelling choice. The engine scans by lowest seat index, deliberately declining to rank candidates by stated confidence, because a confidence comparison moves a statistic of one seat's hand to five others. The cost of that refusal is +0.37 pp of win rate pooled over 30,000 games, range +0.28 to +0.45 pp across five seeds — and a clairvoyant arbitrator, which picks a proposer whose named allocation is actually correct, is worth +0.35 pp on the same design (`research/v05/results/P6-verify-arbitration-cost.md`, §§3–4). The information-safe rule therefore forgoes a third of a percentage point, and no rule of any kind, safe or unsafe, informed or omniscient, recovers appreciably more.

2.2 What is unchanged from the v0.4 and v0.5 studies

Nothing in the dialect, the engine's rule implementation or the deal generator changed between FishBot v0.5 and FishBot v0.6. A reader who follows the citation to the v0.4 report's rules treatment will find it accurate on every clause except two, which no report has yet retracted and which §12.1 corrects here:

1. **An undeclared half-suit does not score nothing.** The v0.4 report justifies the unconditional second stage of its forcing rule on the premise that an unclaimed half-suit is worthless. Under the dialect actually evaluated — by both studies — residue at the action cap is awarded by physical majority, and on a wrong post-horizon declaration the declaring team holds a mean 4.08 of the six cards. The stage converts a probable award into a certain concession. The v0.4 paper states the dialect correctly in its rules section and contradicts it two sections later.
2. **Restarting the forced-endgame ladder sharpens nothing.** The v0.4 report argues that restarting the willingness ladder after each declaration lets early, safer declarations resolve cards and so sharpen the allocations available for later ones. The argument is sound and inoperative: on every occasion a team went cardless with live half-suits — 566 of them at seed 31 — exactly 1 half-suit was live, so there is no ordering to exploit.

Both are restated correctly in §A.3 for a reader who reaches the appendix first, and both bear on this study: the first is a premise the declaration mechanism of §7.4 must not inherit, and the second describes a structure that a more patient declaration policy would begin to produce.

3 Related work

This section covers the work the rest of the report builds on, and one class of work it deliberately declines. The background the main argument does not depend on is collected in §I, the evaluation-

methodology literature is taken up in §10, and §I.10 states, for each declined method, why it is declined. Absence is a design decision in this study, not an omission, and it is recorded as one.

Three of this report's results sit against specific literatures and are placed here rather than left implicit. The optimiser's-curse result of §8.3 belongs to the determinization line, where an unguarded argmax over sampled worlds is a known-in-principle failure that is rarely quantified. The belief result of §5.4 — a policy-aware approximation beating the exact posterior — belongs to the trick-taking inference line, which is the only literature that has measured the same trade-off. And the shared common-knowledge object of §7.3 is the common-information reduction, restricted to the one case in which it is cheap.

3.1 Fish and Literature

The published material on this game is human and informal, and neither of its two substantial sources is quantitative or describes a program. McLeod's Pagat entry [4] is the canonical rules source and records several points on which dialects differ; ours is specified in §2 and listed in full in §A. Develin's chapter [5] is the most detailed strategy writing we located, describes the same dialect, and supplies two observations this design uses: an ask teaches the opponents as much as it teaches a teammate, and a player may not declare a half-suit they hold entirely. Secondary strategy pages [6, 7, 8] restate blackballing, asking the asker, and an endgame delegation heuristic; they are evidence about how the game is played and talked about, not sources of quantitative claims. §I.1 records where the two substantial sources disagree.

Computational prior art remains close to absent. The v0.4 study's searches found no academic work on this game, no public repository containing search, planning or a working learner for it, and no coverage in RLCard [12]; the v0.6 literature refresh re-ran that question against 2025–2026 work and found nothing that changes it. The one agent located, Somani's `literature` [9, 10], plays the four-player, 48-card variant, which does not raise the six-player allocation problem, and publishes no strength numbers; a closed-source Android application advertises bots with no stated methodology [11]. Every novelty statement in this report therefore takes the form "we found no prior published work that ..." and should be read against that scope. One 2026 item is worth naming because it may become the external calibration this line of work lacks: Valet packages twenty-one traditional imperfect-information card games in a common description language with measured branching factors and durations [13], though whether any of the twenty-one is a team game is not stated on its abstract page and we did not fetch the full text, so nothing here is compared against it. The three prior FishBot reports [1, 2, 3] are the record this study extends and, in §12, corrects.

3.2 Determinization, and why double-dummy analysis fails here

Determinization, or perfect-information Monte Carlo — sample hidden cards consistently with the public history, solve the resulting perfect-information game, and play the move with the best average value — is the standard recipe for hidden-hand card games [14, 15], and is unsound however many worlds are drawn. Frank and Basin named the two mechanisms: *strategy fusion*, in which the searcher plays a different strategy in each sampled world although a real player must commit to one strategy per information set, and *non-locality*, in which a node's value depends on parts of the tree outside its own subtree [16, 17]. Long, Sturtevant, Buro and Furtak characterise when the method nevertheless performs well, through leaf correlation, bias and a disambiguation factor [18]; Information Set Monte Carlo Tree Search replaces independent trees with a single tree over information sets [19, 20]; the $\alpha\mu$ family backs up Pareto fronts of world-vectors instead of scalar averages, repairing both mechanisms, and reports Bridge 3NT declarer play at 60.2% to 63.0% with twenty worlds [21]; and knowledge-based paranoia search reasons about the worst case over deduced-consistent worlds rather than the average,

reporting over 1,000 points on the extended Seeger scale in Skat [22]. Its runtime is not stated on the abstract page, so no compute cost is attributed to it anywhere in this report.

Fish breaks the method for a reason none of that literature addresses. Its perfect-information game is not merely fused, it is degenerate: a clairvoyant player names only cards the target actually holds, so never fails an ask, never loses the turn, and declares each half-suit correctly as soon as their team holds all of it. Almost every reachable position carries the same perfect-information value, and averaging that value over deals sampled from the public history collapses to choosing the ask most likely to hit — a one-ply heuristic blind to the information an ask leaks, the information a refusal supplies, and the option value of postponing a declaration. $\alpha\mu$ repairs unsoundness, not degeneracy, so it does not rescue the construction here. §I.2 gives the collapse in full.

What §8 adds is that the binding problem in this game is not the abstract failure but the statistic. One determinization's value is the final half-suit differential, with standard deviation 2.5 — an order of magnitude larger than the differences between the leading candidates — so an argmax over D sampled means selects on residual noise, and does so more confidently the more candidates it is offered. That is the optimiser's curse rather than strategy fusion, and it is measured here at 35.69 points by a control that forces the same code to return the blueprint's own pick (§8.3). This report therefore uses determinization only as a research probe, behind the paired deviation guard of §8.4, and reports the result including the configurations that land at zero (§8.5). The transferable claim is the guard, not the search.

3.3 Search in imperfect-information and cooperative games

States in an imperfect-information game do not have well-defined values, so depth-limited search needs something at the leaf that represents the opponent's ability to choose. Brown, Sandholm and Amos supply it: let the opponent select among k continuation strategies at the depth limit, each yielding its own leaf-value vector, and require the searcher to be good against that choice — demonstrated at master level in heads-up no-limit hold'em on a four-core CPU and 16 GB [27]. That construction is the one piece of this literature this study uses as method rather than context: it is the multi-valued leaf of §7.5 and the prerequisite for §8. The soundness argument is stated for two-player zero-sum games, and Fish's team-level game presents three uncorrelated opponent seats at the leaf, so nothing is inherited. It is adopted here as a robustification heuristic with a measurable consequence, and audited rather than assumed (§7.7).

Two lines address the other half of the problem, which is what a search may prune. Zhang and Sandholm solve subgames over low-order knowledge rather than the full common-knowledge closure, which is the published answer to a closure too large to enumerate — and they prove that the naive form can *increase* exploitability [28]. That warning applies directly to the live-ask gate this policy already ships, which is a knowledge-limited pruning rule that has never been audited, and it is why §7.7 was built before any search work rather than after it. Subgame solving in adversarial team games builds a gadget over the team belief DAG and runs in polynomial time given a best-response oracle *when the blueprint is sparse* [29]; a Fish blueprint is not sparse, so that escape hatch does not open here, and §I.10 records this as a verified non-transfer rather than an unexamined gap.

In cooperative games the constraint is different and it transfers verbatim. SPARTA improves a blueprint purely at test time, 24.08 to 24.61 in Hanabi, and distinguishes single-agent search — which assumes every other agent strictly follows the blueprint — from multi-agent search, in which all agents run the *same* common-knowledge procedure [31]. The moment a second teammate also searches, the first teammate's belief over its partner is computed under a policy that partner is no longer playing, and the improvement guarantee is void. This is the Dec-POMDP common-knowledge caveat, and it is the constraint most easily violated in silence. Fish satisfies it cheaply: all card movement is public, so any *deterministic* procedure that reads only public state and its own hand is automatically common

knowledge among the three teammates, with no shared seed and no communication. The qualifier is load-bearing — a randomised tie-break loses the property unless its randomisation is seeded from the public history, which is a one-integer decision with a real correctness consequence (§7.2).

The remainder of this literature is cited as context. The full-solver line [40, 38, 39] established that test-time re-solving over public belief states is principled rather than a hack, and ReBeL's safety result is the licence under which any test-time search here operates; the algorithms are declined, because each requires iterating or learning over a closure whose cardinality at deal time is $45!/(9!)^5 \approx 1.9\times10^{28}$ deals from one seat. Update equivalence recasts search as replicating a last-iterate-convergent update locally, needing no public-information enumeration at all, and reports matching or exceeding public-belief-state search in Hanabi at two orders of magnitude less search time [30]; its improvement guarantee is for common-payoff games, which Fish is not, so it informs the shape of the mechanism set without licensing it. Learned Belief Search buys 55–91% of exact search's benefit at 4.6–42× less compute [32] and is simply redundant here, because this game's exact belief is already computed and oracle-validated (§5.2). Online planning by policy fine-tuning [33] requires gradient updates at decision time and does not transfer to a CPU-only engine with no network. The depth-limited counter-strategy line [34, 35] carries one warning this study honours — robust-response methods do not compose with limited-look-ahead solving off the shelf — and the learned-model and gadget-refinement work [36, 37] is recorded for its meta-lesson rather than its machinery: when a search subproblem has many near-equivalent optima, which one is selected is not a tie-break, and refinement is reported at more than 50% of exploitability. That is the same observation §6.2 makes about an argmax resolved by array order.

Finally, every method above is published with a caveat that it can increase exploitability, so none of them can be evaluated by win rate alone. Local best response gives a cheap lower bound: a small greedy search, at most two actions deep, showed every entrant in a 2016 poker competition to be more than 3,180 mBB/h exploitable [41]. Its two documented pitfalls are carried into §11.5 — restricting where the response may begin deviating understates exploitability, and a negative score proves nothing — and the figure this report publishes is a lower bound within its response class, not an exploitability number.

3.4 Policy-aware inference

Skat engines enumerate the deals consistent with the public record and reweight them by a model of what each player would have done — a supervised per-card location model [24], or reach probabilities taken from a policy [25] — and a dedicated inference module was reported worth +217 tournament points per 36 games to the engine Kermit on its own [23]. The policy prior of §5.3 is the same idea in a much simpler form: a fitted soft reweighting of the rules posterior, with two parameters and no learned opponent model.

This literature is where the central result of §5 belongs, and it is the only literature that has measured the same trade-off. What is new here is the margin and its direction. In this game the exact conditional posterior is computable in closed form and cheap (§5.2), so the usual excuse for approximating — that exactness is unaffordable — does not apply; and yet substituting the exact posterior into an otherwise identical policy *loses*, at two independent seed banks. The exact path resolves more distinctions and predicts hits worse, and the reason is structural rather than numerical: the block construction consumes the deduction state alone and has no parameter through which behaviour could enter, while the approximate path is policy-aware by construction. Exactness and behaviour-awareness are not available in the same object here, and §7.2 is built on that split. The report is careful about what this does and does not show: the weights were fitted for the approximate belief, so it is a substitution result rather than a verdict on exact inference (§13).

Two adjacent lines are named and not used. Belief modelling in cooperative games — the Bayesian

Action Decoder's public-belief formulation [43], its factorised-prescription successor [47], and the learned belief models above — is aimed at games whose exact belief is intractable, which is the opposite of the situation here. And the exact-counting machinery this study does not need to approximate — Ryser's inclusion–exclusion formula [61], Glynn's sign-vector formula [62], the Jerrum–Sinclair–Vigoda chain [63], matrix scaling [64] and sequential importance sampling [65] — is recorded in §I.8, along with why the block structure of this game sidesteps all of it. History filtering is FNP-complete in general [26]; Fish escapes because its public record determines every card movement, so the only object to reconstruct is the initial deal. No learned belief model appears anywhere in this study, and that is a stated non-goal rather than an oversight.

3.5 Team games and common information

Three teammates share a payoff, hold different information and cannot communicate. The formal statement of that situation is the common-information approach: reformulate the decentralised problem from the point of view of a coordinator who knows the common information and chooses *prescriptions*, maps from each controller's private information to its action [55]. In Fish the common information is the entire public history, so the coordinator's information state is exactly the belief object the engine already maintains — which is the formal content of the mechanism in §7.3 that replaces six per-seat deduction states with one shared object plus per-seat refinement. It is also the reason the SPARTA constraint above exists: if one seat searches and its teammates do not, they are no longer executing the prescription the first seat's belief update assumes, and the reduction silently fails. Solving the coordinator's problem is out of reach — a prescription maps every possible nine-card hand to an ask — so this is a correctness lens, not an algorithm.

The team-game equilibrium literature is cited to delimit scope. The target concept for a Fish team is a team-maxmin equilibrium with correlation [56], connected to extensive-form correlation by Farina and coauthors [57] and made computable through the team belief DAG and the two-player conversion [58, 59], at a representation cost exponential in how much private information distinguishes teammates inside a public state. Three unrestricted nine-card hands is exactly the case that cost forbids. Hidden-role machinery [60] does not apply at all, because Fish teams are public from the deal. Nothing in this report is an equilibrium result of any kind.

Conventions are the other half of team play, and the coordination literature supplies both the vocabulary and the warning. Hanabi is the reference benchmark for coordination through observable actions [42]; Other-Play demonstrated how much of a self-play score is convention rather than skill [45], the Simplified Action Decoder showed what conventions need in order to survive exploratory training [44], and Off-Belief Learning supplies an explicit dial on convention depth together with grounded play at level one [46]. Grounded play is unusually strong in this game because ask legality is itself a hard certificate: an ask proves its asker holds another card of the named half-suit, so reading a teammate's ask grounded is exact rather than approximate. Self-explaining deviations formalise the opposite move — deviate so that a teammate's theory of mind concludes circumstances are abnormal — and produced the first algorithmic finesse plays in Hanabi [48]; in Fish the same channel is supplied by the rules rather than learned, which is a structural advantage and, in cross-play, a structural risk. Joint policy search is the bridge-bidding analogue, scoring simultaneous changes at several information sets, at +0.63 IMPs per board against a strong commercial program [50]; the Hanabi construction that most clearly fails to port is the hat-guessing code, which works because each receiver sees every other receiver's hand [49], and in Fish no player sees any hand.

Human-regularised search is where that risk is priced. piKL anchors a search result to a reference policy with a bounded divergence [51], and the same construction was extended to coordination with real human partners [52] and used as the planning core of a Diplomacy agent whose game family — adversarial teams acting through public actions — is the one Fish belongs to [53]. The evidence

that a small human corpus is not the way in is also published: in the ad-hoc human-AI coordination challenge, an agent trained with no human data at all outperformed a best-response-to-behavioural-cloning baseline [54]. This study uses none of these as method. It takes from them the requirement that the partner regime be a stated parameter of the policy rather than an assumption, which is §7.6, and that a convention-bearing mechanism be reported with its cross-play cell and not only its self-play cell.

The last entry is a counterweight to the rest of this section. A 2026 gold-standard study of lightweight game-playing agents concludes that the binding constraint on strength is information rather than model capacity, and that disciplined evaluation against a held-out expert beats deeper search [74]. That is this study's own thesis stated by someone else, and it is the reason two of this report's four principal results are negative and are reported as such.

4 The simulator: implementation, information safety, verification, and the v0.6 repairs

The simulator is the instrument every number in this report is read from, so its properties are stated before its results. Three of them matter: it cannot leak a hidden field to a policy, it checks its own deductions against the truth while it runs, and a configuration in which every v0.6 mechanism is switched off is FishBot v0.5 bit for bit. The last of those is not a courtesy. It is what makes the ablation table of §11.8 exact rather than approximate, and it is the first thing `engine/experiments_v06.sh` runs.

This section also records four repairs to the engine itself. They are reported here rather than folded silently into a diff because three of them are the reason a measurement in this report was possible at all, and because one of them — a dead command-line option — means a published pass in the v0.4 record was vacuous (§12.1).

4.1 Implementation

The engine is unchanged in structure from the v0.4 report, which describes it in full [2]: a single-header C++20 core in `engine/src/`, a hand as one `uint64_t` over the 54 card indices $c = 6h + i$, and counter-based random streams keyed by *(seed, seat, purpose)* so that a deal and its entire play reproduce from the seed alone. The duplicate-block protocol of §10 depends on that last property and on nothing else.

What moved in v0.6 is the policy layer. `V06Agent` derives from `V05Agent` and defers to it whenever no v0.6 mechanism is enabled, so the v0.5 decision path is not reimplemented but inherited; the determinized search of §8 and the rollout blueprint it calls live in two new headers, `engine/src/v06.hpp` and `engine/src/v06_rollout.hpp`. The diagnostics this report leans on are a new subcommand rather than scratch probes: `fish v6probe -mode=ties` censuses ask decisions and every tie-break rule including hindsight, `-mode=belief` scores several posteriors as predictors on identical states, and `-mode=search` reports how often a guarded search actually deviates from its blueprint. All three are in `engine/src/probe_v06.hpp` and all three are run by the battery, so the figures they produce are regenerated rather than transcribed.

Throughput on the shipped configuration is 303.4 games/s against 276.3 for FishBot v0.5 and 147.9 for FishBot v0.4, all on the same machine at 15 threads. Every compute figure in this report is single-machine and was measured under load, so all of them are lower bounds.

4.2 Information safety

The acting policy never receives the game state. It receives its own `Knowledge` object — the constraint bookkeeping of §5 — and the public record, in which each event carries only the actor, the target, the named card, the outcome, any declared allocation, and the resulting public hand counts. There is no interface through which a policy can read another player's cards, so the class of leak the FishBot v0.3 audit reported in its predecessor [1] is excluded structurally rather than by inspection. The property and the argument for it are unchanged from the v0.4 report [2].

Soundness is the stronger claim, and it is checked rather than argued. After every public event the engine's audit mode verifies, for every player and against the true deal, that every card the constraint state records as resolved is held by the seat it is resolved to; that the true holder of every unresolved card lies in that card's surviving support; that the derived capacities equal the true counts of unresolved cards per player; and that every live ask-legality certificate is satisfied by the true deal. A single violation would mean the belief engine had excluded the truth.

4.3 Verification suite

`fish verify` plays the ordered cross-product of a nine-policy pool — 81 ordered pairs, so both orderings of every pair and every self-play pairing are exercised — with the audit above enabled throughout, and additionally checks deck integrity and card conservation, that exactly nine half-suits are resolved in every completed game, that a declaration only ever assigns cards to the declarer's own teammates, that ask legality is enforced independently on the move generator and on the applier, and that replaying a seed at a fixed thread count reproduces the game bit-identically. It returns a non-zero exit status on failure, which is what lets the battery use it as a gate rather than as a report.

Over 600 deals the suite recorded 0 violations in 23,594,580 checks, with an overall verdict of PASS. The random streams are keyed so that thread count cannot affect a game's outcome, but the suite tests replay identity only at a fixed thread count and does not test that claim directly; this is carried forward from the v0.4 study as an unclosed gap and is listed as one in §G.5.

4.4 The v0.6 engine repairs

Each repair below is stated as a defect, the measurement that shows the defect is real, what the defect blocked, and the measurement that shows it is fixed. Three of the four are correctness-neutral for FishBot v0.5: they change nothing a FishBot v0.5 game does, which is why they survived a release. The fourth changes the Fast posterior and is therefore a flag that defaults off.

BlockDP tables aliased across instances. `engine/src/blockdp.hpp` parked every instance's Unit, Group, forward and backward tables in one `thread_local` buffer pool, and `BlockDP::build` took its storage from that pool without owning it. Two `BlockDP` objects alive on one thread therefore shared storage, and a second instance's `build` silently repointed the first instance's tables at its own data. The driver makes this routine rather than exotic: agents own one `block` member each and rebuild only when dirty, while `Game::forcedEndgame` and `Game::declarationRound` poll several agents back to back between public events.

The defect is real and was measured directly. Reading one instance's own group table twice with an unrelated `build` in between gave 285 mismatches in 294 checks: one observer's posterior tables had been replaced by another observer's (`research/v05/results/P2-forced-endgame.md`, §6). An earlier form of the same check that issued a real query after the clobber died with `SIGBUS`, because the stale forward and backward pointers index a table sized for a different capacity vector.

What it blocked is the whole of §5. The deployed approximate belief never builds a `BlockDP`, so nothing in the v0.4 or v0.5 release path could observe the defect; anything exact hits it on the first

multi-agent poll. That asymmetry is exactly how a fatal defect survives into a release, and it is why the exact posterior could not be evaluated as a predictor before this study.

The repair is a per-thread generation stamp plus a stored copy of the source `Knowledge`. Every `build` increments the counter and records its value; every public query calls `ensureCurrent`, which re-derives the instance lazily from its stored `Knowledge` when it finds its own stamp stale. The check cannot be forgotten at a call site because it lives inside the queries, and it costs $O(1)$ memory. The probe in `engine/src/probe_forcedendgame.hpp` was extended to separate the two things the original check conflated: `QUERY` mismatches, where the same query issued on the same instance before and after an unrelated build returns different answers, and raw shared-pool field reads, where the instance's own pointers still address the pool and the fields behind them have changed. The first must be zero and is 0. The second is 175 and is *not* fixed: the pool is still shared, the raw fields still change, and the guard's contract is that no public query ever reads them stale. Reporting only the first would overstate the repair.

`gateaudit` was dead code for v0.5. The declaration pre-gate audit is instrumentation that counts how many declaration opportunities a cheap pre-gate rejects and how many of those rejections were false negatives — candidates the expensive test would have accepted. The switch that enables it was parsed only in the FishBot v0.4 branch of `engine/src/factory.hpp`. A v0.5 specification therefore parsed `gateaudit=1`, ignored it, and reported a pass over zero opportunities: `fish gateaudit -a=v05:gateaudit=1` could not fail, because it never looked at anything. §12.1 records what that means for the v0.4 record's corresponding claim.

The option is now parsed in the v0.5 and v0.6 branches as well. The audit over 5,472,906 declaration opportunities rejected 14,449,770 at the cheap gate with 0 false negatives, verdict PASS. The instrument is only as good as its opportunity count, so that count is reported first and the verdict second.

Two defects in the tuner. `engine/src/tuner.hpp` chose between the two parameter-passing spellings with `w.size() > 18` where the ask-feature count `NFEAT` — 20 — belongs. The expression is correct today by coincidence, and it is the same class of defect, in the same place, that cost the v0.4 study a fitting round when the ask-feature count grew. It now reads `NFEAT`. The second defect is worse because it is silent and quantitative: the win rate was computed as `st.winsA / (st.games * 2)`, hard-coding the tuner's two rotations into the denominator. Fitting at six rotations — which the duplicate-block design of §10 makes the natural choice — would have reported a third of the true win rate for every candidate, with no error and no warning. It now reads `st.games * mc.rotations`.

Certificate de-duplication, opt-in. `Knowledge::onEvent` in `engine/src/belief.hpp` appended a fresh ask-legality certificate on every repeated ask without checking whether an identical one, or one strictly stronger, was already stored. The store therefore grows carrying no information, and every consumer that iterates it pays for the duplicates. The obvious repair — skip a certificate that duplicates a stored one, or whose card set is a superset of a stored one for the same player — is implemented, and it is a flag that defaults off. The reason is that the duplicates are not inert. The approximate conditioning step of §5 applies a certificate once per stored copy, so removing copies *changes* the Fast posterior and therefore changes play. Making de-duplication the default would have silently altered every FishBot v0.4 and FishBot v0.5 number this report compares against. With the flag off, both stay bit for bit what they were.

The identity control, and why it is a result. `v06:legacy=1` restores FishBot v0.5's parameter vector and zeroes the three v0.6 ask terms, and with every mechanism switch off `V06Agent` defers to

its base class. The battery checks that this is an identity and not a close agreement, by comparing the md5 digest of the entire `fish pathology` output — every pathology counter, not a win rate — between `v06:legacy=1` and `v05` on the same deals and seed. The verdict is PASS.

This matters more than a build-hygiene check. Every ablation in §11.8 and §D is a paired comparison between the v0.6 binary carrying mechanism X and the v0.6 binary carrying nothing. If the all-switches-off configuration merely resembled FishBot v0.5, each row would confound the mechanism with whatever else differed, and the table would be an approximation whose error had no bound. Because the configuration is FishBot v0.5 bit for bit, the difference each row reports is the mechanism and nothing else. The control is cheap, it is run before any strength number is collected, and it is the reason the ablation table can be read as exact.

5 The observer-conditioned deal posterior

This section builds the object the rest of the report measures, and then reports the first of the study's two principal negative results: the exact posterior is a worse predictor than the approximation the engine actually deploys.

The order matters, because the result is easy to misread in either direction. Two facts about the exact posterior are both true. It resolves distinctions the approximation cannot — it is the exact conditional count under the rules, with no independence assumption anywhere in it — and it predicts where cards are worse than the approximation does. Resolution and calibration are different properties, and this section keeps them apart throughout. The reason the exact object is the worse predictor is not that the approximation is secretly better at counting. It is that the exact object is exact *with respect to a uniform prior over deals*, and that prior is wrong.

5.1 The hidden state is the initial deal

In the dialect of §2 a card changes hands in exactly one way, a successful ask, which every player observes; declarations remove cards from play, and that too is public. The v0.4 report proves the consequence [2]: a card whose location has never been publicly revealed is still held by the player to whom it was dealt, the joint hidden state at any time is exactly the initial deal, and the current holder of every card is a deterministic function of the initial deal and the public history. The argument is one line — every change of holder is an announced event, so a card with no such event has not moved — and it is cited here rather than reproved.

What that buys is the elimination of the filtering recursion that dominates inference in most imperfect-information card games. There is no belief that decays and no dynamics to approximate, only a fixed hidden object and an accumulating set of logical constraints on it. Every player computes the posterior from the same public history but conditions additionally on their own private hand, so the six posteriors differ and no single distribution is shared.

The constraints are accumulating rather than merely present, and the rate at which they accumulate governs whether an exact endgame treatment is worth building. A game reaches a decided state after a mean of 76 public events, which is the figure §7 uses to size the endgame regime.

Notation. $\mathcal{H}$ is the set of live half-suits and $H \in \mathcal{H}$ one of them. Fix an observer i. U is the set of cards in live half-suits whose holder is not publicly known to i, and for $c \in U$ the map $\sigma : U \to \{0, \ldots, 5\}$ gives that card's holder, constant in time. $M_c \subseteq \{0, \ldots, 5\}$ is the surviving support of $\sigma(c)$, and q_p is the capacity of seat p, the number of cards of U that seat must hold. Z is the number of assignments σ consistent with the constraints below, the partition function of the posterior. The three quantities the policy consumes are the per-card ownership marginal $\mu_{c,p} = \Pr[\sigma(c) = p]$; the probability $\tau(H, T)$ that team T owns every card of half-suit H; and the probability $\alpha(A)$ that a

named allocation A — an assignment of all six cards of one half-suit to named seats — is correct. Unhatted symbols are values from the exact engine, hatted ones ($\hat{\mu}_{c,p}$, $\hat{\tau}$, $\hat{\alpha}$) the same values from the approximate path. Probabilities are conditional on the observer's information throughout and the conditioning bar is suppressed.

The constraint system. The observer's information is exactly five constraints on σ.
(C1) Own hand. $\sigma(c) \neq i$ for every $c \in U$, and every card i holds is resolved.

(C2) Public transfers and correct declarations. Any card whose holder has been revealed leaves U with its holder known exactly. A successful ask reveals the holder *before* the transfer, and that is the value relevant to every earlier constraint.

(C3) Exclusions. An ask for c proves the asker did not hold c; a miss proves the target did not hold c. Both are permanent, because unresolved cards never move. The surviving support M_c records them.

(C4) Capacities. Hand counts are public, so for every player p

$$\big|\{c \in U : \sigma(c) = p\}\big| \;=\; q_p \;:=\; h_p - k_p, \tag{1}$$

where h_p is p's public card count and k_p the number of live cards already known to be p's. Note $\sum_p q_p = |U|$ automatically, and that a declaration — correct or not — is absorbed by (1) without special handling, because the count drop it causes is public.

(C5) Ask legality. An ask for c in half-suit H by player a proves that a held some *other* card of H at that moment. Because unresolved cards never move, this is the time-independent disjunction

$$\bigvee_{d \in D} [\sigma(d) = a], \qquad D = \{d \in H \setminus \{c\} : d \text{ unresolved},\ a \in M_d\}, \tag{2}$$

vacuous if some card of H is already known to be a's. Every such certificate is confined to a single half-suit, which is the structural fact §5.2 exploits.

The prior is uniform over deals consistent with the rules, so the posterior is uniform over the assignments σ satisfying (C1)–(C5) and $\Pr[\sigma] = 1/Z$ for each of them. That sentence contains the whole of §5.4 in advance: the object is exact with respect to *that* prior, and the prior is a modelling choice, not a rule of the game.

5.2 The exact count law

Set (C5) aside. Counting assignments subject to (C3) and (C4) is a degree-constrained bipartite counting problem, solved exactly by a forward–backward recursion over the vector $n = (n_0, \ldots, n_5)$ of *remaining* capacities. Ordering U as $c_1, \ldots, c_{|U|}$, $F_j(n)$ counts placements of the first j cards leaving n unused and $B_j(n)$ placements of the rest out of n:

$$F_j(n) = \sum_{p \,\in\, M_{c_j} :\, n_p < q_p} F_{j-1}(n + e_p), \qquad F_0(q) = 1, \tag{3}$$

$$B_j(n) = \sum_{p \,\in\, M_{c_{j+1}} :\, n_p \geq 1} B_{j+1}(n - e_p), \qquad B_{|U|}(\mathbf{0}) = 1, \tag{4}$$

$$Z = F_{|U|}(\mathbf{0}) \;=\; B_0(q), \tag{5}$$

with e_p the p-th unit vector. Every card consumes exactly one unit of capacity, so the layer index is recoverable from n and the $|U|$-layer table collapses to two arrays of at most 10^5 entries (§B.2). These recursions count rather than sample, so they and the per-card marginal they contract to are exact for (C1)–(C4).

Ask legality by half-suit block enumeration. Constraint (C5) is disjunctive and breaks the product form of (3)–(4), and inclusion–exclusion over the tens of certificates a game accumulates would cost 2^k passes. The escape is structural: every certificate lives inside one half-suit, so U partitions into half-suit blocks admitting one of two exact treatments. A block with no live certificate is expanded card by card as in (3)–(4). A block with a live certificate is enumerated exhaustively — at most $\prod_{c\in H}|M_c| \le 5^6$ assignments — each tested against (2) directly, and the survivors aggregated into a count-vector table

$$g_H(t) \;=\; \big|\{\text{assignments } \beta \text{ of } H : \text{count}(\beta) = t,\ \beta \models \text{certificates of } H\}\big|, \tag{6}$$

where $\text{count}(\beta)_p$ is the number of H's unresolved cards β gives to seat p. Either way a block consumes a known number of cards, so the layer identity survives and the dynamic program runs over blocks rather than cards. Writing S_t for the path mass of count vector t at its block's entry layer,

$$S_t \;=\; \sum_{|n| \,=\, \text{entry layer}} F(n)\, B(n-t), \tag{7}$$

the three quantities of §5.1 are read straight off the tables:

$$\mu_{c,p} = \frac{1}{Z}\sum_t g_H^{(c\to p)}(t)\, S_t \qquad \text{(per-card ownership)} \tag{8}$$

$$\alpha(A) = \frac{S_{t(A)}}{Z} \qquad \text{(the declaration query)} \tag{9}$$

$$\tau(H,T) = \frac{1}{Z}\sum_{t\,:\,\text{supp}(t)\subseteq T} g_H(t)\, S_t \qquad \text{(does one team hold the half-suit?)} \tag{10}$$

All three are exact under the uniform prior; no independence assumption and no sampling enters. §B.3 gives the mechanics of both block kinds.

Exactness here is cheap. An exact rebuild costs 0.4 ms mean over 10^5 states, against 9.07 microseconds for the approximate path. That ratio is large but the absolute cost is small, and it is small enough that the result of §5.4 is not a budget story. The exact posterior is not too expensive to deploy. It is worse.

What is verified and what is merely unfalsified. Two exactness claims in this machinery are of different standing, and the distinction is stated here rather than buried in the appendix. The count law and the block enumeration are checked against a brute-force enumerator that factorises nothing (§B.6), and the agreement is at integer rather than rounding scale. The team-allocation search, by contrast, does not re-check the survival constraint of (2) on the allocation it returns; it has failed zero times, which is not the same as a proof. §B.7 lists that case and two others — a count routine that falls back to a greedy approximation for heterogeneous non-singleton masks, and an enumeration truncation that leaves a partial table which is then normalised as if complete — each with its reachability argument.

The approximate path. Six observers invoke the posterior after every public event, which puts it in the inner loop of every large experiment, so the engine also implements an approximation and that is what ships. The Fast path keeps the exact bookkeeping of (C1)–(C3) and the exact capacities (C4), fits them by Sinkhorn scaling of the support matrix, and interleaves a conditioning step for (C5) that reweights a certificate's cards by treating their entries as independent Bernoulli indicators before capacity fitting is re-run (§B.4). The independence assumption is false, because the capacity constraints that make (9) necessary also couple the cards inside a certificate. Every performance number in this report was produced by this path.

5.3 The policy prior

The posterior of (C1)–(C5) is exact with respect to the rules and is not the full Bayesian posterior over deals, which would additionally weight each deal x by the likelihood that the observed actions would have been chosen given x. That likelihood is playstyle-dependent: a player who has taken many turns and never asked in half-suit H is, under any plausible policy, less likely to hold cards of H, and the rules alone say nothing about this.

The engine implements a two-parameter family of such corrections as prior weights inside the same machinery,

$$w(c,p) \;=\; \exp\!\big(\theta\, a_{p,H(c)} - \phi\,(a_p - a_{p,H(c)})\big), \tag{11}$$

where $a_{p,H}$ counts p's public asks in half-suit H, a_p their total public asks, and $H(c)$ is the half-suit of card c. The weights replace the uniform initialisation of the Sinkhorn fit, so $\theta = \phi = 0$ recovers the policy-agnostic posterior of §5.1. Both parameters are fitted jointly with the rest of the policy (§9); the shipped values are $\theta = 0.37062$ and $\phi = 0.14525$. Deleting the prior from the shipped configuration, leaving every other coordinate at its fitted value, costs 4.60 points, interval [[2.63, 6.58]].

Two architectural facts follow, and they are what makes §5.4 possible. *First*, (11) is the only channel by which behaviour enters the belief; everything else in (C1)–(C5) is a rule. *Second*, the approximate path has that channel and the exact path structurally cannot: the block construction of §5.2 takes the deduction state alone and has no parameter through which an opponent model could reach it. The two paths are therefore not a fast approximation and its exact reference. They are two different posteriors over the same hidden state, one policy-agnostic and exact under a uniform deal prior, the other policy-aware and approximate.

5.4 The exact posterior is a worse predictor than the approximation

Both posteriors were scored as predictors, on the same states, by two measures that do not involve playing a game: predictive log loss over the unresolved cards of each state, and the realised hit rate of each posterior's own argmax ask — the ask the posterior itself rates most likely to succeed, scored against the truth. The probe is `fish v6probe -mode=belief` (§4.1), and the source of record is `research/v06/notes/R12-mechanism-trials.md`, §3.

The comparison carries no fitted weights on either side. It scores posteriors, not policies, so nothing in it depends on which posterior the ask weights were tuned against. That is why it can settle a question that a substitution experiment in play cannot.

The ordering is unambiguous and it is the wrong way round. The exact count law under a uniform deal prior scores 1.42246 nats per card and 47.94% argmax hit rate. The approximate path with only the certificate half of the prior deleted ($\theta = 0$) scores 1.39083 and 49.99%; with the prior deleted outright ($\theta = \phi = 0$, `research/v06/results/F2-belief-noprior.txt`) it scores 1.39339 and 49.14% — still ahead of the exact law on both measures. The shipped prior scores 1.38218 and 51.49%. The exact object is the worst predictor of the three, and the gap between the second and third rows is the policy prior alone: 2.16 points of argmax hit rate and 0.011 nats.

The same ordering appears on the full live candidate set rather than on the card marginals. Over the ask decisions of a FishBot v0.5 mirror, the approximate argmax converts 51.96% of the time and the exact argmax 48.36%, on identical states, with the two argmaxes disagreeing at 22.91% of decisions; hindsight converts 98.98%. Substituting the exact posterior into the shipped policy therefore does not merely fail to help. It replaces the better predictor with the worse one at every decision on which they differ.

Three consequences follow, and the note draws all three.

1. **"Exact" is exact under a uniform-deal prior, and that prior is wrong.** The exact count law is the worst predictor in Table 1. Part of the gap is (11): deleting the prior costs the approximation

Table 1: Each posterior scored as a predictor on identical states: mean predictive negative log likelihood over the unresolved cards, the probability each posterior assigns to its own argmax ask, and the realised hit rate of that ask. Rows are the exact count law under the uniform deal prior, the approximate path with the policy prior deleted, and the approximate path at the shipped prior, followed by the θ sweep. Lower NLL and higher hit rate are better. The unit is one card for the NLL column and one decision for the hit column.

Posterior	Cards	Mean NLL	Argmax p	Argmax hit
exact (uniform prior)	140,661	1.42246	0.4660	47.94
sinkhorn th=0.000 ph=0.122	140,661	1.39083	0.4896	49.99
sinkhorn th=0.200 ph=0.122	140,661	1.38663	0.4939	50.75
sinkhorn th=0.300 ph=0.122	140,661	1.38465	0.4972	51.06
sinkhorn th=0.445 ph=0.122	140,661	1.38218	0.5035	51.49
sinkhorn th=0.600 ph=0.122	140,661	1.38055	0.5113	51.59
sinkhorn th=0.800 ph=0.122	140,661	1.38055	0.5211	51.64
sinkhorn th=1.100 ph=0.122	140,661	1.38311	0.5336	51.64
sinkhorn th=1.500 ph=0.122	140,661	1.38758	0.5459	51.51
sinkhorn th=2.000 ph=0.122	140,661	1.39221	0.5555	51.40

about a point and a half of argmax hit rate. But it is not the whole gap — with both prior parameters deleted the approximation still predicts better than the exact law, so the remainder is the Sinkhorn fit's own maximum-entropy smoothing, which hedges better than an exact conditional distribution taken under a prior that is wrong. Exactness with respect to the rules is not exactness with respect to the game, because the observed actions carry information the rules do not encode. Calling the block posterior "the exact posterior" without that qualification — as the earlier reports in this corpus do — names it after a property it has only relative to an assumption nobody tested.

2. **The residual belief error is not an approximation error to be engineered away.** This settles the corpus's standing open question — whether a matched-budget refit of the ask weights under `belief=block` would recover the deficit the v0.4 substitution ablation measured — in the negative. The refit is not worth running, because the object it would fit is a strictly less informative posterior. The question stayed open for two studies because it was posed as a budget question, and it is not one: §5.2 shows the exact path is affordable, and this subsection shows it is worse anyway.
3. **The predictive optimum is not the playing optimum.** Raising θ improves prediction well past its fitted value: the sweep in Table 1 minimises NLL at $\theta \approx 0.60 - -0.80$, against the shipped 0.44458. In play the same change loses — 4.4 points of win rate at $\theta = 0.60$ and 7.1 at $\theta = 1.10$, at 1,200 games per cell. The ask weights were fitted at the shipped θ and do not transfer to a different one. Belief and policy are therefore not separable objects to be optimised in turn; any change to θ has to be refitted jointly with the weights that consume it, which is what the fit of §9 does.

The third consequence is the one that generalises beyond this game. A better posterior is not a better agent, and an inference component evaluated on predictive loss can be improved into a policy that plays worse. The two objectives coincide only when the downstream policy is optimal for whatever posterior it is handed, and no fitted linear policy is.

6 Diagnosis: what FishBot v0.5 does wrong, and what it does not

The v0.5 study closed with an unusually candid non-claim: removing a failure mode was worth almost nothing in win rate, and that was the result. This section asks the obvious follow-up question — where, then, is the remaining value — and answers it with four measurements, two of which are negative and are reported as findings rather than as caveats. The negative pair is load-bearing: between them they close off the two largest items on the design register this study inherited, and they are the reason

FishBot v0.6 spends its effort on the measurement apparatus rather than on the policy's feature set.

6.1 The instrument

Every claim below is produced by `fish v6probe`, a diagnostics subcommand added for this study, and is reproducible from a clean build by the commands in Appendix G. Three definitions fix what is being counted.

Definition 1 (Live candidate)**.** An ask (c, t) is *live* for an actor if it is legal and the actor cannot prove from its own hand and the public record that t does not hold c. This is the M1 predicate FishBot v0.5 already gates on; it involves no posterior and no policy prior, so a candidate set restricted to it cannot have been narrowed by a modelling error.

Definition 2 (Exact tie at the top)**.** A decision has an *exact tie at the top* when two or more live candidates receive scores that are equal to within 10^{-9} under the policy's own pre-search score, that is $\lambda\, w^{\top} f + \nu\, Q$ where Q is the one-ply expectimax over the value function. The gap and the spread are reported alongside so that a tie can be distinguished from a flat objective.

Definition 3 (Belief-symmetric tie)**.** A tie is *belief-symmetric* when the tied candidates additionally receive equal probability under the *exact* posterior of §5, not merely under the Sinkhorn fit the deployed policy runs. A belief-symmetric tie is a statement about the information set, not about the approximation.

6.2 More than half of all ask decisions end in a tie

Over 3,773 tied decisions in FishBot v0.5 mirror play, 54.74% of contested decisions — decisions with at least two live candidates, which is 98.77% of all of them — carry an exact tie at the top. The objective is not flat: the mean spread across the candidate set is 7.536 while the mean gap between the leaders is 1.537. The dimension is blind, not the surface.

The tie is structural, and it is structural in a specific place. 93.77% of ties are two *cards of one half-suit at one target*; only 4.59% are one card offered to two different targets. Three causes stack. Eighteen of the twenty ask features are computed per (half-suit, target) and the three per-card features are all functions of the same marginal; the Sinkhorn filter makes exchangeable cards numerically identical by construction; and `askExpectedValue` discards its `target` argument outright, so the expectimax half of the score is constant across targets. The last of these was the defect the v0.5 design register headlined. It is aimed at 4.59% of the tie mass.

6.3 Negative result: the ties are exchangeable, and therefore irreducible

The natural reading of §6.2 is that a subroutine FishBot v0.5 already runs is throwing away a large amount of value, and the register this study inherited priced that value at over three half-suits per game using a hindsight oracle. That reading is wrong, and the measurement that refutes it is simple: re-score the tied candidates under the exact count law of §5.4 and ask how many ties it separates.

tie-break rule	realised hit rate
enumeration order (whichever candidate was emitted first)	43.81%
argmax of the deployed Sinkhorn marginal, within the tie	43.81%
the shipped policy's actual pick (its chain/threat pass)	43.76%
argmax of the *exact* posterior, within the tie	43.81%
hindsight best within the tie	70.24%

The exact posterior separates 0.00% of the ties, and every rule realises the same hit rate to three significant figures. Two cards of one half-suit at one target, tied under the policy's score, are tied under the exact posterior as well. The 70.24% column prices clairvoyance, not headroom.

The scope of that result, and the experiment that fixes it. What the table measures is equality of the *marginal* probability that the target holds the card. It follows that no rule ranking the tied candidates by that marginal — the deployed one, the exact one, or any other — can beat enumeration order, and none does. An earlier draft of this section concluded from that that the ties are irreducible. They are not, and the four rows below are the correction. Each plays the same policy against FishBot v0.5, changing only how the tie group is resolved.

how the tie group is resolved	win rate against FishBot v0.5
enumeration order (the shipped rule)	50.00 by construction
uniformly at random, seeded from the public event stream	50% [47.1756, 52.8244]
by a one-deal rollout	49.5833% [45.9432, 53.2279]
by a twelve-deal rollout ensemble	**54.1667%** [50.5147, 57.7744]

Randomising the choice is worth *exactly* nothing — 50.00%, mean half-suits 4.500 to 4.500 — so this is not a story about a deterministic policy being predictable. One sampled deal is worth nothing either. Twelve are worth several points at the bank in the table, and pooled over every bank the configuration was measured on, 52.64% over 2,160 games with no bank below 52.08%.

The separating information is therefore real, and it is not the mere fact of varying a deterministic choice — the random row rules that out exactly. It is in the *joint* posterior. Two cards can be equally likely to sit with the target while being differently correlated with the rest of the deal — differently placed in the allocations that survive the certificates, differently useful to the half-suits the opponents are collecting — and every marginal, exact or not, integrates that structure away. An ensemble of whole sampled deals does not.

What is not resolved. Whether the search's advantage is *confined* to the tie group is a separate question and this study does not answer it. Restricting the search to the tie group gives 51.94, 50.69, 50.00% over 3 banks (mean 50.88%); guarding the tie group instead gives 49.8611%; the unrestricted search gives 52.64%. At 720 games a cell those intervals all overlap. What is established is the pair of negative controls and the pooled effect, not the attribution. §8 says so again where it matters.

Two corrections to the record follow, and are carried into §12.

1. **Enumeration order does not decide half of all asks.** FishBot v0.5's top-K chain and threat pass re-scores the leaders and moves the pick at 66.23% of ties, so array order settles roughly a fifth of contested decisions rather than more than half — and, per the table, settles them exactly as well as any alternative.
2. **The channel is the card dimension, not the target dimension.** The emphasis the v0.5 register placed on `(void)target;` is aimed at a twentieth of the tie mass.

The design consequence is stated here and used twice later. Because the tie is a property of the information set, a rollout ensemble drawn from that information set is symmetric over it too: the determinizations carry no signal about which of two exchangeable cards to name, only variance. The tie-break question is therefore not a search question, and §8 does not treat it as one.

6.4 Negative result: the FishBot v0.5 fit was sampling noise

The second negative result concerns the interpretation of FishBot v0.5's refit rather than its shipped configuration. Read from the fit trace, the incumbent objective moves over forty generations by less

than one generation's standard deviation, with an ordinary-least-squares slope whose t statistic is below two, and the gap between the best candidate and the incumbent tracks the expected maximum of a population of independent draws of exactly that noise.

The cause is visible in the sampler rather than in the objective. One scalar step size was applied to every coordinate of a vector whose coordinates have ranges spanning two and a half orders of magnitude, so the search was bang-bang on the bounded probability knobs — the worst of them clipped at 0.00% of generation-zero draws under the old setting — and effectively frozen on the unbounded weights. The per-coordinate step-size flag the interface advertised was parsed and discarded. §9 is the repair, and §11 is what a working optimiser then finds.

Taken with the independent observation that running the v0.5 *mechanisms* on v0.4's frozen parameter vector reproduces essentially the whole v0.5 result, the conclusion is that the v0.4→v0.5 transition was mechanical and that its parametric machinery never moved. That is not a small correction: it means the shipped vector this study starts from is v0.4's, and v0.4's was produced by a search carrying a separate aliasing defect of its own.

6.5 What is actually still leaking

With the tie channel removed from the register and the parametric channel re-opened, the residual loss channels are smaller than the inherited register suggested, and they are ranked here with the paired figure each was measured at.

- **Asks into a half-suit the actor's own team already owns outright.** 9.75% of all asks — guaranteed misses that donate the turn. M1 cannot see them: the enumerator only offers opponents as targets, and no single seat can *prove* a teammate holds a card. Pricing the posterior mass the team already carries is the graded substitute, and §7 reports that it is worth nothing at the resolution §10 establishes as necessary.
- **Declaration errors**, 2.07% of declarations in mirror play. The pre-gates that produce them are, as §4 reports, exact: 0 false negatives in 14,449,770 rejections. The error is in the allocation, not in the gate.
- **The move class M1 deletes.** A provably-dead ask at a chosen opponent is how a human blackballs, donates the turn deliberately, or pays for a signal. §8 shows that FishBot v0.5's score cannot price it even when it is offered, and that unbanning it wholesale trades a higher win rate for a policy that kills 10.00% of its games at the action limit.

None of these is large. That is the diagnosis: FishBot v0.5 sits on a plateau of its own policy class, and the leverage available to FishBot v0.6 is in the apparatus that measures and fits it, not in another feature.

7 FishBot v0.6: the mechanism set

FishBot v0.6 is implemented as a policy that *derives* from FishBot v0.5 rather than forking it, and this is not a packaging decision. With every FishBot v0.6 switch off and the v0.5 parameter vector restored, each override defers to the base class and the two policies are identical — checked, not asserted, by comparing the md5 of the complete diagnostic output of a 60-deal mirror run (PASS). Every ablation in §11 is therefore exact: there is no "FishBot v0.6 without mechanism X differs from FishBot v0.5 for unrelated reasons" confound of the kind that has cost this project two studies.

The mechanisms are grouped by what they change. Each carries the defect it was keyed to, its measured verdict, and — where the verdict is negative — what that verdict rules out for the next study. Four of the seven are reported as null, and they are reported here rather than buried in an appendix, because the register they close is the register a reader of the v0.5 study would otherwise work from.

7.1 Shipped: the apparatus

Three changes ship unconditionally because they are corrections, not strategies, and they make no strength claim of their own.

- **The exact posterior's table aliasing** (§4.4), which made every exact-belief mechanism unsound in multi-agent play and which no strength measurement could ever have caught, because the deployed policy never queried the affected object.
- **The repaired optimiser** (§9): per-coordinate step sizes, a paired per-deal objective, an explicit minimax-regret dispatch, and a recovery test the previous harness provably fails.
- **The commit gate** (§10): the pathology KPIs are evaluated before any strength number is collected, because two of the highest-scoring rows of the v0.5 ablation table are configurations that kill 14% of their games at the action limit.

7.2 Measured and null: the tie-break

The mechanism the inherited register ranked first was to resolve the exact ties of §6.2 with a signal the Sinkhorn filter cannot represent — specifically, the exact count law, which costs about 428.6 µs to build and is affordable at a decision. It was built, and §6.3 is its verdict: the exact posterior separates 0.00% of the ties, and every tie-break rule realises the same hit rate. The mechanism is retained as an *instrument* and not as a strategy, and the distinction is literal: the exact-posterior switch is not read by the ask rule and the shipped policy never builds the exact posterior. What survives is the diagnostic that measured the null.

What this rules out for the next study is broader than the mechanism. Any tie-break candidate must first be shown to separate the tied set *under the exact posterior*. Signals that cannot — per-card ask history, rank priors, teammate-derived terms — are re-descriptions of the same symmetric information and cannot beat enumeration order.

7.3 Measured and null: three ask terms

Three terms were added to the ask score, each fitted rather than hand-set, and each keyed to a measured channel.

- **Void creation**: the probability that a hit takes the target's *last* card of the half-suit. Ask legality requires the actor to hold a card of the half-suit, so voiding an opponent permanently removes their right to ask in it — a one-way asset that 41.65% of FishBot v0.5's successful asks create by accident while nothing in its score rewards creating one on purpose.
- **Team-ownership discount**: the posterior mass the actor's own team already carries on the asked card, which is the graded substitute for the proof M1 cannot have and is aimed at the 9.75% of asks that are guaranteed misses into a half-suit the team already owns.
- **The last-live-opponent split**, separating "empty one of three opponents" from "empty the last one and trigger the forced endgame".

All three are null as *mechanisms*. One consequence has to be stated because it makes the obvious ablation confounded: a non-zero extra weight is what selects the v0.6 scoring path, and the v0.6 scoring path is the one that does not run v0.5's chain and threat re-scoring. Switching the extra terms off therefore switches that pass back on. §11.8 reports the four-way ablation that separates them. The void term cleared a 95% paired interval at four hundred games per cell on one bank and returned zero at a thousand on two; the last-live term fires too rarely to change any output. The fit gives all three small coefficients and the ablation that removes them outright is in Table 13. §10 is where that pattern — significant at the corpus's budget, zero at this one — is turned into a protocol rather than an anecdote.

7.4 Measured and rejected: the deliberate miss

M1 deleted the move class every multi-step human tactic is built from. Re-admitting it raises the win rate and destroys the policy (§8.5); rationing it against the linear score is inert, because the linear score provably cannot price it. It is offered to the search instead, which is the only evaluator in the system that can, and it is the best-performing search configuration measured. §8 reports what the search as a whole is worth.

7.5 Shipped off: test-time search

§8 in full. Off, instrumented, and reported with the conditions under which its verdict would change.

7.6 Shipped: the fitted vector

What is left, once four mechanisms have been measured null and one rejected, is the parameter vector — and §6.4 is the reason that is not an anticlimax. FishBot v0.5's vector is v0.4's, because v0.5's forty-generation refit moved nothing, and v0.4's was produced by a search carrying an aliasing defect of its own. Nobody has ever fitted this policy class with an optimiser that works. §9 builds one and §11 reports what it finds.

7.7 Reported separately: the partner regimes

The evaluation harness built one policy object for all three seats of a team, so "FishBot v0.6 with two bot partners" and "FishBot v0.6 with two human-model partners" — the two regimes the project owner asked to see reported separately — could not be expressed at all. Per-seat specification is added and both regimes appear as separate columns in §11.2. The self-play row is never the headline.

8 Test-time search, and the statistic that decides whether it helps

Every prior FishBot chooses its ask by maximising a fitted score over the legal set. The obvious next step, and the one the project owner asked for, is to reason forward: to evaluate a candidate by what happens after it rather than by a static function of the position it is played from. This section builds that and measures it. It is the one mechanism in this study that beats a static rule — and getting there required correcting a statistic whose cost, 35.69 points, is the most transferable result in the paper.

8.1 Why determinization here is not double-dummy search

The v0.4 study eliminated perfect-information Monte Carlo on structural grounds and this study reconfirms them: under perfect information the team on turn takes every half-suit not dealt outright to its opponents, so a double-dummy evaluator scores every legal action alike and the average over determinizations cannot discriminate. Fish is a game about who knows what; a clairvoyant evaluator prices none of it.

The search built here determinizes the *deal* — which is the entire hidden state because every card movement in this game is public — but does not make anybody clairvoyant. Each of the six players in the continuation is reconstructed *at its own information set*: the public deduction state, which is common knowledge and is therefore one object rather than six, refined by the hand that determinization gives that seat. They miss asks, emit certificates and misdeclare in the proportions the real game does. This is the cooperative single-agent search of Lerer et al. [31], with the belief taken from an exactly enumerable posterior rather than a learnt one.

Table 2: Rollout-blueprint fidelity. Win rate against FishBot v0.5 and single-thread throughput. Reducing the Sinkhorn iteration count, which the corpus had not tried, is the cheap axis: it buys most of the speed of the independent-marginal blueprint at a small fraction of its strength cost.

Rollout blueprint	Win rate vs FishBot v0.5	95% CI	Games/s
`v05:belief=indep,topk=0`	12.00	[10.33, 13.75]	996.2
`v05:topk=0,souter=1,sinner=1`	46.08	[43.33, 48.83]	146.8
`v05:topk=0,souter=1,sinner=3`	46.75	[44.00, 49.50]	114.9
`v05:topk=0,souter=2,sinner=4`	51.42	[48.67, 54.17]	78.7
`v05:topk=0`	47.75	[44.92, 50.50]	30.7

The reconstruction is checked against ground truth rather than argued. Over 3,216 states and 103,644 card checks, the reconstructed information set differs from the seat's real one on 0.3338% of cards and is *wider* in 346 of 346 cases (narrower in 0), replicated at 2 seed banks: it is a strict under-approximation of what the seat knows, so no simulated player is ever credited with knowledge it does not have.

8.2 The search

At an ask decision the actor (i) builds the exact capacity dynamic program on its own deduction state and draws D deals from it, rejecting draws that violate an ask-legality certificate, so accepted draws are exact samples of the policy-agnostic posterior, and re-weighting them by the policy prior as an importance weight so that the sampler stays exact and the prior stays auditable; (ii) takes the K leading candidates under the blueprint score, extended through every candidate that ties the K-th, because a plain top-K would cut an exact tie arbitrarily and so reproduce the very defect the search is being asked about; (iii) seats six blueprint agents at their reconstructed information sets, applies the candidate, and plays the continuation out; (iv) and repeats every candidate on the *same* determinizations, so the comparison is paired.

Two design choices are forced by measurement rather than taste. The blueprint used inside the rollout is the deployed policy with its two-ply refinement disabled and its Sinkhorn iteration count reduced; Table 2 gives the cost/fidelity frontier, which the corpus had not swept. The cheapest blueprint available is 62× cheaper per event than the deployed policy and 38 points weaker, and a search whose continuation model is 38 points weak is searching a different game.

8.3 The result: the optimizer's curse is worth 35.69 points

The first working build of the search chose the candidate with the highest mean rollout return and scored 13.61% against FishBot v0.5. The control that isolates the cause is the same code, the same rollout blueprint, the same budget, the same seed bank and the same sample size, with the blueprint's preference made to dominate: it scores 49.31%. The machinery is correct and the *statistic* was wrong, and the gap between those two rows — 35.69 points — is the price of the argmax.

One caveat about that control, because it is the row the whole comparison rests on. The blueprint's preference dominates only where the blueprint *has* one: inside its own tie group the added term is identically zero and the deviation penalty is waived, so the control still lets the search choose among tied candidates. It scores at parity, which is a second, independent confirmation of §6.3: searching inside an exchangeable tie group changes nothing.

One determinization's return is the final half-suit differential, whose standard deviation is about two and a half half-suits, while genuine differences between the leading candidates are an order of magnitude smaller. An argmax over D such means therefore selects on residual noise and lands on

Table 3: The deviation-threshold ladder, six rotations, held-out bank. The first row is the control that forces the blueprint to decide. The range across the ladder is 40.28 points.

Configuration	Win rate vs FishBot v0.5	95% CI	n
`s1=1,det=8,cand=6,blend=1000000`	49.31	[45.83, 52.78]	720
`s1=1,det=8,cand=6,kappa=0`	13.61	[10.97, 16.39]	720
`s1=1,det=12,cand=4,kappa=0`	23.33	[20.42, 26.39]	720
`s1=1,det=12,cand=4,kappa=1`	42.78	[39.31, 46.25]	720
`s1=1,det=12,cand=4,kappa=2.5`	53.75	[50.42, 57.22]	720
`s1=1,det=12,cand=4,kappa=4`	53.89	[50.42, 57.36]	720
`s1=1,det=16,cand=6,kappa=2,maxq=26`	48.89	[45.14, 52.64]	720
`s1=1,det=16,cand=6,kappa=2,maxq=26,deadsearch=2`	50.28	[46.67, 53.89]	720

a candidate the blueprint had ranked below the top. Replacing the argmax with a paired lower-confidence-bound rule — deviate from the blueprint's own choice only when the improvement over it, on the same determinizations, clears κ standard errors of the paired difference — recovers the whole deficit monotonically in κ (Table 3).

This is a property of determinized evaluation in this game rather than of this implementation, and it is stated as the section's main contribution. Any search over a high-variance terminal return that selects by an unguarded argmax will pay it, and the price here — 40.28 points across the ladder — is larger than every mechanism the three prior studies fitted, combined.

8.4 Where the advantage lives

Guarded, the search stops losing and starts winning: at $\kappa = 2.5$ it takes 52.64% against FishBot v0.5 pooled over 3 banks and 2,160 games. The remaining question is whether that win is the kind a working search produces, and the deviation rate answers it — but only once the rate is decomposed. The falsifier the literature supplies is explicit: a healthy test-time search differs from its blueprint on a small percentage of decisions, and one that differs on tens of percent is mis-scaled rather than smart. At $\kappa = 2.5$ the search moves the action on 33.49% of the decisions it searches (Table 4), and that rate does not fall below about 31% however far κ is raised. Read naively this is an order of magnitude above the band the literature associates with a search that refines a good policy, and it was so read in an earlier draft. It is the wrong reading, and the reason is §6.3.

The deviation penalty is waived inside the blueprint's own tie group, because there the blueprint expresses no preference. That group is more than half of all decisions, so the search always re-chooses inside it, and those re-choices are neutral by construction — the candidates are exchangeable. The floor of 31% is that churn, not signal. Applying the same penalty inside the tie group collapses the rate to 2.70% at $\kappa = 2.5$ and 0.11% at $\kappa = 4$.

That decomposition also bears on where the win sits. Applying the penalty inside the tie group — so that the search deviates only on the 2.70% of decisions where it has evidence against the blueprint's stated preference — takes the win rate to 49.8611% [46.2194, 53.5043]: the advantage disappears. Forbidding deviation *outside* the tie group instead, so the search only ever re-chooses among candidates the blueprint cannot separate, takes it to 54.1667% [50.5147, 57.7744] — better than the unrestricted search.

What is established, and what is not. The pooled effect is established twice over. On FishBot v0.5's parameter vector the guarded search takes 52.64% against FishBot v0.5 over 2,160 games and 3 banks, none below 52.08%. On FishBot v0.6's own vector — so that the search is being asked to improve the best static policy this project has, not the previous one — it takes 52.08% against

Table 4: Search deviation rate against the deviation threshold. A healthy search moves a small percentage of decisions; this one either moves a third of them or none.

κ	Decisions searched (%)	Moved the action (%)	Games/s
0	98.58	65.67	0.173
1	98.51	41.51	0.165
2.5	98.51	33.49	0.144
4	98.32	31.40	0.144
6	98.32	31.59	0.144

FishBot v0.6 over 2,880 games, with 4 of 4 cells above parity and the worst at 50.83%. That second result is what makes FishBot v0.6-Search a configuration worth naming. So are the two negative controls of §6.3: randomising the tie group is worth exactly zero and one sampled deal is worth nothing, so what the ensemble is reading is joint information and not the mere fact of varying a deterministic choice.

What is *not* established is where inside the search the effect sits. Guarding the tie group gives 49.8611%; restricting the search to the tie group gives 51.94, 50.69, 50.00% across 3 banks; the unrestricted search gives 52.64%. At 720 games a cell those intervals overlap each other and the unrestricted figure, and an earlier draft of this section read a single favourable bank as an attribution. It is not one. Resolving it needs roughly an order of magnitude more games at a configuration that costs 0.144 games per second on one thread, against 303.4 for the deployed policy across all threads — which is the reason it is not resolved here.

8.5 The one move class that exists only under search

M1 restricts the candidate set to asks the actor cannot prove will miss. That fixed v0.4's two-question cycle, and it deleted the move class every multi-step human tactic is built out of: blackballing, choosing which opponent receives the turn, and paying for a signal with a safe ask are all a guaranteed miss at a *chosen* seat. Such an ask exists at 79.2% of FishBot v0.5's decisions and at two or more distinct chosen opponents at 50.1%.

Unbanning it wholesale is measured and rejected. With dead asks re-admitted and the ownership features gated by hit probability so that the $p = 0$ incentive behind v0.4's cycle cannot fire, the configuration scores 51.1667% [48.3393, 53.9866] against FishBot v0.5 — higher — and fails the commit gate outright: 35.85% of its asks are provably dead, its longest dead run is 365, and 10.00% of its games are killed by the action limit. This is the trap §10 exists to prevent, and it is the same trap the v0.5 ablation table contains.

Rationing it does not help either, and the reason is instructive: *the linear score cannot price a deliberate miss.* With the four ownership features zeroed on a dead candidate, the admission margin was swept at four settings and the output was bit-identical at every one, because the hit probability term is zero on such an ask by definition and every remaining term of the score is a penalty. Its value is the value of the position the donated turn produces, which no static feature of the ask computes.

The search can price it, and this is the clearest case in the study of a move class that exists only under search: the deliberate miss is offered to the rollouts as a candidate, one per distinct target, under an anti-repeat guard that is provably free. A dead (c, t) pair stays dead unless the card publicly moves to t, and that movement is observable, so forbidding the unmoved repeat forbids nothing that could have become live. This is exactly the distinction the blanket repetition guard of the v0.5 study missed, and why that guard cost six points: it also forbade *live* repeats, which are frequently the best ask on the board once a card has moved.

With the deliberate miss in the candidate set the endgame-restricted search scores 50.28%. It is the best search configuration measured and it is still not separated from the blueprint.

8.6 What ships

The search ships **off**, and the reason is cost and unresolved scope rather than a null result. On the ladder's best setting it is ahead of FishBot v0.5 with an interval that excludes parity, but it is three orders of magnitude slower than the deployed policy and it moves a third of all decisions — so what it is is not a refinement of the shipped policy but a distinct, far more expensive one whose evaluation this study could not complete at the resolution §10 demands of everything else. It is retained, switchable and fully instrumented. §13 states what would have to be measured before it could ship on.

9 Fitting

The v0.5 study ran 40 generations of a cross-entropy search over 34 coordinates and reported that the refit was worth nothing measurable. §6.4 shows that this was the correct reading of the trace and the wrong conclusion to draw from it: the search made no progress because the search was broken, in four separate and independently diagnosable ways. This section presents the four defects, the repair for each, and — for each — the measurement that shows the repair did what it was supposed to do rather than the intention that it should.

The four defects are stated together first, because three of them compound.

1. **One scalar step size for every coordinate.** A single $\sigma_0 = 0.6$ was applied to all 34 coordinates, whose admissible ranges span 0.10 to 40. At generation zero this put 93.4% of `declareMargin` proposals on a clamp bound while `valueWeight` moved 1.5% of its range: the optimiser was simultaneously bang-bang on the bounded probability knobs and frozen on the unbounded weight knobs. The command-line flag that would have fixed this, `-sigmaparams`, was parsed into a string that the rest of the program never referenced.
2. **An unpaired objective.** Candidates were compared on absolute win rate against a common seed bank. Common random numbers pair the *deal*; they do not pair the comparison, and the deal-level variance stayed in every candidate's score.
3. **An objective documented as a soft minimum that is a weighted mean.** Differentiating the soft minimum shows its gradient is a convex combination of the panel's win rates with softmax weights, so at the compiled temperature it ranks candidates by mean win rate minus a small multiple of the spread. On FishBot v0.4's shipped profile the maximum-to-minimum gradient weight ratio is 1.88 against the 1 a true mean would give and the ∞ a true minimum would give, and the "minimum" sits 11.2 points away from the actual minimum. Two different temperatures were in use in one pipeline, and neither was recorded in any trace; the correction that owes to the earlier record is in §12.2.
4. **The rotation defect.** The tuner computed each candidate's win rate as wins over `games * 2`, hard-coding its own two rotations into the denominator (§4.4). Fitting at six rotations, which the duplicate-block design of §10 otherwise recommends, would have reported a third of the true win rate for every candidate, silently.

Defects (i) and (ii) together explain the flat trace directly: an optimiser that cannot move most of its coordinates, scoring what it can move through an estimator that leaves the deal variance in, will produce a sequence of generations whose best candidate is an order statistic of noise. Defect (iii) means that what little signal there was pointed at the wrong objective. Defect (iv) never fired, because the tuner never ran at six rotations, and is included because it is a trap rather than a loss.

9.1 Per-coordinate step sizes

The repair is to derive each coordinate's step size from its own admissible range. The optimiser now takes a relative step $\sigma_d = \sigma_{\text{rel}} \cdot (\text{hi}_d - \text{lo}_d)$, with an explicit per-coordinate vector overriding it when one is supplied — which is what `-sigmaparams` was advertising and is now wired to. A coordinate's step is therefore a fixed fraction of what that coordinate can be, rather than a fixed number of units of whatever the coordinate happens to be measured in.

The check is not that the code changed. Every generation record in the run trace now carries the per-coordinate clip fraction — the fraction of that generation's population whose proposal for that coordinate landed on a clamp bound — so the pathology the previous harness had can be read off the artifact while a fit is running. At generation zero the worst coordinate of run A clips at 0.00%, against the 93.4% the v0.5 trace records for the same quantity. Run C, which fits the wider v0.6 vector at a smaller relative step, clips at 50.00%; that is reported as measured rather than as passed, and §C.3 prints the clip column per coordinate so a reader can see which coordinates it is.

9.2 Paired scoring on common random numbers

The incumbent is candidate zero of every generation and is played on exactly the same deals as every other candidate. The paired per-deal margin over it is therefore already available and costs nothing to compute, and the repair is simply to score each candidate by that margin rather than by its absolute win rate. The pairing now happens inside the objective rather than in post-processing, so the selection, the elite set and the recorded score are all the paired quantity.

The invariant that catches a mis-wired estimator is a policy scored against itself. Under a correct paired estimator, the per-deal margin of a configuration against its own incumbent is exactly 0.0000 on every deal, so the margin is exactly zero and the bootstrap interval has zero width — not a small number, and not an interval containing zero. This is asserted at startup. It is worth stating why so weak-looking a test is the right one: every plausible wiring error — pairing against the wrong candidate, pairing across generations, dividing by the wrong rotation count, resampling games instead of deals — turns that exact zero into something else, and none of them can be seen in a fit's output otherwise.

9.3 A minimax-regret objective

The soft minimum is retained as one option and is no longer the only one. The optimiser now dispatches explicitly over {`softmin`, `min`, `mean`, `regret`, `minimaxregret`}, the chosen objective is written into the run record rather than inferred from a compiled default, and the fits of record use `minimaxregret`: for each opponent on the panel the regret of a candidate is the best win rate any candidate in that generation achieved against that opponent minus this candidate's, and the objective is the negative of the worst such regret.

The reason for preferring regret to the minimum itself is that the panel cells are not equally hard, so a minimum over raw win rates is dominated by whichever opponent is hardest and carries almost no gradient from the rest. The reason for preferring it to the soft minimum is that the soft minimum was never the object it was named after. Two temperatures in one pipeline — one compiled into the tuner, one defaulted in the selection script, neither recorded — is itself a correction owed to the earlier record, and §12.2 makes it.

9.4 Commit-gate KPIs

No configuration in this study was committed on a win rate. The pathology instrument runs as a gate with a non-zero exit status, as step one of `engine/experiments_v06.sh` rather than as a report at the

end, and the identity controls of §4.4 run in the same step. The thresholds are 1 in number and the shipped configuration's measured value on each is: dead asks 0.01%, longest dead run 1, repeated asks 2.22%, wrong declarations 2.37%, asks inside a half-suit the actor's own team already owns outright 11.69%, and starved turns 31,321,233%.

A gate that has never failed has never been tested, so the battery runs a negative control: a configuration that the gate must reject. Unbanning the deliberate miss — allowing asks the actor can prove will fail, with the ownership features gated so that the zero-probability incentive cannot fire — scores 51.1667% against FishBot v0.5, interval [48.3393, 53.9866], which is higher than the shipped configuration and would be selected by any procedure ordered on win rate. The same configuration plays 35.85% of its asks dead, runs 365 dead asks without interruption at its worst, and reaches the action cap in 10.00% of games. The gate rejects it. §8.5 treats what that configuration is actually doing; the point here is narrower and is about ordering. A battery that collects strength first and pathology afterwards selects this build, and the v0.5 study's own ablation table contains two configurations of the same character.

9.5 Schedule, selection, and the winner's curse

Two fits of record were run. Run A starts from FishBot v0.5's vector and fits the 34 v0.5 coordinates over the panel v05+v03+lockout+withholder, at population 20, 150x2 deals by rotations per cell, relative step 0.050, for 30 generations from fitting seed 20260823. Run C starts from the v0.6 base and fits the wider 37-coordinate vector over the panel v05+v03+withholder+feint, at population 14, 200x2, relative step 0.040, for 14 generations from fitting seed 20260824. Both are paired, both use `minimaxregret`, and both write a header record as the first line of the trace carrying the base specification, the objective, the temperature, the pairing flag, the population and elite sizes, the generation count, the deals and rotations per cell, the seed, the relative step and the full per-coordinate step vector. That header is why §C can name the run behind the shipped vector: the v0.5 study had to recover its own fitting temperature by regular expression over a source file and a markdown document, because nothing in the trace recorded it.

The winner's curse. The per-generation best is a maximum over a population evaluated on shared seeds and is biased upwards; on the v0.5 trace the gap between the best and the incumbent was 0.0404, against 0.0447 predicted by the order statistic of 24 draws of the observed noise alone. The optimiser therefore re-scores both the distribution mean and the running best on a common guard bank and returns the higher. The guard is retained and it is honestly weak: the guard bank is derived from the same root seed as every generation bank, so it is disjoint from them but not independent of the fitting process. Selection is guarded, not held out. The held-out check is the evaluation protocol of §10, and every number in §11 comes from a bank the fit never touched.

Two acceptance tests. The first is the self-comparison of §9.2: a policy scored against itself returns a paired margin of exactly 0.0000 with a zero-width interval. It is an invariant and it passes by construction, which is the point — it fails loudly the moment the estimator is rewired.

The second is a recovery test on a known gap, and it is the one the previous harness could not pass. A deliberately handicapped base was constructed by zeroing the leading ask weight, the hit-probability coefficient, which is the single most important coordinate in the vector; that configuration scores 45.89% against FishBot v0.5. A 10-generation fit from that base with the repaired optimiser recovers to 48.33% against FishBot v0.5 and to 76.83% against FishBot v0.3. The v0.5 harness's own 40-generation trace on the same objective moved its incumbent at +0.00049 per generation with $t = 1.60$, which is no movement at all; a recovery of this size in 10 generations is outside anything

that harness produced in 40. This test is not part of `engine/experiments_v06.sh` and its command is given in §G.2 so that it can be re-run.

What the repaired optimiser was worth. It is the only thing in this study that produced a replicated effect. The mechanism trials of §7 are null: single-coordinate and single-feature perturbations of the v0.5 policy class are worth zero at the budget §10 shows is required to see them. The joint refit is not, and §11.3 reports it against the panel on two disjoint banks. The plateau this line of work has been sitting on is in the mechanisms, not in the parameters — but the parameters were never actually searched, because the searcher could not move.

10 Evaluation protocol

Fish has a single chance node — the deal — followed by a public transcript, and that structure permits an unusually strong variance-reduction design. This section fixes the experimental unit, the interval construction, the fitting/evaluation separation, and the metrics of §11.

It also fixes something the two preceding studies did not, and the omission cost this one five mechanisms. A protocol section is normally read as bookkeeping. In this study it is a result: §10.4 shows that the budget most of the corpus's published ablation rows were run at cannot separate an effect of the size those rows report from a seed draw, and that five separate mechanisms cleared a nominal 95% paired interval at that budget and returned exactly zero when re-run larger on two disjoint banks. Every claim in this report is paired and re-run on a second bank for that reason.

A reader should be able to tell, for every number in §11, whether it is paired or unpaired, which bank it came from, and whether that bank was used in fitting. The earlier reports could not support that reading, which is why several of their comparisons are corrected in §12.

10.1 Six-rotation duplicate blocks

Seats alternate between the two teams, so the cyclic group of six seat rotations of a fixed deal forms a balanced block: three seat shifts by two team orientations. The odd rotations exchange which team holds which hand-triple; the even rotations permute hands within a team. Playing all 6 rotations of every deal therefore has each policy hold every hand-triple of that deal exactly once, so the intrinsic advantage of the deal cancels from the comparison rather than merely being averaged down. A matchup reported over D deals is $6D$ games organised into D blocks of six. This is the duplicate-block convention of competitive play [66], strengthened: the competition design balances the team label, and the six-seat rotation additionally balances the assignment of individual hands within a team.

One guarantee of that convention does not survive, and it is stated here rather than in a footnote because it silently halves several denominators. In a *mirror* match — both teams running the same policy specification — the two team orientations of a deal produce byte-identical games, so each game is played twice and the effective sample is 2% of the nominal one. Rates are unaffected; denominators are not. Every table in §11 states which convention its n uses, and a mirror `match` at two rotations returns exactly 50% for that reason and not because the policies are balanced.

10.2 Deal-clustered bootstrap

The six rotations of one deal are one correlated cluster: they share the same card distribution and differ only in seat assignment. Resampling *games* would treat those six observations as independent and understate the variance. Every interval in this report therefore resamples *deals* — a cluster bootstrap of the per-deal win count for a single arm, and a paired bootstrap of the per-deal difference

for a comparison between two configurations played on the same deals — taking the 2.5% and 97.5% quantiles of 20,000 resamples, for 95% intervals. Both are implemented in `engine/src/arena.hpp`.

These are *percentile* bootstrap intervals with the deal as the resampling cluster, and they inherit the known coverage error of that construction. We keep it for two reasons. Clustering by deal is the correction that matters most in this design, because the within-deal correlation across the six rotations is large and ignoring it understates the variance by a substantial factor, whereas the percentile-versus-BCa distinction is second-order at these sample sizes. And we have not implemented the bias-corrected and accelerated or bootstrap-t corrections, so no claim of nominal coverage is made: intervals should be read as approximate, and a difference whose interval sits close to zero should not be treated as resolved. §10.4 is what that caution looks like when it is applied rather than merely stated.

10.3 Paired estimators

The headline comparison is a paired difference on common random numbers, and every ablation is paired against its own control. The pairing is what makes the design competitive with the estimator-side variance reduction developed for poker [67, 68] without needing an auxiliary value function: in this game the deal is the whole chance node, so replaying it cancels the dominant variance component directly. §9.2 gives the measured justification, and the same estimator is used in fitting and in evaluation so that a candidate is scored during the search by the quantity it will be judged on afterwards.

One quantity in this report is not available paired. Throughput (§11.12) is a property of one binary on one machine rather than of a matchup, so there is no common deal to pair on; it is reported as a single-arm figure measured under load and is a lower bound for that reason.

10.4 The resolution requirement

Five mechanisms in this study cleared a nominal 95% paired interval at 400–1,600 games per cell and returned exactly zero at 3,000 games per cell on two disjoint banks. Table 5 is the record.

Table 5: Five mechanisms that resolved at a small budget and did not replicate. Sign convention is the corpus's: δ = reference − variant, so a *negative* δ means the variant is better. Every entry is a paired per-deal margin with a deal-clustered percentile interval. The large-budget column gives two independent seed banks. Source: `research/v06/notes/R12-mechanism-trials.md`, §6.1.

Mechanism	Small-budget paired δ	Large-budget paired δ, two banks
`wVoid` at 1.5	-0.0292 [-0.0537, -0.0037], n=400	+0.0007 [-0.0213, +0.0227] and +0.0200 [-0.0017, +0.0413], n=1,000
`wVoid` at 3	-0.0313 [-0.0593, -0.0033], n=500	as above
delete the perseveration bonus	-0.0200 [-0.0356, -0.0044], n=800	-0.0012 [-0.0137, +0.0112] and +0.0035 [-0.0092, +0.0158], n=1,000
`declareMargin` more negative	+1.75 pp unpaired, n=1,200	+0.0133 [+0.0013, +0.0247] — the reference is better
delete the negative value-of-information term	-0.0053 [-0.0244, +0.0141], n=800	not separated

The planning rule that follows from this is arithmetic, not a convention. For an unpaired win-rate comparison near even the half-width of a nominal 95% interval is

$$1.96\sqrt{\frac{p(1-p)}{N}} \;\approx\; \frac{0.98}{\sqrt{N}} \quad \text{at } p \approx \tfrac{1}{2}, \tag{12}$$

that is, about $98/\sqrt{N}$ percentage points at N games. At $N = 400$ that is a half-width of nearly five points; at $N = 1{,}000$ it is about three. An effect of one to two points — the size every mechanism in Table 5 appeared to have — is therefore indistinguishable from a seed draw at the budget most of this corpus's published ablation rows were run at. Pairing buys back a large factor, but it buys it back from the deal variance and not from the requirement to check that the result is not the bank.

Two rules follow and both are enforced in this report. Every claim is *paired*: the variant and its control are played on the same deals and the interval is on the per-deal difference. And every claim is *re-run on a second, disjoint bank*, and reported as replicated only if it holds in sign and size on both. Where a result resolves on one bank and not the other, that is said, and the per-cell rows are given so a reader can see which half of the comparison carries it.

The general lesson is uncomfortable and is stated plainly in §13: at the budget this project used before v0.6, a mechanism that does nothing has a substantial chance of being published as a mechanism that does something, and several were.

10.5 Seed separation and what was held out

Fitting used base seeds 20260823 (run A) and 20260824 (run C). Every seed bank used from §11 onwards is disjoint from both by construction, and the disjointness is enforced by a registry of the 20260823, 20260824 reserved seeds that the battery checks, rather than by discipline. The evaluation banks are `90210`, `31337`, `515151`, `777001` and `424242` for the head-to-head; `515253` for the per-style panel; `606060` for the frozen-policy ablations; `717171` for calibration; `828282` for the rule dialects; `313131` for the partner regimes; `90210` again for the search ladder, the deviation-rate probe and the pre-gate audit; `31` for the pathology KPIs and the tie and belief probes; and `515253` with `6543210` for the exploitability probe's fitting and evaluation halves respectively. 5 of them are held out and 40 are fitting banks. Banks carried forward from the v0.5 study are named in §G.2 so that the corresponding rows are directly comparable.

The separation is narrower than "held out" usually implies, and the report scopes its claims accordingly. Deals are held out: no deal used in any reported experiment appears in any fitting bank. Opponent *identities* largely are not — the fitting panels of §9.5 include FishBot v0.5, FishBot v0.3 and two scripted archetypes that also appear in the evaluation panel — so the head-to-head and per-style results measure generalisation to new deals rather than to new strategic populations. The exceptions, which are held out in both senses, are named in §11.2.

10.6 Metrics

The primary metric is win rate, reported with its paired margin over the control. Sets differential — mean half-suits won out of nine — accompanies it, because a policy can win while converting a smaller fraction of its asks and this report contains a matchup in which that happens. Ask hit rate and declaration accuracy are diagnostic rather than primary: they describe the decision quality of one component and a policy can trade either against the other.

The pathology KPIs of §9.4 are gates, not metrics. They do not enter any comparison and no configuration is ranked on them; they decide only whether a configuration is eligible to be measured at all.

One standing rule governs how the opponent panel is reported: *never headline an aggregate over it.* The headline is the worst case and the minimax regret, with the full per-opponent profile printed alongside. A mean over a scripted panel is a weighted average whose weights are the panel's composition, and the panel's composition is a choice this study made. For the same reason rating systems are deliberately not used as a headline here [71, 72, 73, 70]: a single scalar over a fixed scripted panel

Table 6: Pathology KPIs in mirror play, 600 games per arm at seed 31. These gate every commit. The v0.4 column is the failure mode v0.5 removed; the FishBot v0.6 column must not reintroduce any of it.

KPI	FishBot v0.4	FishBot v0.5	FishBot v0.6
provably dead asks (% of asks)	39.04	0.01	0.01
longest dead run	286	1	1
games with a dead run ≥ 6 (%)	34.33	0.00	0.00
exact repeat asks (%)	40.03	2.61	2.22
declarations wrong (%)	10.44	2.07	2.37
ask hit rate (%)	34.25	55.63	53.29
events/game	143.60	96.72	94.59
games killed by the action limit (%)	0.00	0.00	0.00

is exactly the aggregate the rule forbids, and the transitivity it assumes is not established for this population. That is stated rather than the systems being omitted silently.

11 Results

Every number in this section comes from `engine/experiments_v06.sh`, whose artifacts are listed in Appendix G. The battery is ordered so that the commit gate runs before any strength measurement: a configuration that scores higher while failing the pathology KPIs is rejected, and §10 explains why that ordering is not a formality.

11.1 Controls

The identity control passes: FishBot v0.6 carrying v0.5's parameter vector with every mechanism switched off produces a diagnostic transcript identical to FishBot v0.5's under md5 (PASS). The verification suite passes with 0 soundness violations in 23,594,580 checks (PASS); no agent's deduced knowledge ever excludes the truth, and the deal-sampling, card-counting and block engines agree.

The block dynamic program's table-aliasing defect (§4.4) is repaired at the query level: 0 query mismatches, against 285 in 294 before the fix. Raw reads of the shared pool still differ (175) and are documented rather than fixed, because the pool is what keeps the memory footprint of six concurrent observers bounded.

The declaration pre-gate audit, which was dead code for FishBot v0.5 because the option was parsed only in the v0.4 branch, now runs: over 5,472,906 declaration opportunities and 14,449,770 gate rejections it reports 0 false negatives (PASS). FishBot v0.5's pre-gates are exact, where v0.4's were not.

11.2 The commit gate

11.3 Head to head

Over the 5 dedicated held-out banks of E3, six rotations each and 9,000 games in total, FishBot v0.6 wins 51.03% against FishBot v0.5 with a per-bank range of 50.11% to 51.83%, and 50.86% against FishBot v0.4 (50.28% to 52.22%).

Those cells are not enough to answer the question, and an earlier draft of this paper drew the wrong conclusion from them. At 1,800 games a cell the project's own planning rule gives a half-width of about 2.3 points, which cannot resolve a one-point difference. Pooling every FishBot v0.6-versus-FishBot v0.5 cell in the battery — including the default-dialect row of the rules

Table 7: High-power head-to-head against FishBot v0.5, 3,000 deals × 6 rotations per bank. Every bank is above parity.

Seed bank	Win rate (%)	Games
90210	51.12	18,000
31337	50.66	18,000
515151	50.48	18,000
777001	50.95	18,000
424242	51.16	18,000
828282	50.29	18,000
606060	51.54	18,000

Table 8: Held-out head-to-head against FishBot v0.5, one row per seed bank, six rotations.

Win rate (%)	95% cluster CI	Mean sets FishBot v0.6	Mean sets FishBot v0.5
51.44	[49.11, 53.78]	4.560	4.440
50.44	[48.11, 52.78]	4.526	4.474
51.33	[49.06, 53.67]	4.541	4.459
50.11	[47.78, 52.44]	4.523	4.477
51.83	[49.56, 54.11]	4.546	4.454

experiment and the FishBot v0.5 column of the paired ablation panel, both of which fall below parity — gives 50.53% with a naive z of 1.13: a null. The draft reported that null.

It is a power artefact. Re-run at **18,000 games a bank** across 7 disjoint banks — the same five, plus the two that had gone the other way — FishBot v0.6 is above parity on **all 7**, worst bank 50.29%, and pooled **50.89% over 126,000 games** [50.61, 51.16]. The two contrary cells return *above* parity at the larger sample. Against FishBot v0.4, 51.11% over 36,000 games with 3 of 3 banks above parity [50.59, 51.63].

Head to head, FishBot v0.6 beats FishBot v0.5 by 50.89% − 50 = 0.89 points. The effect is real and it is small; §10.4 is the protocol that made the difference between reading it as a null and measuring it.

This is reported before anything else so that no reader has to hunt for it. What FishBot v0.6 changes is not here.

11.4 Exploitability

The v0.5 study named the absence of any exploitability measurement as the single largest hole in its own evaluation. It is measured here. A responder is fitted from the same policy family, with the repaired optimiser and the frozen target as its entire opponent panel, and is then re-measured on a fresh seed bank because the win rate it reached during fitting is a maximum over a population on shared seeds and is upward biased.

The control reproduces: against FishBot v0.4 the response reaches 50.69% [49.06, 52.31] over 3,600 games, whose interval overlaps the published figure. Against FishBot v0.5 it reaches 50.31% [48.75, 51.89]. Against FishBot v0.6 it reaches **48.36%** [46.75, 49.97] — an interval that excludes parity. The same optimiser, the same budget, the same policy family and the same fresh evaluation bank fail to reach parity against FishBot v0.6 and succeed against both of its predecessors.

The reading has to be careful, and the v0.4 findings document got it wrong in exactly this place (§12.1). The number is a *lower bound within the searched class*: a response that fails to reach parity

Table 9: Local best-response probe. The response win rate is what a fitted exploiter achieved, so *lower is better* for the frozen policy. The FishBot v0.4 row is the positive control: the v0.4 study published 51.19% [49.67, 52.72] for the same experiment.

Frozen target	Response win rate	95% CI	Games
v04	50.69	[49.06, 52.31]	3,600
v05	50.31	[48.75, 51.89]	3,600
v06	48.36	[46.75, 49.97]	3,600

Table 10: Per-opponent win rate over the full style set, pooled over 3 disjoint seed banks (515253, 90210, 424242), six rotations per cell. The headline is the *minimum*, not the mean. Minimax regret is measured against the best of the three arms on each opponent and is therefore relative; §13 states what it is carried by.

Opponent	FishBot v0.6	FishBot v0.5	FishBot v0.4	games
v05	51.00	50.00	48.39	4,800
v04	50.00	50.94	50.00	4,800
v03	76.06	73.71	73.33	4,800
v02	80.00	81.72	83.06	1,800
lockout	78.25	79.50	77.94	4,800
detective	76.71	75.88	77.28	4,800
diversifier	95.44	94.11	92.89	1,800
hunter	98.44	97.67	97.67	1,800
bluffer	99.83	99.89	99.94	1,800
random	100.00	100.00	100.00	1,800
silent	83.87	81.96	78.89	4,800
feint	53.37	53.44	51.11	4,800
withholder	79.15	70.54	67.50	4,800
worst case	**50.00**	**50.00**	**48.39**	
worst opponent	v04	v05	v05	
mean (descriptive)	78.63	77.64	76.77	
minimax regret	**3.06**	**8.60**	**11.65**	

does not prove the target is hard to exploit, only that this search did not find the exploit. What the three rows support is the *comparison* — identical class, identical budget, identical protocol — and on that comparison FishBot v0.6 is the least exploitable of the three. It is also the first exploitability number this project has for FishBot v0.5, whose own study named its absence as the largest hole in its evaluation.

11.5 The per-style profile and the worst case

Pooled over 3 banks, FishBot v0.6's worst cell is 50.00% (against v04) and FishBot v0.5's is 50.00% (its own mirror, which is 50% by construction), while minimax regret over the style set is 3.06 against 8.60 and 11.65. FishBot v0.6 is ahead of FishBot v0.5 on 7 of the thirteen styles, behind on 5 and equal on 1, with a largest single-style loss of 1.72 points.

The single large cell is the deception archetype the project owner brought here from live play: the **withholder**, which holds cards of a half-suit it was asked for and then declines to ask back in it. FishBot v0.6 takes 79.15% against FishBot v0.5's 70.54%, a gain of 8.60 points over 4,800 games, and it replicates at all 3 banks. It was in the fitting panel, and §13 says what that means for how the

Table 11: Each posterior scored as a predictor on identical states: mean predictive log loss over the unresolved cards, and the realised hit rate of that posterior's own argmax ask. The exact posterior is the worst row.

Posterior	Cards	Mean NLL	Argmax p	Argmax hit
exact (uniform prior)	140,661	1.42246	0.4660	47.94
sinkhorn th=0.000 ph=0.122	140,661	1.39083	0.4896	49.99
sinkhorn th=0.200 ph=0.122	140,661	1.38663	0.4939	50.75
sinkhorn th=0.300 ph=0.122	140,661	1.38465	0.4972	51.06
sinkhorn th=0.445 ph=0.122	140,661	1.38218	0.5035	51.49
sinkhorn th=0.600 ph=0.122	140,661	1.38055	0.5113	51.59
sinkhorn th=0.800 ph=0.122	140,661	1.38055	0.5211	51.64
sinkhorn th=1.100 ph=0.122	140,661	1.38311	0.5336	51.64
sinkhorn th=1.500 ph=0.122	140,661	1.38758	0.5459	51.51
sinkhorn th=2.000 ph=0.122	140,661	1.39221	0.5555	51.40

Table 12: FishBot v0.6 against FishBot v0.5 over a seven-opponent panel, 800–1,000 games per cell, on 3 disjoint seed banks. A positive margin means FishBot v0.6 is ahead.

Seed bank	FishBot v0.6 − FishBot v0.5	95% paired CI
606060	+2.69	[+0.98, +4.35]
515253	+2.67	[+1.24, +4.11]
90210	+1.13	[-0.31, +2.56]

number should be read.

11.6 Belief as a predictor

11.7 The paired panel comparison

The comparison the rest of this section rests on. Every variant plays the same deals against the same panel; the statistic is the per-deal margin, bootstrapped resampling deals as clusters.

All 3 banks are positive, 2 of them with an interval excluding zero, mean 2.16 points and smallest 1.13. This is the separated result; the head-to-head of §11.3 is not.

11.8 Paired ablations

11.9 The search ladder

Tables 3 and 4 in §8. The result is that the statistic, not the machinery, decides whether a determinized search helps, and that once the statistic is corrected the search has nothing left to add.

11.10 Partner regimes

FishBot v0.6's advantage over FishBot v0.5 is 2.25 points when its partners are copies of itself and between −0.8 and +1.4 when they are not, with no mixed row separated from zero at this sample size. A refit whose objective is scored in self-play buys a policy that coordinates with copies of itself, and that coordination is exactly what a human partner does not supply. This is the project owner's standing decision measured rather than asserted, and §13 carries it forward as the sharpest limitation on everything in this section.

Table 13: Paired mechanism ablations against FishBot v0.6: every variant plays the same deals against the same panel, and the per-deal margin is bootstrapped resampling deals as clusters. A positive delta means FishBot v0.6 beats the variant.

Variant	Pooled	FishBot v0.6 − variant	95% paired CI
`v05`	65.73	-2.69	[-4.35, -0.98]
`v06:legacy=1`	65.73	-2.69	[-4.35, -0.98]
`v06:xf=0`	68.54	+0.12	[-1.67, +1.92]
`v06:s1=1,det=16,cand=6,kappa=2.0,maxq=26`	69.77	+1.35	[-0.04, +2.79]

Table 14: Win rate against three FishBot v0.5 opponents with the two remaining seats of the acting team played by the named policy, 800 games per cell at seed 313131. The self-play row is not the headline, and the three rows below it are why.

Partners	FishBot v0.6	FishBot v0.5
self	52.25	50.00
v03	34.50	33.12
detective	33.50	33.62
withholder	34.75	35.50

11.11 Rule dialects

The result is reported in Appendix A: the ordering of FishBot v0.6 and FishBot v0.5 does not change under the legacy dialect, with out-of-turn declaration disabled, or with cardless declaration disabled.

11.12 Throughput

12 Corrections to the record

This project's convention is that corrections are stated as corrections, in their own section, and not silently patched into the artifacts they correct. The v0.5 study opened a twelve-entry register against the v0.4 record; four of its entries never reached print, and this study adds five of its own. All are listed here with the measurement that forces them.

12.1 Entries carried forward from the v0.5 register that were never published

C7 — "an undeclared half-suit scores nothing". The v0.4 paper justifies its second forcing horizon on the ground that an unclaimed half-suit is worth zero. It is not, in this harness: residue at the action cap is adjudicated by physical majority, and on a wrong post-horizon declaration the declaring team holds a mean 4.08 of the six cards. The v0.4 paper contradicts itself on this point — its own rules section states the dialect correctly. Neither paper printed the correction. It is printed here, and it matters beyond bookkeeping: the argument it supports is the one v0.5 removed as mechanism M8, so the mechanism was right for a reason its own justification got wrong.

C9 — restarting the willingness ladder. The v0.4 paper argues that restarting the forced-endgame ladder lets early, safer declarations resolve cards and sharpen the later ones. The premise is never satisfied: essentially every forced endgame in this dialect contains exactly 1 live half-suit (566 occasions at seed 31), so there are no later declarations to sharpen.

The fitted adversary that "fails to reach 50%". The v0.4 findings document reports that a policy fitted specifically against v0.4 fails to reach parity with it. The same study's own local best-response

Table 15: Games per second, self-play, all threads.

Policy	Games/s
`v04`	147.9
`v05`	276.3
`v06`	303.4
`v05:topk=0`	286.0
`v05:belief=indep,topk=0`	9289.0

table records the response achieving 51.19% with a confidence interval of $[49.67, 52.72]$, and the paper states the null result correctly. The findings document turns a null result into a win. This is the single most consequential unretracted claim in the corpus, because it is the only place anything resembling an exploitability number has ever been reported.

"Modelling declaration timing as optimal stopping is worth measurable win rate". The same document. The same study's own ablation puts the value-based stopping rule at $+0.12$ points with an interval of $[-1.23, +1.47]$ against a fixed threshold, and the v0.4 specification says in terms that the ablation does not resolve a benefit.

12.2 Corrections this study adds

The tie channel is not a defect, and it is not where it was said to be. The v0.5 design register, and the recon that opened this study, priced the exact-tie channel as the largest available loss and located it in the target dimension. Both halves are wrong. 93.77% of ties are two *cards* of one half-suit at one target, not one card at two targets; and the exact posterior separates 0.00% of them, so no tie-break rule can beat enumeration order — measured, all four rules realise the same hit rate to three significant figures. The associated "+3.16 half-suits per game of headroom" is a hindsight bound and prices clairvoyance. See §6.3.

Enumeration order decides a fifth of contested decisions, not more than half. FishBot v0.5's top-K chain and threat pass re-scores the leaders and moves the pick at 66.23% of ties. The frequently-quoted figure conflates the tie rate with the rate at which array order is the deciding rule.

"Exact inference costs six points" is a statement about the prior, not about exactness. Scored as predictors on identical states, the exact posterior under a uniform-deal prior is the *worst* of the three inference paths measured, at 1.42246 nats and 47.94% argmax hit rate against the deployed approximation's 1.38218 and 51.49%. The policy prior is the entire difference. The matched-budget refit under the exact belief that three studies have listed as the outstanding experiment is therefore not worth running: the object it would fit is a less informative posterior. See §5.4.

The v0.5 fit resolved nothing, and the shipped vector is v0.4's. §6.4. The consequence for the reader of the v0.5 paper is that its fitting section describes a procedure that did not, at the budget and step size used, move the policy; and the consequence for this study is that a working optimiser had to be built before any mechanism could be evaluated.

The exact reference engine was unsound in multi-agent play. The block dynamic program parked every instance's tables in a shared per-thread pool, so any second construction on the same thread silently repointed the first instance's tables: 285 mismatches in 294 checks. Every previously published number computed with the exact belief in a configuration where more than one seat holds such an object is affected. The deployed approximate-belief numbers are not, because that path never queries the object — which is exactly why the defect survived three studies. See §4.4.

12.3 A note on comparability

Two of the corrections above change how earlier numbers should be read rather than what they are, and one changes the numbers themselves. Readers comparing across versions should note that this study's evidence standard is not the corpus's: every claim here is paired, and every claim that survived is re-run on a second, disjoint seed bank at a per-cell budget three to seven times larger than the published ablation rows of the v0.3, v0.4 and v0.5 studies. §10.3 gives the measurements that forced that change, including five mechanisms in this study that cleared a 95% interval at the corpus's budget and returned zero at this one.

13 Threats to validity

The head-to-head margin is small, and it took 126,000 games to see it. FishBot v0.6 beats FishBot v0.5 by about nine tenths of a point (§11.3); at the 1,800-game cells this project normally uses, the same comparison reads as a null and two cells read as a loss. Its claim is on the axis this project has always said it cares about — the paired panel comparison and the worst case across playstyles — together with a set of corrections and negative results that close off most of the design register it inherited.

The worst-case improvement is carried by one column. Minimax regret over the thirteen-style set is 3.06 for FishBot v0.6 against 8.60 for FishBot v0.5, but delete the withholder column and the three arms converge. The withholder was in the fitting panel of both fits. The correct reading is that the refit bought robustness against a deception archetype it was shown, which is what a minimax-regret objective over a panel containing that archetype is supposed to do, and not that FishBot v0.6 is uniformly more robust.

The null results are null *at this budget and against this panel.* Five mechanisms are reported as worth zero. Each was measured paired, at a thousand deals per cell, on two disjoint banks, against a six- or seven-opponent panel. That resolves effects of roughly a point; it does not resolve effects of a quarter of a point, and a policy improvement of a quarter of a point per mechanism, compounded across a dozen mechanisms, would be a real policy improvement. What this study establishes is that none of the mechanisms on the inherited register is large, not that the sum of many small ones is zero.

The search verdict is conditional on two quantities. Test-time search is reported as not paying, and the conclusion rests on the variance of the terminal return and the fidelity of the rollout blueprint. Both are movable. A shaped or truncated return with a well-fitted leaf evaluator would cut the variance; a blueprint closer to the deployed policy at the same cost would cut the bias. This study measured the frontier for the second (Table 2) and did not build the first, because the leaf evaluator available is algebraically close to a rescaling of the hit probability. A study that rebuilds the value function first should re-open the search question; §8.6 says so, and the machinery is retained switched off for exactly that reason.

Exchangeability is established for the tied set, not for the whole candidate set. The finding of §6.3 is that candidates the policy's score cannot separate are also candidates the *exact posterior* cannot separate. It is not a claim that the exact posterior is uninformative in general — it disagrees with the deployed approximation about the best ask at 22.91% of decisions — only that where the deployed score ties, the information ties too.

FishBot v0.6's advantage is a self-play advantage and does not transfer to a change of partner. This is the sharpest limitation in the paper and it is measured, not conjectured. Against three FishBot v0.5 opponents, FishBot v0.6 beats FishBot v0.5 by 2.25 points when its two partners are copies of itself, by 1.4 when they are FishBot v0.3, by −0.1 when they are the detective and by −0.8 when they are the withholder (Table 14; 800 games a cell, none of the three mixed rows separated from zero). A refit whose objective is scored in self-play buys a policy that coordinates with copies of itself, and that coordination is exactly what a human partner does not supply. The project owner's standing decision — never headline the self-play configuration as though it were the one a human will meet — is why this table exists, and it is why the paper does not claim FishBot v0.6 is a better partner for a person.

The opponent panel is a modelling choice and the deception archetypes are hand-built. The withholder, the silent and the feint are stylised implementations of manoeuvres the project owner reported from live play; they are not humans. A gain against the withholder is evidence about robustness to a class of behaviour, not a measurement of play against a person. The partner-regime columns have the same limitation from the other side: a "human-model partner" is a weaker or deceptive bot, not a human.

Exploitability is a lower bound within a searched class. The best-response probe fits a responder from the same policy family with the same optimiser. A stronger response outside that family would give a larger number. A response that fails to reach parity therefore proves nothing, which is precisely the reading error §12.1 corrects in the v0.4 record.

The engine correction changes previously published numbers. Any earlier result computed with the exact block belief in a configuration where more than one seat held such an object was affected by the table-aliasing defect. This study re-runs what it quotes; it does not re-run the whole of the v0.4 and v0.5 studies, and it does not claim to know the size of the effect on their published tables.

Fitting seeds are disjoint from evaluation banks, but the panel is not. The refit's panel contains opponents that also appear in the evaluation panel. The head-to-head and per-style results are therefore in-panel for those cells and held-out only in the seed sense. The cells that are held out in both senses — the styles absent from the fitting panel — are marked as such in Table 10, and they are the ones a sceptical reader should weigh.

Termination remains empirical. As in v0.5, no configuration shipped here hits the action cap in any measured run, but that is a measurement rather than a theorem. The structural backstop is M1, whose gate is a hard deduction and cannot be wrong; what is not proved is that M1 plus the stopping rule terminates on every reachable position.

14 Conclusion

Three FishBot studies in a row have ended with a policy that is not measurably stronger than the one before it. This one explains why.

The design register the project had accumulated — a blind target dimension, a large tie channel, an exact posterior held back only by its cost, a two-ply search that could be deepened — described a policy with several points of accessible headroom. Measured at a resolution the project had not previously used, none of it is there. The ties are exchangeable and therefore irreducible. The exact posterior is a worse predictor than the approximation, because exactness is bought by discarding the

policy prior. The search is dominated by the variance of its own return, to the tune of 35.69 points, and once that is corrected it has nothing left to say. Four further mechanisms, each keyed to a measured channel and each fitted rather than hand-set, are worth zero.

What was actually broken was the apparatus. The optimiser could not move a policy: one step size for thirty-four coordinates spanning two and a half orders of magnitude, an unpaired objective, and an objective documented as a worst case that was a weighted mean. The evaluation could not resolve a point: five mechanisms in this study passed a 95% interval at the budget the corpus uses and failed at three times it. And the exact inference engine, the one component nobody doubted, was returning another observer's tables whenever two observers held one.

FishBot v0.6 fixes those, and the fixed optimiser then finds what forty generations of the old one could not: a policy that is ahead of FishBot v0.5 on the paired panel comparison at all three banks it was measured on, and that gains nine points against the deception archetype the project owner brought here from live play. Head to head it wins 50.89% over 126,000 games with every one of 7 banks above parity — nine tenths of a point, and invisible at the per-cell budget this project has always used, which is why an earlier draft of this work reported it as a null. That is a smaller headline than a new mechanism would have been. It is also the only one the evidence supports, and stating it that way is the point.

Three things the next study should take from this. The tie-break question is closed — any candidate signal must first be shown to separate the tied set under the exact posterior, and none of the obvious ones can. The search question is open but conditional: it should be re-opened only after the leaf evaluator is rebuilt, because the present one is algebraically close to a rescaling of the hit probability and cannot support a depth-limited search. And the evidence standard is now the binding constraint on everything else: at a thousand deals per cell, paired, on two banks, this policy class looks flat, and a study that wants to find a quarter-point mechanism will need a per-decision objective rather than a per-game one.

References

[1] FishLab Research Project, "FishBot v0.3: A Count-Conditioned, Empirically Tuned Policy for Six-Player Canadian Fish," project technical report, 2026. Project repository: https://github.com/dylann29/fish-optimization

[2] D. Nguyen, FishLab Research Project, "FishBot v0.4: Observer-Conditioned Deal Inference and Value-Based Declaration in Six-Player Canadian Fish," project technical report, 2026. Project repository: https://github.com/dylann29/fish-optimization

[3] D. Nguyen, FishLab Research Project, "FishBot v0.5: A Self-Inflicted Deadlock, Its Measurement, and the Repair of the Ask Policy That Caused It," project technical report, 2026. Project repository: https://github.com/dylann29/fish-optimization

[4] J. McLeod, "Literature," *Pagat*, 2006–2025, last updated 1 July 2026. https://www.pagat.com/quartet/literature.html

[5] M. Develin, "Canadian Fish," chapter 9 of *Card Games*, author's online manual. https://web.archive.org/web/20170720075756/http://bantha.org/~develin/cardgames.html#ch9

[6] Deposit Genius, "Literature Game Strategy (Canadian Fish)," online article. https://depositgenius.com/literature-strategy-canadian-fish/

[7] Canadian Fish Project, "Strategy notes," 2026.
https://canadian-fish.vercel.app/strategy

[8] Wikipedia, "Literature (card game)," online encyclopedia entry.
https://en.wikipedia.org/wiki/Literature_(card_game)

[9] N. Somani, "literature: card game implementation and learning bot," GitHub repository, MIT licence, Python, 2018–.
https://github.com/neelsomani/literature

[10] N. Somani, "literature-server," GitHub repository, with a live deployment at https://literature.neelsomani.com/
https://github.com/neelsomani/literature-server

[11] "Literature: Fish Card Game" (`com.cards.game.literature`), Google Play store listing, closed source.
https://play.google.com/store/apps/details?id=com.cards.game.literature

[12] D. Zha, K.-H. Lai, S. Huang, Y. Cao, K. Reddy, J. Vargas, R. Wei, J. Guo, and X. Hu, "RLCard: A Toolkit for Reinforcement Learning in Card Games," arXiv:1910.04376, 2019.

[13] M. Goadrich, A. Morenville, and É. Piette, "Valet: A Standardized Testbed of Traditional Imperfect-Information Card Games," arXiv:2603.03252, 2026. 21 games in the RECYCLE description language; whether any of them is a team game is not stated on the abstract page and the full text was not fetched.
https://arxiv.org/abs/2603.03252

[14] D. Levy, "The Million Pound Bridge Program," *Heuristic Programming in Artificial Intelligence: The First Computer Olympiad*, Ellis Horwood, 1989.

[15] M. L. Ginsberg, "GIB: Imperfect Information in a Computationally Challenging Game," *Journal of Artificial Intelligence Research* 14:303–358, 2001.

[16] I. Frank and D. Basin, "Search in Games with Incomplete Information: A Case Study Using Bridge Card Play," *Artificial Intelligence* 100(1–2):87–123, 1998.

[17] I. Frank, D. Basin, and H. Matsubara, "Finding Optimal Strategies for Imperfect Information Games," *AAAI-98*, pp. 500–507, 1998.

[18] J. Long, N. R. Sturtevant, M. Buro, and T. Furtak, "Understanding the Success of Perfect Information Monte Carlo Sampling in Game Tree Search," *AAAI-10*, pp. 134–140, 2010.

[19] P. I. Cowling, E. J. Powley, and D. Whitehouse, "Information Set Monte Carlo Tree Search," *IEEE Transactions on Computational Intelligence and AI in Games* 4(2):120–143, 2012.

[20] J. Goodman, "Re-determinizing Information Set Monte Carlo Tree Search in Hanabi," arXiv:1902.06075, 2019.

[21] T. Cazenave and V. Ventos, "The $\alpha\mu$ Search Algorithm for the Game of Bridge," arXiv:1911.07960, 2019.
https://arxiv.org/abs/1911.07960

[22] S. Edelkamp, “Knowledge-Based Paranoia Search in Trick-Taking,” arXiv:2104.05423, 2021. Skat; over 1,000 points on the extended Seeger scale. The runtime is not stated on the abstract page, so no compute cost is attributed to this method in this report.
https://arxiv.org/abs/2104.05423

[23] M. Buro, J. R. Long, T. Furtak, and N. R. Sturtevant, “Improving State Evaluation, Inference, and Search in Trick-Based Card Games,” *IJCAI-09*, pp. 1407–1413, 2009.

[24] C. Solinas, D. Rebstock, and M. Buro, “Improving Search with Supervised Learning in Trick-Based Card Games,” *AAAI-19*, 2019; arXiv:1903.09604.

[25] D. Rebstock, C. Solinas, M. Buro, and N. R. Sturtevant, “Policy Based Inference in Trick-Taking Card Games,” *IEEE Conference on Games 2019*, 2019; arXiv:1905.10911.

[26] C. Solinas, D. Rebstock, N. R. Sturtevant, and M. Buro, “History Filtering in Imperfect Information Games: Algorithms and Complexity,” *NeurIPS 2023*, 2023; arXiv:2311.14651.

[27] N. Brown, T. Sandholm, and B. Amos, “Depth-Limited Solving for Imperfect-Information Games,” *NeurIPS 2018*, 2018; arXiv:1805.08195. Master-level heads-up no-limit hold’em on a 4-core CPU and 16 GB, via multi-valued states. The soundness argument is for two-player zero-sum games and is not inherited by the team game studied here.
https://arxiv.org/abs/1805.08195

[28] B. H. Zhang and T. Sandholm, “Subgame Solving without Common Knowledge,” *NeurIPS 2021*, 2021; arXiv:2106.06068.
https://proceedings.neurips.cc/paper/2021/hash/c96c08f8bb7960e11a1239352a479053-Abstract.html

[29] B. H. Zhang, L. Carminati, F. Cacciamani, G. Farina, P. Olivieri, N. Gatti, and T. Sandholm, “Subgame Solving in Adversarial Team Games,” *NeurIPS 2022*, 2022. Cited from the proceedings abstract page; the mirrored PDF did not decode.
https://proceedings.neurips.cc/paper_files/paper/2022/hash/aa5f5e6eb6f613ec412f1d948dfa21a5-Abstract-Conference.html

[30] S. Sokota, G. Farina, D. J. Wu, H. Hu, K. A. Wang, J. Z. Kolter, and N. Brown, “The Update-Equivalence Framework for Decision-Time Planning,” *ICLR 2024*, 2024; arXiv:2304.13138. Mirror-descent search at two orders of magnitude less search time than public-belief-state search in Hanabi.
https://openreview.net/forum?id=JXGph215fL

[31] A. Lerer, H. Hu, J. Foerster, and N. Brown, “Improving Policies via Search in Cooperative Partially Observable Games,” *AAAI 2020*, 2020; arXiv:1912.02318. SPARTA; Hanabi 24.08 to 24.61 at test time.
https://ojs.aaai.org/index.php/AAAI/article/view/6208

[32] H. Hu, D. J. Wu, A. Lerer, J. Foerster, and N. Brown, “Learned Belief Search: Efficiently Improving Policies in Partially Observable Settings,” arXiv:2106.09086, 2021.
https://arxiv.org/abs/2106.09086

[33] A. Fickinger, H. Hu, B. Amos, S. Russell, and N. Brown, “Scalable Online Planning via Reinforcement Learning Fine-Tuning,” *NeurIPS 2021*, 2021; arXiv:2109.15316.
https://arxiv.org/abs/2109.15316

[34] D. Milec, O. Kubíček, and V. Lisý, "Continual Depth-limited Responses for Computing Counter-strategies in Sequential Games," arXiv:2112.12594, 2021. Robust-response methods do not compose with limited-look-ahead solving off the shelf.
https://arxiv.org/abs/2112.12594

[35] D. Milec, V. Kovařík, and V. Lisý, "Adapting Beyond the Depth Limit: Counter Strategies in Large Imperfect Information Games," arXiv:2501.10464, 2025.
https://arxiv.org/abs/2501.10464

[36] O. Kubíček and V. Lisý, "Look-ahead Reasoning with a Learned Model in Imperfect Information Games," arXiv:2510.05048, 2025; ICLR 2026 per the arXiv PDF header.
https://arxiv.org/abs/2510.05048

[37] O. Kubíček, V. Lisý, and T. Sandholm, "Equilibrium Refinements Improve Subgame Solving in Imperfect-Information Games," arXiv:2601.17131, 2026.
https://arxiv.org/abs/2601.17131

[38] N. Brown, A. Bakhtin, A. Lerer, and Q. Gong, "Combining Deep Reinforcement Learning and Search for Imperfect-Information Games," *NeurIPS 2020*, 2020; arXiv:2007.13544.
https://arxiv.org/abs/2007.13544

[39] M. Schmid, M. Moravčík, N. Burch, R. Kadlec, J. Davidson, K. Waugh, N. Bard, F. Timbers, M. Lanctot, G. Z. Holland, E. Davoodi, A. Christianson, and M. Bowling, "Student of Games: A Unified Learning Algorithm for Both Perfect and Imperfect Information Games," *Science Advances* 9(46), 2023; arXiv:2112.03178. Circulated 2021–2022 as "Player of Games."

[40] M. Moravčík, M. Schmid, N. Burch, V. Lisý, D. Morrill, N. Bard, T. Davis, K. Waugh, M. Johanson, and M. Bowling, "DeepStack: Expert-Level Artificial Intelligence in Heads-Up No-Limit Poker," *Science* 356(6337):508–513, 2017.

[41] V. Lisý and M. Bowling, "Equilibrium Approximation Quality of Current No-Limit Poker Bots," arXiv:1612.07547, 2017. Local Best Response; every ACPC 2016 entrant is more than 3,180 mBB/h exploitable.
https://arxiv.org/abs/1612.07547

[42] N. Bard, J. N. Foerster, S. Chandar, N. Burch, M. Lanctot, H. F. Song, E. Parisotto, V. Dumoulin, S. Moitra, E. Hughes, I. Dunning, S. Mourad, H. Larochelle, M. G. Bellemare, and M. Bowling, "The Hanabi Challenge: A New Frontier for AI Research," *Artificial Intelligence* 280, 2020; arXiv:1902.00506.

[43] J. N. Foerster, H. F. Song, E. Hughes, N. Burch, I. Dunning, S. Whiteson, M. Botvinick, and M. Bowling, "Bayesian Action Decoder for Deep Multi-Agent Reinforcement Learning," *ICML 2019*, 2019; arXiv:1811.01458.
https://arxiv.org/abs/1811.01458

[44] H. Hu and J. N. Foerster, "Simplified Action Decoder for Deep Multi-Agent Reinforcement Learning," *ICLR 2020*, 2020; arXiv:1912.02288.

[45] H. Hu, A. Lerer, A. Peysakhovich, and J. N. Foerster, "'Other-Play' for Zero-Shot Coordination," *ICML 2020*, 2020; arXiv:2003.02979.

[46] H. Hu, A. Lerer, B. Cui, L. Pineda, N. Brown, and J. Foerster, "Off-Belief Learning," *ICML 2021*, 2021; arXiv:2103.04000.
https://arxiv.org/abs/2103.04000

[47] S. Sokota, E. Lockhart, F. Timbers, E. Davoodi, R. D'Orazio, N. Burch, M. Schmid, M. Bowling, and M. Lanctot, "Solving Common-Payoff Games with Approximate Policy Iteration," *AAAI 2021*, 2021; arXiv:2101.04237.
https://arxiv.org/abs/2101.04237

[48] H. Hu, S. Sokota, D. J. Wu, A. Bakhtin, A. Lupu, B. Cui, and J. N. Foerster, "Self-Explaining Deviations for Coordination," *NeurIPS 2022*, 2022; arXiv:2207.12322. IMPROVISED; the first algorithmic Hanabi finesse plays.
https://proceedings.neurips.cc/paper_files/paper/2022/hash/faa6276ea12d7afeb3e42b210c86f688-Abstract-Conference.html

[49] C. Cox, J. De Silva, P. DeOrsey, F. H. J. Kenter, T. Retter, and J. Tobin, "How to Make the Perfect Fireworks Display: Two Strategies for Hanabi," *Mathematics Magazine* 88(5):323–336, 2015.

[50] Y. Tian, Q. Gong, and T. Jiang, "Joint Policy Search for Multi-Agent Collaboration with Imperfect Information," *NeurIPS 2020*, 2020; arXiv:2008.06495. Contract Bridge: +0.63 IMPs/board against WBridge5.
https://arxiv.org/abs/2008.06495

[51] A. P. Jacob, D. J. Wu, G. Farina, A. Lerer, H. Hu, A. Bakhtin, J. Andreas, and N. Brown, "Modeling Strong and Human-Like Gameplay with KL-Regularized Search," *ICML 2022*, 2022; arXiv:2112.07544.
https://arxiv.org/abs/2112.07544

[52] H. Hu, D. J. Wu, A. Lerer, J. Foerster, and N. Brown, "Human-AI Coordination via Human-Regularized Search and Learning," arXiv:2210.05125, 2022.
https://arxiv.org/abs/2210.05125

[53] Meta Fundamental AI Research Diplomacy Team (FAIR), A. Bakhtin, N. Brown, E. Dinan, G. Farina, C. Flaherty, D. Fried, A. Goff, J. Gray, H. Hu, and others, "Human-Level Play in the Game of Diplomacy by Combining Language Models with Strategic Reasoning," *Science* 378(6624):1067–1074, 2022.
https://www.science.org/doi/10.1126/science.ade9097

[54] T. Dizdarević and others, "Ad-Hoc Human-AI Coordination Challenge," *ICML 2025*, 2025; arXiv:2506.21490.
https://proceedings.mlr.press/v267/dizdarevic25a.html

[55] A. Nayyar, A. Mahajan, and D. Teneketzis, "Decentralized Stochastic Control with Partial History Sharing: A Common Information Approach," *IEEE Transactions on Automatic Control* 58(7):1644–1658, 2013; arXiv:1209.1695.
https://arxiv.org/abs/1209.1695

[56] A. Celli and N. Gatti, "Computational Results for Extensive-Form Adversarial Team Games," *AAAI 2018*, 2018; arXiv:1711.06930.

[57] G. Farina, A. Celli, N. Gatti, and T. Sandholm, "Connecting Optimal Ex-Ante Collusion in Teams to Extensive-Form Correlation: Faster Algorithms and Positive Complexity Results," *ICML 2021*, PMLR 139:3164–3173, 2021.

[58] B. H. Zhang, G. Farina, A. Celli, and T. Sandholm, "Team Belief DAG: Generalizing the Sequence Form to Team Games for Fast Computation of Correlated Team Max-Min Equilibria via Regret Minimization," *ICML 2023*, 2023; arXiv:2202.00789.

[59] L. Carminati, F. Cacciamani, M. Ciccone, and N. Gatti, “A Marriage between Adversarial Team Games and 2-Player Games: Enabling Abstractions, No-Regret Learning and Subgame Solving,” *ICML 2022*, 2022; arXiv:2206.09161.

[60] L. Carminati, B. H. Zhang, G. Farina, N. Gatti, and T. Sandholm, “Hidden-Role Games: Equilibrium Concepts and Computation,” *ACM Conference on Economics and Computation (EC) 2024*, 2024; arXiv:2308.16017.
https://arxiv.org/abs/2308.16017

[61] H. J. Ryser, *Combinatorial Mathematics*, Carus Mathematical Monographs 14, Mathematical Association of America, 1963.

[62] D. G. Glynn, “The Permanent of a Square Matrix,” *European Journal of Combinatorics* 31(7):1887–1891, 2010.

[63] M. Jerrum, A. Sinclair, and E. Vigoda, “A Polynomial-Time Approximation Algorithm for the Permanent of a Matrix with Non-Negative Entries,” *Journal of the ACM* 51(4):671–697, 2004.

[64] N. Linial, A. Samorodnitsky, and A. Wigderson, “A Deterministic Strongly Polynomial Algorithm for Matrix Scaling and Approximate Permanents,” *Combinatorica* 20(4):545–568, 2000.

[65] Y. Chen, P. Diaconis, S. P. Holmes, and J. S. Liu, “Sequential Monte Carlo Methods for Statistical Analysis of Tables,” *Journal of the American Statistical Association* 100(469):109–120, 2005.

[66] N. Bard, J. Hawkin, J. Rubin, and M. Zinkevich, “The Annual Computer Poker Competition,” *AI Magazine* 34(2):112–118, 2013; and the competition rules describing duplicate matches and common seeds.

[67] M. Zinkevich, M. Bowling, N. Bard, M. Kan, and D. Billings, “Optimal Unbiased Estimators for Evaluating Agent Performance,” *AAAI-06*, pp. 573–578, 2006.

[68] N. Burch, M. Schmid, M. Moravčík, D. Morrill, and M. Bowling, “AIVAT: A New Variance Reduction Technique for Agent Evaluation in Imperfect Information Games,” *AAAI-18*, 2018; arXiv:1612.06915.

[69] K. Waugh, D. Schnizlein, M. Bowling, and D. Szafron, “Abstraction Pathologies in Extensive Games,” *AAMAS 2009*, pp. 781–788, 2009.

[70] D. Balduzzi, K. Tuyls, J. Pérolat, and T. Graepel, “Re-evaluating Evaluation,” *NeurIPS 2018*, 2018; arXiv:1806.02643.

[71] R. Coulom, “Whole-History Rating: A Bayesian Rating System for Players of Time-Varying Strength,” *Computers and Games 2008*, 2008.

[72] R. Herbrich, T. Minka, and T. Graepel, “TrueSkill™: A Bayesian Skill Rating System,” *Advances in Neural Information Processing Systems* 19, 2006.

[73] M. Van den Bergh, “Comments on Normalized Elo,” Fishtest technical note; and Stockfish contributors, “Statistical Methods and Algorithms in Fishtest.”
https://cantate.be/Fishtest/normalized_elo_practical.pdf

[74] N. Kelidari, M. Haghi, and M. Salmani, “A Gold-Standard Study of What Makes a Lightweight Game-Playing Agent Strong,” arXiv:2607.06854, 2026. Gin Rummy and Leduc; concludes that the binding constraint is information, not capacity.
https://arxiv.org/abs/2607.06854

A The rules dialect in full

The dialect is unchanged from the v0.4 and v0.5 studies, so this appendix is a summary and a delta rather than a re-derivation. Table 16 is the index: it lists every consequential choice against the engine switch that changes it. The v0.4 report's Appendix A states each choice at the length required to re-implement it, including the ask-legality predicate, the deal and shuffle, declaration resolution, the arbitration orders and the scoring rule [2]; that treatment is accurate except on the two clauses of §A.3. The two files that carry the definitions are `engine/src/fish.hpp` (deck, indexing, the `Rules` record, the legality predicate) and `engine/src/game.hpp` (the driver: declaration rounds, turn handling, the forced endgame, the action cap).

Table 16: The rules dialect used for every experiment in this report unless stated otherwise, and the engine switch that changes each choice. The switches are command-line flags of the `fish` driver; the underlying fields are members of `Rules` in `engine/src/fish.hpp`. Rows marked "modelling choice" are not fixed by any published statement of the rules. Unchanged from the v0.4 and v0.5 studies.

Choice	Value used	Switch
Deck composition	54 cards in nine half-suits (low and high band of each suit, plus the four eights and two jokers); nine cards per player	`--sets=8` gives the 48-card, eight-half-suit form
Ask legality	Live half-suit; asker holds another card of it and not the named card; target is an opponent and still holds cards	—
Turn transfer	A successful ask retains the turn; a miss passes it to the target	—
Out-of-turn declarations	Permitted at any moment, including during an opponent's turn	`outOfTurnDeclare` (`--no-out-of-turn`)
Cardless players may declare	Permitted	`cardlessMayDeclare` (`--no-cardless-declare`)
Incorrect declaration	Awards the half-suit to the opposing team	—
Declaration aftermath	The six cards leave play and the turn returns to the player who held it	—
Cardless turn-holder	Chooses which live teammate receives the turn, from its own belief (a modelling choice, and more restrictive than the rules of record; §2.1)	—
Forced endgame (whole team cardless)	The live team declares every remaining half-suit, sharing only willingness, over an eight-level ladder $\{0.995, 0.98, 0.95, 0.90, 0.80, 0.65, 0.50, \text{best guess}\}$	`--legacy` gives the two-level ladder $\{0.38, \text{best guess}\}$
Simultaneous declarations	Lowest seat index declares first (a modelling choice; cost measured in §2.1)	`declArbitration` (`--arb=low|high|turn`)
Action cap	400 asks (360 under the v0.3 dialect), any residue adjudicated neutrally by majority physical holding, ties to the holder of the lowest card	`--maxasks`
Conventions	No prearranged signalling conventions; team communication is limited to the willingness bit described above	—

A.1 Clause table and what changed

Nothing in Table 16 changed between FishBot v0.4, FishBot v0.5 and FishBot v0.6. The engine repairs of §4.4 are inference and bookkeeping defects, not rule changes, and none of them alters a game's outcome given the same actions. Two entries in the table are worth reading against the v0.4

appendix rather than in place of it. The cardless-turn-holder row is narrower than the rules of record permit, and the row for simultaneous declarations is a choice the rules do not make at all; both are quantified in §2.1, and both measure at approximately zero, which is why neither is a FishBot v0.6 mechanism.

Dialect robustness is reported rather than assumed: §11.11 replays the primary head-to-head result under 4 perturbations of this table over 1,500 deals at seed 828282, spanning 0.47%.

A.2 Clauses the FishBot v0.6 mechanisms depend on

Four clauses carry weight in this report, in the sense that a mechanism of §7 or a measurement of §5 would be different under a different choice. They are stated here in full so that the dependence is auditable.

Residue is adjudicated by physical majority. The driver counts asks and stops the game at `maxAsks`, 400 under the dialect studied. Awarding the unresolved residue to either side would bias the outcome towards whichever policy stalls more readily, so each remaining half-suit is instead awarded to the team physically holding the majority of its six cards, ties going to the team holding the lowest-indexed card of that half-suit. Two consequences matter. An unclaimed half-suit is worth its majority holding rather than nothing, which is the premise §A.3 corrects; and the cap is a backstop against unbounded play rather than part of the rules, so termination is an empirical property of the fitted policies and is reported as one, at 0.00% incidence.

The cardless-player rule. A seat with no cards leaves the asking cycle: it cannot ask, and it is an illegal target, because ask legality requires the target to hold cards. It remains in the game in two ways. It can be named in a teammate's declaration as the holder of a card — impossible while it is genuinely empty, so its emptiness constrains the allocations a teammate may name, which is a hard constraint the exact count law of §5.2 consumes. And, with `cardlessMayDeclare` enabled, it may itself declare. The turn can reach a cardless seat only immediately after a declaration removed its last cards, and that seat then chooses which live teammate receives it.

An opponent void is permanent. Cards move in exactly one direction: from a target to an asker, and only on a successful ask. Asking in half-suit h requires the asker to hold another card of h. A seat holding no card of h therefore can neither ask in h nor receive a card of h, and no sequence of legal actions restores it. Voids are monotone, they are public, and a void created against an opponent removes that opponent's right to ask in the half-suit for the rest of the game. This is a rules-level fact rather than a tactical one, and it is what the offensive-void term of §7.1 prices.

The forced endgame. When one team holds no cards at all, no legal ask remains for either side and the live team must declare every remaining half-suit. Teammates may negotiate openly but may exchange nothing except willingness to declare, so the driver implements the selection as a threshold sweep over the eight-level ladder of Table 16 rather than as a comparison of confidences: for each threshold in descending order the driver scans the live team's seats over the remaining half-suits and asks each whether it is willing to declare at confidence at least that threshold; the first seat that says yes declares, the ladder restarts, and the sweep repeats until every half-suit is resolved. Declarations here resolve exactly as elsewhere, including the possibility of misdeclaring and handing a half-suit to the cardless team, and are recorded separately in the event history. If every seat refuses at every level, the remainder is awarded by physical majority as above.

A.3 Two clauses corrected

The v0.4 appendix is accurate on every clause but two. Both errors are stated in that report and repeated in the source; neither was retracted by the v0.5 paper, which measured them and did not print the result. They are restated correctly here so that a reader who reaches this appendix first is not misled, and §12.1 gives them as corrections with the file and line of the text being corrected.

An undeclared half-suit does not score nothing. The v0.4 report's declaration section justifies the unconditional second stage of its forcing rule — declare the best candidate whatever its estimated probability, past a second horizon — on the ground that an undeclared half-suit scores nothing. Inside this harness it does not. The action cap calls the residue adjudicator described above, which awards each unresolved half-suit by physical majority, and on a wrong post-horizon declaration the declaring team holds a mean 4.08 of the six cards, which is the majority, which is what adjudication would have given it. The stage does not convert nothing into a coin flip; it converts a probable award into a certain concession. The premise is true only of a dialect that awards nothing at the cap, and the v0.4 study changed the harness away from that dialect after its parameter vector was frozen, without updating the justification. The v0.4 rules section states the dialect correctly, so that report contradicts itself within two sections.

Restarting the ladder sharpens nothing in observed play. The v0.4 appendix states that a consequence of restarting the willingness ladder is that early, safer declarations resolve cards and thereby sharpen the allocations available for the later ones. The reasoning is correct and the situation does not arise. Over every occasion on which a team went cardless with live half-suits remaining — 566 of them at seed 31 — the number of live half-suits at that moment was 1 in every case, so there is nothing to order and every ordering counterfactual is identical to the non-ordering one by construction. The structure is not impossible: a cardless team requires only that the other team hold a multiple of six cards. It does not occur because these policies cash locked half-suits promptly. A policy made more patient about declaring would begin to produce multi-half-suit forced endgames, at which point the restart would start doing the work the v0.4 appendix credits it with, and the machinery should be built alongside the patience change rather than after it.

B Inference: derivations and implementation notes

This appendix supplies the derivations §5 states without proof and the implementation detail a reimplementer needs. It also carries the exactness audit in full, because the headline result of §5 is a comparison against an object claimed to be exact, and a claim of that kind is worth exactly what its audit is worth.

The source files are `engine/src/belief.hpp` (constraint bookkeeping and the approximate path), `engine/src/blockdp.hpp` (the exact count law), and `engine/src/oracle.hpp` (the brute-force enumerator). Notation is that of §5.1.

B.1 The constraint system

The observer's state is a `Knowledge` object: a resolved-owner array, a per-card surviving-support bitmask M_c, the public hand counts, per-seat and per-half-suit ask and miss tallies, and a vector of live ask-legality certificates. (C1)–(C3) are maintained incrementally as bit operations on the support masks, and a support that collapses to a single seat resolves the card, which in turn removes any certificate the resolution satisfies.

(C4) is derived rather than stored. Capacities are recomputed from the public hand counts by (1) after every event, and a seat with no spare capacity is removed from the support of every unresolved card, which can cascade. This is the only place the two constraint families interact, and it is why a declaration — correct or incorrect — needs no special handling anywhere in the system. A declaration removes six cards from play and changes the public counts of the seats that held them; the six cards leave U with their holders known, the half-suit leaves $\mathcal{H}$, and each h_p in (1) drops by the number of those cards that seat held. The same identity that absorbs a successful ask absorbs a declaration. An incorrect declaration is not a special case either: it reveals the true holders of all six cards, which is strictly more information than a correct one gives the opposing team, and it arrives through the same channel.

(C5) is stored as a list. Each certificate is a pair — the asking seat and the bitmask D of (2) — and a certificate whose card set collapses to a single card resolves that card immediately rather than being stored. Two properties of the store matter downstream. First, every certificate is confined to one half-suit, which is what §B.3 exploits. Second, the store admits duplicates: a repeated ask in the same half-suit under the same support produces an identical certificate, and `onEvent` appended it without checking. The de-duplication repair of §4.4 is opt-in for a reason that belongs in this appendix rather than in the main text: the approximate conditioning step of §B.4 applies each stored certificate once, so two copies of one certificate condition twice and the duplicate is not inert. De-duplicating is therefore a change of posterior, not a change of representation, and it is off by default so that the FishBot v0.4 and FishBot v0.5 numbers this report compares against are unchanged.

B.2 The capacity-vector count law

The layer identity. Order the unresolved cards $c_1, \dots, c_{|U|}$ and let q be the capacity vector. A partial assignment of the first j cards leaves a vector n of unused capacities, and because every one of those cards consumes exactly one unit of capacity from exactly one seat,

$$\sum_p n_p \;=\; \sum_p q_p - j \;=\; |U| - j,$$

so j is recoverable from n and the layer index need not be stored. That is what collapses the natural $|U|$-layer table into two single arrays. Each is laid out as one array of size $\prod_p (q_p + 1)$, addressed by the mixed-radix offset $\mathrm{off}(n) = \sum_p n_p s_p$ with strides $s_p = \prod_{r<p}(q_r + 1)$, and a companion index sorts the reachable states by $\sum_p n_p$ so that a sweep visits one layer at a time. Count vectors are packed one byte per seat into a single `uint64_t`, which makes the dominance test $n \geq t$ that guards a block transition one branch-free word operation.

The bound. The observer's own cards are resolved by (C1), so $q_i = 0$ and the table is a product over the five other seats. Also $|U| \leq 45$: at the deal the 54 cards lie in nine live half-suits, the observer holds nine and those are resolved, and thereafter a card leaves U when its holder is revealed or its half-suit is declared and never re-enters. Subject to $\sum_{p \neq i} q_p = |U|$, the product $\prod_{p \neq i}(q_p + 1)$ is maximised at equal capacities, by the arithmetic–geometric mean inequality, so

$$\prod_{p \neq i}(q_p + 1) \;\leq\; \Big(\frac{\sum_{p \neq i}(q_p + 1)}{5}\Big)^5 \;=\; \Big(\frac{|U|}{5} + 1\Big)^5 \;\leq\; 10^5. \tag{13}$$

Each state has at most six outgoing transitions, so a full forward–backward pass costs at most 6×10^5 multiply–adds regardless of the position and the two arrays together occupy under two megabytes. The implementation refuses to build above a fixed state cap and reports the failure to the caller rather than degrading silently.

The per-card marginal. The marginal is the contraction of the two tables at card j,

$$\mu_{c_j,p} \;=\; \frac{\mathbf{1}[\,p \in M_{c_j}\,]}{Z} \sum_{\substack{n\,:\,|n|=|U|-j+1 \\ n_p \geq 1}} F_{j-1}(n)\; B_j(n-e_p), \tag{14}$$

which is exact: every consistent assignment is counted once, by splitting it at card j into a prefix counted by F_{j-1} and a suffix counted by B_j. Both tables hold integer-valued counts in double precision, so the arithmetic is exact while the counts stay below 2^{53}, which (13) guarantees on every reachable state.

The direct sampler. The backward table doubles as a sampler. Having placed $c_1, \ldots, c_{j-1}$ and arrived at remaining capacities n, card c_j goes to seat p with probability proportional to $B_j(n - e_p)$, restricted to $p \in M_{c_j}$ with $n_p \geq 1$. A seat receives positive weight only when the suffix can still be completed, so every prefix is extendable and no draw is discarded: the procedure is rejection-free for (C1)–(C4) and uniform over the assignments satisfying them, because the weights are path counts. When a live (C5) certificate is present the sampler is unchanged and the completed assignment is tested against the literal disjunction (2), drawing again on failure, so the combined procedure is exact but not rejection-free. The sampler is off the deployed decision path and is used as ground truth in the cross-checks of §B.6.

B.3 Half-suit block enumeration

U is partitioned by half-suit and each group is classified.

Card group.
No live certificate in that half-suit. The group's cards are expanded one at a time by (3)–(4), branching factor $|M_c| \leq 5$.

Block group.
At least one live certificate. The group's assignments are enumerated exhaustively by depth-first search over the supports — at most $5^6 = 15{,}625$ candidates for a six-card half-suit — each tested against every certificate of that half-suit, and the survivors aggregated by count vector into $g_H(t)$ of (6). Alongside $g_H(t)$ the engine accumulates the per-card breakdown $g_H^{(c\to p)}(t)$ that (8) needs, so one enumeration serves all three queries.

A group of either kind consumes a known number of cards, so the layer identity survives and the outer recursion runs over groups: $F_b(n) = \sum_t g_b(t) F_{b-1}(n+t)$, with g_b the indicator of a single card placement in the card-group case. Nothing in this treatment is approximate: the enumeration is exhaustive, the certificate test is the literal disjunction, and the outer recursion is a counting argument. Inclusion–exclusion over k certificates would instead cost 2^k passes over the whole of U, which is the cost the half-suit locality of (C5) avoids.

Corollary 4 (Equal probability within a count vector)**.** *Under the uniform posterior of §5.1, any two allocations of the same half-suit that satisfy (C1)–(C5) and share a count vector have equal probability.*

Proof. By (9) the probability of a surviving allocation A depends on A only through $t(A)$, since S_t is a function of the count vector and of the tables outside the block. □

The maximum-probability allocation is therefore decided by the count vector alone, and the choice among survivors within a count vector is arbitrary. The caveat is the operative half: an allocation that violates a certificate has probability zero *even when its count vector is shared by survivors*, so survival must be checked explicitly and cannot be inferred from the count vector. The count-vector table records only survivors, which makes the corollary usable and makes the caveat necessary at the same time. §B.7 records which of the two consumers of this table actually performs the check.

B.4 The approximate conditioning step

The approximate path maintains a matrix $p_{c,p}$ over unresolved cards and seats, initialised to the prior weights (11) on the support M_c and zero off it. One Sinkhorn iteration normalises each row to sum to one, then scales each column p by $q_p / \sum_c p_{c,p}$, enforcing (1) in expectation. The deployed setting is four outer sweeps of eight such iterations, with a conditioning step for the certificates applied between consecutive sweeps and a final row normalisation.

The conditioning step derives as follows. Fix a certificate with asking seat a and card set D, and let E be the event $\bigvee_{d \in D} [\sigma(d) = a]$. Treating $\{p_{d,a}\}_{d \in D}$ as independent Bernoulli indicators,

$$\Pr[\neg E] \;=\; \prod_{e \in D} (1 - p_{e,a}), \qquad \Pr[E] \;=\; 1 - \prod_{e \in D} (1 - p_{e,a}).$$

For $d \in D$ the event $\sigma(d) = a$ implies E, so $\Pr[\sigma(d) = a \mid E] = p_{d,a} / \Pr[E]$. For a seat $p \neq a$, the event $\sigma(d) = p$ is compatible with E only if some other card of D goes to a, whose independent probability is $1 - \prod_{e \in D \setminus \{d\}} (1 - p_{e,a})$. Together,

$$\Pr[\sigma(d) = a \mid E] = \frac{p_{d,a}}{1 - \prod_{e \in D} (1 - p_{e,a})}, \qquad \Pr[\sigma(d) = p \mid E] = p_{d,p} \, \frac{1 - \prod_{e \in D \setminus \{d\}} (1 - p_{e,a})}{1 - \prod_{e \in D} (1 - p_{e,a})}. \tag{15}$$

Rows are renormalised afterwards and the next sweep restores the capacity constraints the reweighting disturbed. Two degenerate cases are guarded: when $\Pr[E]$ underflows the mass is forced onto the certificate rather than dividing by a near-zero denominator, and when $\Pr[\neg E]$ underflows the certificate is already implied and the step is skipped.

The independence assumption in (15) is false, because the capacity constraints couple the cards of D, so interleaving it with capacity fitting does not converge to the exact marginals of (8). That is the sense in which the deployed path is approximate, and it is the only sense: the bookkeeping of (C1)–(C4) it runs on is exact.

The deployed declaration estimator. The declaration query (9) asks for a joint probability and the approximate path has only marginals, so it evaluates the joint by sequential conditioning. For a candidate allocation assigning cards $c_1, \ldots, c_6$ to seats $p_1, \ldots, p_6$,

$$\hat{\alpha}(A) \;=\; \prod_{j=1}^{6} \hat{\mu}^{(j)}_{c_j, p_j},$$

where $\hat{\mu}^{(j)}$ is recomputed on the state in which $c_1, \ldots, c_{j-1}$ have already been resolved to $p_1, \ldots, p_{j-1}$. Each step fixes one card, decrements that seat's capacity, propagates the support reductions and refits. A card already resolved contributes one if it matches and zero otherwise, a card whose support excludes its assigned seat returns zero immediately, and the product short-circuits once it falls below a floor, since candidates that weak are rejected regardless. The chain rule is exact — the joint is the product of successive conditionals in any fixed order — so the estimator is exact in structure and approximate only through the marginal solver. A product of *unconditioned* marginals would not be: it ignores the capacity coupling that makes (9) necessary in the first place.

B.5 The policy prior, derived

Equation (11) is a likelihood correction expressed as prior weights. The full Bayesian posterior over deals weights each deal x by $L(\mathcal{I} \mid x) = \prod_t \pi_{p_t}(a_t \mid \mathcal{I}_{<t}, x)$, the probability that the observed actions would have been chosen given x. That object is not available: it requires a model of every opponent's policy and a product over the whole history. What (11) does instead is retain the one sufficient statistic

of the history that any plausible policy makes informative — how many of a seat's public asks fell in a given half-suit, against how many fell elsewhere — and expose its two coefficients to the same optimiser that fits the policy. θ rewards a seat's asks in the card's own half-suit and ϕ penalises its asks elsewhere.

The weights enter as the initialisation of the Sinkhorn fit rather than as a post-hoc multiplication, which is what keeps the capacity constraints exact: $\theta = \phi = 0$ recovers the uniform initialisation and hence the policy-agnostic posterior. It also means the prior has no route into the exact path. `BlockDP::build` takes the `Knowledge` object alone; there is no argument through which a coefficient could reach it, and the exact path never calls the weight function at all. Any belief-mode ablation in this corpus is therefore confounded with a prior ablation, and that confound is not a defect of the experiment but a property of the architecture.

The exact/approximate predictive comparison in full. The comparison of §5.4 is run by `fish v6probe -mode=belief` and its design is as follows. At every ask decision of the measured team, on the acting seat's own `Knowledge` object, each candidate posterior is computed on the *same* state: the exact count law of §5.2, and the approximate path at a list of (θ, ϕ) settings that includes $\theta = \phi = 0$ and the shipped values. Two quantities are then accumulated per posterior.

1. **Predictive log loss.** For every unresolved card of the state, the true holder is read from the ground-truth deal and $-\log \mu_{c,\text{truth}}$ is accumulated, floored to avoid an infinite contribution from a posterior that assigns exact zero. The reported figure is the mean over cards. The unit is one card, not one decision, so a state with a large unresolved pool contributes proportionally more — which is the right weighting, because those are the states inference is for.
2. **Argmax hit rate.** The legal asks are enumerated at that state and the candidates the acting seat's own knowledge already proves dead are discarded. Among the remainder the posterior's own maximiser of $\mu_{c,t}$ is selected, and the accumulator records whether the target really holds the card. The unit is one decision, and each posterior chooses its own ask, so the measure is the quality of the posterior's own recommendation rather than of a shared one.

Two properties of this design carry the argument. It is played on identical states, so no posterior is scored on a trajectory it induced. And it involves no fitted ask weights on either side — the argmax is over the posterior itself, not over the linear utility of §7 — so nothing in the comparison depends on which posterior the policy was tuned against. That is what makes it a verdict on the posteriors rather than a substitution result, and it is why §5.4 can close a question that the v0.4 study's belief-substitution ablation could not.

The result is Table 1. The exact count law under a uniform deal prior scores 1.42246 nats per card and 47.94% argmax hit rate; the approximate path with the prior deleted scores 1.39083 and 49.99%; the shipped prior scores 1.38218 and 51.49%. The exact object is last of the three on both measures. The approximate path beats the exact one even at $\theta = \phi = 0$, where the two encode the same rules-only information; the remaining gap to the shipped row is the prior alone, worth 2.16 points of hit rate and 0.011 nats.

The interpretation is the one §5.4 gives and is repeated here because the appendix is where a sceptical reader arrives: exactness is relative to a prior, the uniform deal prior is a modelling assumption and not a rule, and on this evidence it is a worse assumption than a crude count of who has asked where.

B.6 The exactness audit and its coverage

Agreement between the count law and the block construction is weak evidence, because they share a derivation. The audit of record is therefore a brute-force enumerator that factorises nothing: for one observer's constraint state the unresolved cards are searched by depth-first assignment to seats,

candidates are pruned against the surviving support (carrying (C1)–(C3)) and against the capacity (carrying (C4)), and (C5) is tested at each complete leaf against the literal disjunction. Surviving leaves are counted into Z, tallied into per-card ownership counts, and tallied into a per-half-suit table keyed by the packed assignment. Nothing is factorised, so the count behind every named allocation is an explicit count of leaves.

That audit was run and reported by the v0.4 study, and this report does not re-run it [2]. Its two properties matter here and are restated rather than re-measured. The deterministic comparisons agree *exactly* — not to a tolerance — because both sides compute integer counts over the same finite set of assignments and hold them in double precision below 2^{53}, form the same ratio from identical operands, and obtain identical doubles. And its coverage is a subsample: the search is exponential in $|U|$, so a state is attempted only when the capacity dynamic program reports a small enough number of consistent deals, and a state that exceeds the budget is counted as skipped and reported rather than dropped. Skipping is correlated with exactly the property that makes a state hard — a large unresolved pool — so the audit is evidence about small reachable states and is silent about the high-entropy early game where inference matters most. A pass is exact on a subsample, and that is all it is.

Two v0.6 facts attach to this. The first is that the audit's premise did not hold when it was run. §4.4 shows that a `BlockDP` instance's tables could be repointed by an unrelated build on the same thread; the oracle comparison was unaffected because its driver builds one instance and queries it before anything else can rebuild, which is a property of the driver rather than of the engine. After the repair it is a property of the engine. The second is that the exactness audit answers a narrower question than the phrase suggests: it establishes that the block construction computes the posterior of (C1)–(C5) correctly, and §B.5 establishes that this posterior is not the best available one.

The v0.6-era checks that were run are the reachability measurements of §B.7.

B.7 Latent approximations inside functions documented as exact

Three routines in the inference stack are documented as exact and are not, in regimes no current caller reaches. Each is recorded here with its reachability argument, because unreachable-today is a property of the call sites and not of the code, and every one of these is a trap for the next study rather than an error in this one. The source is `research/v06/notes/R2-inference-stack.md`, §8.

A greedy branch in the count routine. The masked counting routine in `engine/src/belief.hpp` has a fast path for masks that are all equal and one for singletons. For masks that are neither — heterogeneous and non-singleton — it falls through to a greedy sequential assignment that picks the seat with the most remaining capacity, which is an approximation inside a function whose comment says exact. No current caller reaches it: over 673 uniform and 1,633 general queries the routine agreed with the exact posterior to 1.7×10^{-15}. A future caller asking a heterogeneous per-card question — "is each of these cards on my team?" is the natural one — would get a silent approximation.

The allocation search does not re-check survival. The block table's maximum-probability allocation search materialises an assignment from a count vector and returns it without re-testing it against the certificates of (2), whereas the allocation-probability query on the same table does re-test and says why. The search is on the deployed decision path. It was measured at 0 failures in 35,161 checks, 10,374 of them on states carrying a live certificate, over 40 deals at seed 4242. That is unfalsified and it is not proved. Corollary 4 does not supply the missing step: it says survivors sharing a count vector are equiprobable, and says nothing about whether the assignment the search materialises is one of them.

Silent truncation of a group table. The group enumeration returns a failure flag when the number of distinct count vectors exceeds 512, but the caller that populates the table ignores the flag and returns, leaving a *partial* table which the team-ownership query then normalises as if it were complete. The regime is currently unreachable by arithmetic: the number of count vectors for six cards over six seats is $\binom{11}{5} = 462 < 512$, and no truncation was observed in 35,161 checks. It becomes reachable the moment the half-suit size or the table capacity moves. Two further caps have the same shape and the same status: a 16-certificate limit per group, and 2,000,000-node guards on the two depth-first searches.

C Configuration and fitted parameters

This appendix records the frozen configuration in enough detail to reconstruct it exactly. It is a reproducibility artifact and not a description of strategy: the ask features are mutually correlated and normalised to different effective ranges, so the sign and magnitude of a single coefficient do not identify the contribution of the corresponding feature, and §9 explains why the point applies with more force to a vector fitted by a repaired optimiser than to one fitted by a broken one.

The appendix is arranged so that the freeze/parse round trip of §9.5 is checkable by a reader: the layout of the flat vector, the printed values, the specification string that reproduces them, the bounds applied on parse, and the provenance string that names the run the values came from.

C.1 The ask features

The ask utility is a fitted linear score over 20 features, unchanged in definition from the v0.4 and v0.5 policies. Table 17 defines them. All are normalised to roughly $[0, 1]$ so the fitted weights are comparable in scale; H denotes the named card's half-suit and $p = \hat{\mu}_{c,t}$ the approximate posterior that the target holds the named card, so every feature is computed from the deployed inference path of §5.2 rather than from the exact one.

FishBot v0.6 adds 3 further ask terms. They are the mechanism of §7.2 and not a rescaling of the existing score: `wVoid` rewards an ask whose hit takes the target's last card of the half-suit, `wTeamHas` penalises an ask for a card the actor's own team already holds, and `wLastLive` separates the case where the target is the last live opponent. They enter the same linear term and are fitted as three further coordinates rather than set by hand, and what each is worth is reported in §11.8.

C.2 The frozen parameter vector

The v0.6 flat vector has 37 coordinates in one fixed order:

Coordinates	Contents
$0 \ldots 20 - 1$	the ask weights $w_0, \ldots, w_{19}$ of Table 17
next 14	the v0.5 decision knobs, in the order of Table 19
last 3	`wVoid`, `wTeamHas`, `wLastLive`

The offset at which each block begins is derived from the ask-feature count and the v0.5 knob count in both the parser and the freeze step; it is not hard-coded anywhere. That is not a stylistic preference. A hard-coded offset in exactly this place aliased two ask weights onto the leading two knobs in the v0.4 study when the ask-feature count grew, and a hard-coded `18` survived in the optimiser until §4.4 removed it.

The same vector can be supplied to any build of the engine as a single `allparams=` string of 37 pipe-separated coordinates in that order, which is the reconstruction key: a specification `v06:allparams=...`

Table 17: The 20 ask features φ_k. Probabilities written $\Pr[\cdot]$ are approximate per-card marginals. Features 3, 5, 7 and 15 are the ownership terms, and in FishBot v0.5 and FishBot v0.6 each is multiplied by an ownership gate that is zero when the ask cannot succeed; that gate is the mechanism the v0.5 study introduced to stop the ownership terms paying for an ask of probability zero, and §7 measures what it costs.

k	Name	Definition
0	hit probability	$p = \hat{\mu}_{c,t}$, the posterior that the target holds the named card
1	squared hit	p^2; lets the fit curve the trade-off between certainty and value
2	certain hit	the indicator that p exceeds the near-one threshold
3	own progress	cards of H in hand, over six, gated
4	team control	expected fraction of H held by our team
5	lock completion	$\prod_{d \in H \setminus \{c\}} \Pr[d \text{ is ours}]$: the probability this ask alone completes team ownership of H, gated
6	continuation	best posterior on any other missing card of H
7	completion bonus	a step at four or more cards of H in hand and a smaller one at three, gated
8	reply threat	$(1-p)\cdot$ the best ask the target could make against us if handed the turn
9	information leak	the indicator that our team has never publicly revealed an interest in H
10	target hand size	target's public card count over nine
11	empties the target	p if the target holds exactly one card, else zero
12	repeats our half-suit	the indicator that this repeats our own previous half-suit
13	known team cards	publicly located cards of H on our team, over six
14	location entropy	the binary entropy of p
15	team owns H	$\prod_{d \in H} \Pr[d \text{ is ours}]$ before the ask, gated; an independence proxy, not the exact query (10)
16	exposure on a miss	$(1-p)\cdot$ our cards visible in half-suits the target has asked in
17	trailing pressure	p when behind by two or more half-suits
18	runway	expected length of the run this ask begins, from the best remaining per-card posteriors
19	leak magnitude	feature 9 scaled by how much of H we hold

rebuilds the configuration from the numbers alone. A shorter vector is accepted and the missing coordinates take their compiled defaults, which is how a v0.5-length vector of 34 coordinates parses into a v0.6 build with the three new terms at zero — and that, together with `legacy=1`, is what makes the identity control of §4.4 expressible.

C.3 Bounds and per-coordinate step sizes

Bounds are applied when a sampled vector is turned into a configuration, by the engine's parameter application in `engine/src/v06.hpp` and by the freeze step in `engine/freeze_config_v06.py`, so a coordinate outside its bound is clamped rather than rejected and the vector scored during fitting is the vector compiled into the binary. The two integer coordinates are rounded to the nearest integer after clamping.

The clip column is the evidence for §9.1 and belongs in this appendix per coordinate rather than as a single summary, because the defect it diagnoses is a *distribution* over coordinates and not a maximum: the previous optimiser was bang-bang on the bounded knobs and frozen on the unbounded ones at the same time, and a single worst-case figure hides the second half of that. Every generation record of a fit trace carries the full per-coordinate clip vector, so the table below is read directly out of the run file rather than recomputed. At generation zero the worst coordinate of run A clips at 0.00% and of run C at 50.00%.

Table 18: The frozen FishBot v0.6 configuration: 37 coordinates in the order above. Generated by `engine/build_tables_v06.py` from the run named in the provenance string of §C.3. This table is a reproducibility record, not a description of strategy.

Coordinate	FishBot v0.5	FishBot v0.6
— *artifact not regenerated* —		

Table 19: Bounds applied to each coordinate of the 37-coordinate vector on parse. The ask weights and the three v0.6 ask terms are unbounded; the 14 knobs are listed in the vector's own order. The per-coordinate step size used in fitting is σ_{rel} times the width of the bound (§9.1), so an unbounded coordinate takes the step of its nominal search range rather than an unbounded one.

Coordinate	Quantity	Bound
ask weights	$w_0, \ldots, w_{19}$	unbounded
1	declaration threshold	[0.5, 0.9999]
2	locked-half-suit threshold	[0.5, 0.99999]
3	ask floor	[0, 0.9]
4	patience pool	[0, 45], rounded to an integer
5	opponent-card floor	[0, 20]
6	value weight λ_{val}	$[0, \infty)$
7	linear weight λ_{lin}	$[0, \infty)$
8	minimum team probability	[0.05, 0.99]
9	declaration margin	unbounded
10	prior θ	[0, 2]
11	prior ϕ	[0, 1]
12	lookahead width K	[0, 24], rounded to an integer
13	chain weight λ_{chain}	$[0, \infty)$
14	threat weight λ_{threat}	$[0, \infty)$
v0.6 terms	`wVoid`, `wTeamHas`, `wLastLive`	unbounded

Provenance. `engine/freeze_config_v06.py` writes the *whole* flat vector into the compiled defaults and, next to it, a provenance string recording the run file it came from, whether the values are that run's final weights record or a generation's distribution mean, the first sixteen hex digits of the run file's SHA-256, the generation count, the objective, the pairing flag, the opponent panel, the fitting seed, the deals and rotations per cell, and the relative step size. The field is never empty: an unfrozen build carries a string that says so. This is the general form of a defect the v0.4 study shipped — a configuration whose coefficients were not the ones any recorded fit produced, because the freeze step wrote only part of the vector — and the provenance string closes it for everything the freeze step writes. What it does not cover is named in §C.4.

C.4 Value-function coefficients

The value function that supplies the one-ply lookahead term and the stopping rule is a linear score over sixteen state features: a bias and the score differential; three control terms built from the expected fraction of each half-suit our team holds, together with a sharpened version and the contested mass; the signed count of half-suits already owned each way; counts of what is left to play for, from the card differential and the size of the unresolved pool to the near-complete half-suits leaning each way; and side to move with its interactions with control and with the unresolved pool. Features are collected from all six seats at every decision point rather than from the mover alone, because under mover-only collection the side-to-move feature is constant and its coefficient would be unidentified.

These sixteen coefficients are *not* part of the 37-coordinate vector, are not searched by the optimiser, and are not written by the freeze step. They are compiled defaults carried forward unchanged from the v0.4 study, and the v0.4 study's own provenance gap — that the compiled coefficients differ numerically from the ones in its recorded ridge fit, so its reported goodness-of-fit describes an artifact rather than the deployed vector — therefore stands in this report as well. It is recorded in §G.5 rather than repaired, because repairing it would change the configuration that produced every number here and would break the identity control of §4.4.

C.5 The policy-specification grammar

Every agent in the engine is named by a specification string, parsed by `makeAgent` in `engine/src/factory.hpp`. The grammar is

`name` or `name:key=value,key=value,...`

where `name` selects the policy family and each `key=value` pair overrides one field of that family's configuration. Keys are order-independent, there is no whitespace, and **unrecognised keys are ignored**. The last property is worth stating rather than tolerating: a misspelt key fails silently and the default is used, so a specification always parses and a typo is indistinguishable from an intentional default. This is how a v0.5 specification could carry `gateaudit=1` for an entire study while the option was parsed only in the v0.4 branch (§4.4), and it is why the run scripts of §G.2 write every key explicitly even when it equals the compiled default.

The keys used in this report fall into four groups. The inference keys are `belief`, which selects the posterior path and accepts `fast` (the deployed default), `block` (the exact count law of §5.2), `exact`, `exactdisj`, `sinkhorn`, `indep` and `hybrid`, together with `souter`, `sinner` and `particles` for the solver. A caution attaches to these: only `fast` calls the policy prior of §5.3 at all, so every belief-mode row is confounded with a prior ablation, as §B.5 explains. The policy keys — `decl`, `lockthr`, `minteam`, `askfloor`, `pool`, `oppfloor`, `vweight`, `lweight`, `vmargin`, `ptheta`, `pphi`, `topk`, `chain`, `threat` — set the corresponding coordinates of Table 19, and individual coefficients are addressable as `w0`–`w19`, or in bulk as `weights=`, `vweights=` and `allparams=`. The instrumentation keys are `gate` and `mgate` for the declaration pre-gates and `gateaudit` for the audit of §4.4. The v0.6 keys are listed in §C.6.

C.6 Mechanism switches and their shipped states

Table 20 lists one row per v0.6 mechanism: the switch that enables it, its state in the shipped configuration, and the section that reports the measurement which decided that state. A mechanism that is off is off because a measurement said so, and the measurement is named; nothing here is off because it was not tried.

Two conventions in that table are deliberate. A switch shown as `--` is a parameter of a mechanism that is itself off, so it has no shipped state; its row exists because the sweep over it is a result even though the mechanism is not deployed. And the certificate de-duplication flag of §4.4 is not in the table, because it is not a mechanism: it is an engine repair whose default is off precisely so that it changes nothing.

D Extended ablations

D.1 Design

Every ablation in this study is paired. A variant plays the *same* deals against the *same* panel as the reference, and the statistic is the per-deal margin, bootstrapped resampling deals as clusters because

Table 20: The v0.6 mechanism switches. "Shipped" is the state in the frozen configuration of §C.2. "Fitted" distinguishes a mechanism whose parameters were coordinates of the fit from one run at a hand-set value. Every row's evidence is a paired comparison at the budget §10.4 requires.

Switch	Mechanism	Shipped	Evidence
`legacy`	restore the v0.5 vector exactly	off	§4.4
`xf`	the 3 v0.6 ask terms	on, fitted	§7.2
`wvoid`	void creation	fitted	§11.8
`wteam`	team-ownership discount	fitted	§11.8
`wlast`	last-live-opponent split	fitted	§11.8
`extie`	exact-posterior tie resolution	off	§6.3
`exactp`	exact posterior as the ask marginal	off	§5.4
`s1`	determinized information-set search	off	§8.3
`kappa`	the paired deviation threshold	—	§8.4
`maxq`	endgame-restricted search	—	§8.4
`dead`	the deliberate miss, rationed	off	§8.5
`deadsearch`	the deliberate miss as a search candidate	off	§8.5
`gateaudit`	declaration pre-gate instrumentation	off	§4.4

the rotations of one deal are a single correlated observation. This is the 'ablate' command, and it is the only estimator used for a mechanism claim anywhere in this paper.

The reason is in §10.3 and is worth restating here, because it is the single most consequential methodological fact this study produced. Five mechanisms cleared a 95% paired interval at four hundred to sixteen hundred games per cell — the budget at which most published ablation rows in this project's three prior studies were collected — and returned *exactly* zero at three thousand, on two disjoint seed banks. Table 21 is that table.

D.2 The mechanisms that vanished

Table 21: Five mechanisms, at the corpus's usual per-cell budget and at this study's. A negative delta means the variant beats the reference. Every large-n row is the result of record.

mechanism	small-n paired delta	large-n, two disjoint banks
void creation, $w = 1.5$	-0.0292 $[-0.0537, -0.0037]$	$+0.0007$ $[-0.0213, +0.0227]$; $+0.0200$ $[-0.0017,$
void creation, $w = 3$	-0.0313 $[-0.0593, -0.0033]$	as above
delete the perseveration bonus	-0.0200 $[-0.0356, -0.0044]$	-0.0012 $[-0.0137, +0.0112]$; $+0.0035$ $[-0.0092,$
declaration margin -0.06	$+1.75$ points unpaired	$+0.0133$ $[+0.0013, +0.0247]$
delete the negative value-of-information term	-0.0053 $[-0.0244, +0.0141]$	not separated

Two readings, and the second is the one that generalises. The first is that none of these mechanisms works. The second is that the evidence standard this project has used since v0.3 cannot separate a real one-point effect from a seed draw, which means the design register the three prior studies built — and which this study inherited and set out to work through — was partly assembled from noise.

D.3 The switched-off mechanisms

FishBot v0.6 ships several mechanisms switched off. Each is listed with why, so that a later study does not rebuild it:

- **Test-time search** (§8). Off by default and shipped as the separate FishBot v0.6-Search configuration: it is the one mechanism here that beats a static rule, and it costs three orders of magnitude in throughput.

- **Ownership features scaled by hit probability.** Measured at −1.33 points by the v0.5 study once M1 removes the case the scaling was for.
- **A blanket (card, target) repetition guard.** Measured at −6.13 points — the largest single-switch effect in this codebase — because cards move, so a repeat after a public transfer is frequently the best ask on the board. The *dead*-ask repeat guard v0.6 uses is a different object and is provably free (§8.5).
- **Certificate de-duplication.** Correct, and it changes the approximate posterior, so it is opt-in and off by default to keep v0.4 and v0.5 bit-identical.

D.4 Per-opponent detail

Table 22: Paired ablations against FishBot v0.6 over the six-opponent panel.

Variant	Pooled	FishBot v0.6 − variant	95% paired CI
`v05`	65.73	-2.69	[-4.35, -0.98]
`v06:legacy=1`	65.73	-2.69	[-4.35, -0.98]
`v06:xf=0`	68.54	+0.12	[-1.67, +1.92]
`v06:s1=1,det=16,cand=6,kappa=2.0,maxq=26`	69.77	+1.35	[-0.04, +2.79]

D.5 The four-way that separates two confounded switches

A non-zero extra ask weight is what sets the flag that selects the v0.6 scoring path, and the v0.6 scoring path is the one that does *not* run v0.5's top-K chain and threat re-scoring. Switching the extra terms off therefore switches that pass back on, so the obvious ablation moves two things at once. Table 23 runs all four cells.

Table 23: The four-way. A negative delta means the variant beats FishBot v0.6. All four are null.

Arm	Configuration	Pooled	FishBot v0.6 − variant	95% pair
`v06:wvoid=0,wteam=0,wlast=0`	extras off, chain off	68.88	-0.46	[-1.15, +
`v06:chain2=1`	extras on, chain ON	69.17	-0.75	[-2.44, +
`v06:xf=0`	extras off, chain ON	68.54	-0.12	[-1.96, +
`v06:rtie=1`	extras on, chain off, ties at random	69.10	-0.69	[-2.54, +

Two things follow. The extra ask terms are worth nothing, and so is the chain pass — which is why FishBot v0.6 drops it, and why FishBot v0.6 is the fastest policy in the family (303.4 games per second against FishBot v0.5's 276.3). And resolving the tie group at random is worth nothing too, which is the control that makes §6.3's reading of the search result possible.

D.6 Specification of every arm

Arms are named by the configuration grammar of Appendix C. `v06:legacy=1` is FishBot v0.6 carrying v0.5's parameter vector and is bit-identical to FishBot v0.5; it is the arm that isolates the refit from every other change. `v06:xf=0` removes the three extra ask terms. `v06:s1=1,...` switches the search on at the stated budget.

E Test-time search: implementation detail

E.1 Sampling the deal

The capacity dynamic program of §5.2 is an exact sampler as well as an exact counter: descending it with the backward table as the branch weights draws uniformly from the deals satisfying constraints C1–C4. Certificates (C5) are enforced by rejection, exactly as the reference posterior does, so an accepted draw is an exact sample of the policy-agnostic posterior.

The policy prior is then applied as an *importance weight* rather than being folded into the sampler. Two reasons. The sampler stays exact and testable against the brute-force enumerator; and the prior stays a separable, auditable object, which matters because §5.4 shows the prior is carrying more of the posterior's predictive value than the exactness is. The weighted statistics use Kish's effective sample size, so the deviation rule's standard error is not overstated when the weights are uneven.

Measured cost at a decision: the dynamic program builds in tens to hundreds of microseconds depending on how much of the deal is still unresolved, and one accepted draw costs well under a microsecond. Determinization is not the expensive part of this search; the rollout is.

E.2 Reconstructing the six information sets

The reconstruction rests on a structural fact about the deduction state: every write it performs is a function of the public event alone, except the two lines that maintain the observer's own hand. Six such objects fed the same event stream from an all-possible start therefore agree bit for bit outside those two fields, so the public deduction state is *one* object, replayed once per decision, and a seat's information set is that object refined by the hand the determinization gives it.

This is checked against ground truth rather than argued. Over 3,216 states and 103,644 card checks the reconstruction differs from the seat's real deduction state on 0.3338% of cards, and in 346 of 346 differing cases the reconstruction is *wider*: it is a strict under-approximation. No simulated player is ever credited with knowledge the real one lacks, which is the direction the soundness argument needs.

The cost of reconstructing all six seats is a few microseconds, because the shared object is replayed once and the six refinements are linear in the deck.

E.3 The blueprint used inside the rollout

A rollout is only as informative as the policy that plays it. Table 2 gives the frontier. Two axes were available and the corpus had only ever used one of them:

- **Dropping the two-ply refinement** costs little strength and removes most of the per-decision cost, because that refinement rebuilds the approximate posterior twice per candidate.
- **Reducing the Sinkhorn iteration count**, which the corpus had not tried, is the cheap axis. The deployed setting is four outer and eight inner sweeps; two and four is statistically indistinguishable from the full blueprint at a fraction of the cost, and one and one costs a few points.
- **Replacing the belief with independent marginals** is 62× cheaper per event and 38 points weaker. It was the first blueprint tried and it is why the first search build scored in single digits.

E.4 The deviation rule

Candidate 0 is the blueprint's own choice. For each other candidate r the search forms the paired difference $d_r = \sum_d w_d\,(v_{r,d} - v_{0,d})$ over the shared determinizations, its weighted variance, and a standard error using the effective sample size; it deviates to $\arg\max_r (d_r - \kappa\,\mathrm{se}_r)$ only when that lower bound is positive.

Inside the blueprint's own tie group the penalty is waived, because there the blueprint expresses no preference and "deviating" from whichever candidate the enumerator emitted first is not a deviation. That waiver is load-bearing: applying the penalty inside the tie group as well takes the win rate to 49.8611% — the advantage goes with it.

E.5 The deviation rate, and why it has to be decomposed

Table 4 is the diagnostic, and read naively it says the search is mis-scaled: it moves 33.49% of the decisions it searches at the best setting, and the rate never falls below about 31% however far κ is raised.

The floor is the waived tie group. Applying the same penalty there collapses the rate to 2.70% at $\kappa = 2.5$ and 0.11% at $\kappa = 4$, so the search's *evidence-backed* deviation rate is inside the band the literature associates with a refinement, and the 31% floor is re-choosing among candidates the blueprint could not separate. Since the win goes away when that waiver is removed (49.8611%), the tie group is where the search is doing its work — though §8.4 states why the attribution is not resolved at this sample size.

F Calibration

F.1 What is forecast

Two probabilities are emitted and can be scored against outcomes. The ask forecast is the posterior probability that the named target holds the named card, recorded at every ask by the measured team. The declaration forecast is the policy's own probability that the allocation it is about to name is correct, recorded at every voluntary declaration.

F.2 Reliability

See `research/v06/results/E6-calibration.txt` for the raw ten-bin reliability diagrams and the Brier, log-loss and expected-calibration-error summaries for both forecasts.

F.3 The calibration/resolution distinction

§5.4 turns on a distinction that a single calibration number hides. A forecaster can be well calibrated and poorly resolved: if it says 0.46 and is right 46% of the time, it is calibrated, however uninformative 0.46 is. The exact posterior and the deployed approximation are *both* well calibrated in this game — the mean forecast of each tracks its realised frequency closely — and they differ in resolution, in the approximation's favour, because the approximation carries a soft model of how opponents choose their asks and the exact object does not. Any comparison of inference paths that reports only calibration will therefore report that they are equally good, and be wrong.

F.4 Caveats

Forecasts are collected from one team only, so the sample is not independent across seats within a game; the intervals in the reliability table are indicative rather than inferential. The declaration forecast is a probability under the policy's own posterior, not under the exact one, and §5 gives the size of the gap between them.

G Reproducibility and artifact manifest

Every number in this report must be reachable from a command in this appendix. That is the standard the v0.4 report set and this one matches it, with one addition the earlier studies did not have: the numbers pipeline is arranged so that a figure which has *not* been regenerated is visibly wrong on the page rather than plausibly stale (§G.3).

G.1 Source layout

The engine is a single C++20 translation unit assembled from headers in `engine/src/`. The rules, the card and half-suit types and the action encoding are in `engine/src/fish.hpp`. Constraint tracking (C1–C5) and the approximate posterior are in `engine/src/belief.hpp`; the exact count law of §5.2 — its partition function, its marginals, its allocation query and the table-aliasing guard of §4.4 — is in `engine/src/blockdp.hpp`; the brute-force enumerator that audits it is in `engine/src/oracle.hpp`.

The policies are layered. `engine/src/v04.hpp` and `engine/src/v05.hpp` hold the two preceding generations, and `engine/src/v06.hpp` holds `V06Agent`, which derives from `V05Agent` and defers to it when no v0.6 mechanism is enabled; the determinized search's rollout machinery is in `engine/src/v06_rollout.hpp`. The scripted opponent population is in `engine/src/baselines.hpp`. The game driver, which owns turn order, declaration rounds and the information-safety audit, is in `engine/src/game.hpp`; the match runner and both bootstrap procedures are in `engine/src/arena.hpp`; the optimiser of §9 is in `engine/src/tuner.hpp`. The v0.6 diagnostics are in `engine/src/probe_v06.hpp` and the aliasing probe in `engine/src/probe_forcedendgame.hpp`. The specification parser is in `engine/src/factory.hpp` and the entry point in `engine/src/main.cpp`.

Build with `make` in `engine/`. The build produces a single binary, `engine/fish`.

G.2 Commands

The battery is `engine/experiments_v06.sh`, which runs sixteen blocks labelled `E0`–`E15` and writes one artifact per block into `research/v06/results/`. It is presented below in the order the script executes them, which is not numerical order: the two gates run first by construction. `--games` counts *deals*, not games, so every game count in this report is the product of the deal count and the rotation count.

```
# ---- build -------------------------------------------------------------
cd engine && make

# ---- E0  GATE: identity controls ---------------------------------------
#      md5 of the whole pathology block, v06 with every switch off
#      against v05; then the BlockDP aliasing probe on both policies.
#      Nothing downstream is collected if this fails.
./fish pathology --a=v06:legacy=1 --b=v06:legacy=1 --games=60 --seed=31
./fish pathology --a=v05          --b=v05          --games=60 --seed=31
./fish blockalias --a=v04 --games=25 --seed=31
./fish blockalias --a=v05 --games=25 --seed=31

# ---- E1  rules, information safety and belief soundness ----------------
./fish verify --games=600

# ---- E2  GATE: the pathology KPIs --------------------------------------
./fish pathology --a=v06 --b=v06 --games=300 --rotations=2 --seed=31
```

```
./fish pathology --a=v05 --b=v05 --games=300 --rotations=2 --seed=31
./fish pathology --a=v04 --b=v04 --games=300 --rotations=2 --seed=31
./fish pathology --a=v06 --b=v05 --games=300 --rotations=2 --seed=90210

# ---- E3  head-to-head, five held-out banks ---------------------------
for S in 90210 31337 515151 777001 424242; do
  for OPP in v05 v04; do
    ./fish match --a=v06 --b=$OPP --games=300 --rotations=6 \
                 --seed=$S --json
  done
done

# ---- E4  per-opponent profile; the headline is the MINIMUM -----------
for OPP in v05 v04 v03 v02 lockout detective diversifier hunter \
           bluffer random silent feint withholder; do
  for A in v06 v05 v04; do
    ./fish match --a=$A --b=$OPP --games=300 --rotations=6 \
                 --seed=515253 --json
  done
done

# ---- E5  paired mechanism ablations ----------------------------------
./fish ablate --ref=v06 --panel=v05,v04,v03,lockout,detective,withholder \
    --games=400 --rotations=2 --seed=606060 \
    --variants="v05;v06:legacy=1;v06:xf=0;v06:s1=1,det=16,cand=6,kappa=2.0,maxq=26"

# ---- E6  calibration -------------------------------------------------
./fish calibrate --a=v06 --b=v05 --games=400 --seed=717171

# ---- E7  rule dialects -----------------------------------------------
for FLAG in "" --no-out-of-turn --no-cardless-declare --legacy; do
  ./fish match --a=v06 --b=v05 --games=250 --rotations=6 \
               --seed=828282 $FLAG --json
done

# ---- E8  exchangeable ties, and belief as a predictor ----------------
./fish v6probe --mode=ties --a=v06 --b=v06 --games=150 --seed=31
./fish v6probe --mode=ties --a=v05 --b=v05 --games=150 --seed=31
./fish v6probe --mode=ties --a=v05 --b=v03 --games=150 --seed=90210
./fish v6probe --mode=belief --a=v05 --b=v05 --games=120 --seed=31

# ---- E12 the search ladder: the curse, and what a guard recovers ------
#      row 1 is the control: the same code with the blueprint forced.
for CFG in s1=1,det=8,cand=6,blend=1000000 \
           s1=1,det=8,cand=6,kappa=0 \
           s1=1,det=12,cand=4,kappa=0 \
           s1=1,det=12,cand=4,kappa=1 \
           s1=1,det=12,cand=4,kappa=2.5 \
           s1=1,det=12,cand=4,kappa=4 \
```

```
           s1=1,det=16,cand=6,kappa=2,maxq=26 \
           s1=1,det=16,cand=6,kappa=2,maxq=26,deadsearch=2; do
  ./fish match --a="v06:legacy=1,$CFG" --b=v05 --games=120 \
               --rotations=6 --seed=90210 --json
done

# ---- E15 the deliberate miss: the gate's negative control -------------
./fish pathology --a=v05:m1=0,m1p=1 --b=v05:m1=0,m1p=1 \
                 --games=200 --rotations=2 --seed=31
./fish match --a=v05:m1=0,m1p=1 --b=v05 --games=200 --rotations=6 \
             --seed=90210

# ---- E14 search deviation rate ----------------------------------------
for K in 0 1 2.5 4 6; do
  ./fish v6probe --mode=search --a="v06:legacy=1,s1=1,det=12,cand=4,kappa=$K" \
                 --b=v05 --games=20 --seed=90210
done

# ---- E13 rollout-blueprint fidelity -----------------------------------
for R in "v05:belief=indep,topk=0" "v05:topk=0,souter=1,sinner=1" \
         "v05:topk=0,souter=1,sinner=3" "v05:topk=0,souter=2,sinner=4" \
         "v05:topk=0"; do
  ./fish match --a="$R" --b=v05 --games=200 --rotations=6 \
               --seed=90210 --json
  ./fish bench --a="$R" --b="$R" --games=100 --threads=1
done

# ---- E11 partner regimes ----------------------------------------------
for P in "" v03 detective withholder; do
  for A in v06 v05; do
    ./fish match --a=$A --b=v05 --partners="$P" --games=400 \
                 --rotations=2 --seed=313131 --json
  done
done

# ---- E9  throughput ---------------------------------------------------
for S in v04 v05 v06 "v05:topk=0" "v05:belief=indep,topk=0"; do
  ./fish bench --a="$S" --b="$S" --games=200
done

# ---- E10 declaration pre-gate audit -----------------------------------
#      dead code for v0.5 until this study; see sec:engine-repairs.
./fish gateaudit --a=v06:gateaudit=1 --games=400 --rotations=6 --seed=90210

# ---- the whole battery ------------------------------------------------
./experiments_v06.sh

# ---- exploitability: fit an exploiter, re-measure it on a fresh bank ----
#      v0.4 is probed first as a POSITIVE CONTROL, because its published
```

```
#      figure is known and the probe must reproduce it before any new
#      number from the same machinery is believed.
BASE=v05 TARGETS="v04 v05" ./exploitability_v06.sh

# ---- fitting: the two runs of record -----------------------------------
./fish tune --base=v05 --panel=v05,v03,lockout,withholder --full --paired \
            --obj=minimaxregret --sigmarel=0.05 --pop=20 --elite=5 \
            --games=150 --gens=30 --seed=20260823 \
            --out=../research/v06/runs/fitA.jsonl
./fish tune --base=v06 --panel=v05,v03,withholder,feint --full --paired \
            --obj=minimaxregret --sigmarel=0.04 --pop=14 --elite=4 \
            --games=200 --gens=14 --seed=20260824 \
            --out=../research/v06/runs/fitC.jsonl

# ---- harness acceptance test: recovery from a handicapped base ----------
#      NOT part of experiments_v06.sh.  The exact panel and seed of the
#      run reported in sec:fit-schedule are not recorded; see
#      app:repro-gaps.
./fish match --a=v05:w0=0 --b=v05 --games=300 --rotations=6 --seed=<bank>
./fish tune  --base=v05:w0=0 --full --paired --obj=minimaxregret \
             --sigmarel=0.05 --gens=10 --seed=<recorded in the trace> \
             --out=<trace>
./fish match --a="v05:allparams=$w" --b=v05 --games=300 --rotations=6 \
             --seed=<bank> --json
./fish match --a="v05:allparams=$w" --b=v03 --games=300 --rotations=6 \
             --seed=<bank> --json

# ---- freeze a fitted vector into the compiled defaults -------------------
python3 freeze_config_v06.py ../research/v06/runs/fitC.jsonl --dry
python3 freeze_config_v06.py ../research/v06/runs/fitC.jsonl
```

The battery ran in 1.5 hours on 15 cores. Throughput figures were measured under that load and are lower bounds for that reason (§10.3).

G.3 The build and the numbers pipeline

No number appears as a literal anywhere in `paper/sections_v06/`. Every figure is a macro, and the macros come from a deliberately two-file scheme introduced by the v0.5 study and kept here.

`paper/numbers_v06.tex` is the placeholder file. It declares every macro the manuscript uses, with `\providecommand` so that partial regeneration cannot clash, and it splits them into two classes that are handled differently. A macro naming a quantity the v0.6 battery produces carries a deliberately impossible literal — `XX.XX`, `X,XXX` — so that a forgotten regeneration is loud on the page rather than silently plausible. A macro naming a quantity *transcribed* from an artifact that already exists on disk, such as a figure carried forward from the v0.4 or v0.5 record, carries its real value under a comment naming the file it was read from, because leaving it as `XX.XX` would make the sentence quoting it unreadable rather than visibly unfinished.

`engine/build_tables_v06.py` reads `research/v06/results/` and writes `paper/numbers_v06_generated.tex`, which the manuscript inputs *after* the placeholder file, so a generated value always wins. Three disciplines are enforced in the generator rather than by convention: every macro carries a

comment naming the artifact and the field it came from; a macro whose artifact is missing is not emitted at all, so the placeholder survives and the omission is visible; and the emission is `\providecommand` followed by `\renewcommand`, so input order cannot change the result. The same script writes the row bodies of the tables in `paper/tables_v06/`, and a table whose artifact is absent is guarded at the `\input` site so the manuscript still compiles.

Two checkers back this up. `paper/check_provenance.py` verifies that every transcribed macro sits under a comment naming an artifact and that the artifact exists on disk, and reports the transcribed/generated split that §G.4 quotes. `paper/check.py` enforces the no-literal rule and a manuscript-specific list of banned phrasings; a v0.6 profile still has to be added to its profile table, and until it is, the no-literal rule for this manuscript is enforced by reading rather than by tooling. That is a gap and is listed as one.

The phrases banned for this manuscript are:

- any wording that calls the deployed approximate posterior *exact*. It is exact in its bookkeeping of (C1)–(C4) and approximate in its treatment of (C5), and §B.4 says exactly where.
- any wording that treats the *substitution* result — swapping the exact posterior into a policy fitted for the approximate one — as a verdict on exact inference. The verdict in this report rests on the predictive comparison of §B.5, which carries no fitted weights on either side; the substitution result is confounded and is reported as confounded.
- any claim of a first.
- any headline aggregate over the opponent panel. The headline is the worst case and the minimax regret (§10.6).

The manuscript is built by `npm run paper:v06`, which flattens the `\input` tree into `paper/fishbot_v06_standalone.tex` for submission and then compiles `paper/fishbot_v06.tex` with `tectonic` into `output/pdf/`.

G.4 Artifact manifest

Table G.4 is generated by `engine/build_tables_v06.py` from the artifact files themselves: the script walks `research/v06/results/`, records each file's size and SHA-256, and pairs it with the command that produced it. It is not maintained by hand.

Artifact manifest. Artifact manifest for the experiments reported here. "ID" is the experiment identifier used throughout the text, "Artifact" the file under `research/v06/results/`, "Design" the population and experimental unit recorded by the generator, and the last column the leading hex digits of the file's SHA-256. The full digests, byte counts and exact command lines are in `research/v06/results/MANIFEST`

Artifact	Bytes	sha256 (first 12)
E0-identity.txt	594	d4228f6ac7fc
E1-verify.txt	119	1cf3b1e6f994
E10-gateaudit.txt	372	22a9cbda2a33
E11-partners.jsonl	4,374	b752dad1df7c
E12-search.jsonl	4,858	63b1e2954c4f
E13-rollout.jsonl	3,162	cb0e9e57e64d
E14-searchdev.txt	2,840	b7c28fc98676
E15-deliberate-miss.txt	2,127	1dbff1355e7f
E2-pathology.txt	2,928	4b0aec8137d4
E3-headtohead.jsonl	5,286	47cf9d834d4f
E4-perstyle.jsonl	20,486	154ad97702f1

Artifact	Bytes	sha256 (first 12)
E5-ablations.json	818	6761baedb9ce
E6-calibration.txt	891	8fb4ca32941c
E7-rules.jsonl	2,185	f9056ed0672e
E8-belief.txt	916	d892d62d8555
E8-ties.txt	3,686	adaf2d51e124
E9-throughput.txt	330	32a244f942a0
F0-search-confirm.jsonl	5,062	2aa98d49ff21
F1-chain2x2.json	816	d663f80c0f8a
F10-panel-banks.json	730	2663c7fd665d
F11-bighead-v05.jsonl	2,797	ea2d0c16339b
F12-bighead-extra.jsonl	2,801	616edf8e37b5
F2-belief-noprior.txt	846	f5c2579cbd9b
F3-oracle.txt	700	94fab4303d6f
F3-selftest.txt	289	35936f9f0376
F4-perstyle-banks.jsonl	17,129	0c812e296d1d
F5-exploitability.log	1,962	3565de5dd586
F6-reconstruction.txt	588	0a6d8611301c
F7-tieguard.txt	1,229	000af9d5b4f9
F7-winrate.txt	235	2b70dcb2e7c9
F8-tiesearch.txt	530	684396fe47c5
F9-tieonly.txt	507	fae275343f67
MANIFEST.json	5,367	5e36ca2488ab
X1-lbr-v04.jsonl	7,498	008b9f2625ac
X1-lbr-v05.jsonl	7,474	8a54fb672b99
X1-lbr-v06.jsonl	7,496	da5e5a201c9f
X1-lbr.jsonl	2,536	624d30143c6e

`research/v06/results/MANIFEST.json` additionally records the commit the manifest was generated at, whether the working tree was clean at that moment, the shipped policy specification, the belief mode, the fitted-parameter count, the ask-feature count, the fitting seeds and every evaluation bank of §10.5. Fitting traces are in `research/v06/runs/`, and the authoritative record of which configuration was frozen is the provenance string of §C.3, which names the run file and its digest inside the compiled header itself.

The manifest holds 37 artifacts. Of the figures quoted in this manuscript, 211 are generated straight from an artifact and 84 are transcribed from a report that already existed; §13 states what that split means for how much of this report is machine-checked.

G.5 Known reproducibility gaps

Recorded rather than repaired, because repairing any of them would change a configuration this report measured.

1. **The harness acceptance run is not scripted.** The recovery test of §9.5 — a fit from the handicapped base `v05:w0=0` — is not one of the blocks of `engine/experiments_v06.sh`, its panel and seed are not recorded in any committed script, and `engine/build_tables_v06.py` does not emit its figures. Those figures are therefore transcribed, not generated, and the run can be repeated but not reproduced exactly.
2. **The value-function coefficients are outside the freeze.** The sixteen coefficients of §C.4 are

compiled defaults carried forward from the v0.4 study, are not part of the fitted vector, and are not written by `engine/freeze_config_v06.py`. The v0.4 study's own gap — that its compiled coefficients are not the ones its recorded ridge fit produced — is inherited here unchanged.

3. **The exactness audit was not re-run.** The brute-force oracle comparison of §B.6 is carried forward from the v0.4 study. Its coverage was a subsample of small reachable states then and no larger now, and the v0.6 battery adds no block that re-runs it.
4. **Replay identity is tested at one thread count.** The random streams are keyed so that thread count cannot affect a game's outcome, but `fish verify` checks replay identity only at a fixed thread count and does not test that claim directly (§4.3).
5. **The no-literal rule is not yet tooled for this manuscript.** `paper/check.py` has profiles for the v0.4 and v0.5 manuscripts and not for this one, so the rule of §G.3 is currently enforced by review.
6. **A raw shared-pool read still differs across an unrelated build.** The aliasing guard of §4.4 makes every public query correct and leaves the underlying buffer pool shared, so 175 raw field reads in the probe still differ. Code that reaches into a `BlockDP`'s fields rather than calling its queries is unsupported, and nothing in the engine does.

H Implications for human play

This project exists because a person plays this game and wanted a better opponent. This appendix answers the questions that came from live play, in the order they were asked, and separates what the evidence supports from what it does not.

H.1 Does FishBot play for itself or for the team?

For the team, unambiguously, and this is a property of the objective rather than a tuning choice. Every quantity the value function reads is team-relative: the score differential is the team's, the card counts are the team's totals, and the per-half-suit control term is the expected fraction of that half-suit held by the *team*, not by the seat. There is no per-seat score in this game and none in the policy. A configuration that maximised the acting seat's own holdings would be a different program, and it is not the one that ships.

What is true, and is the substantive half of the question, is that the *policy* is unilateral. Through v0.5, FishBot maintained a deduction state for itself and for nobody else — the only two clones of that object anywhere in the policy are clones of its own — so no term could ask what a partner knows, or which of two teammates is better placed to act. Teammates entered only as aggregate probability mass. v0.6 adds the machinery for per-seat modelling, uses it inside the search, and reports that at the resolution of §10 the terms built on it are worth nothing. That is a finding about this game rather than a concession: with every card movement public, most of what a partner knows, you already know.

H.2 Why a well-trained human still out-reasons it

The honest answer from this study is that the gap is in the *action space*, not in the arithmetic. M1, the mechanism that fixed v0.4's deadlock, restricts the candidate set to asks the actor cannot prove will miss. That is the right fix for the deadlock and it deletes the move class that most named human tactics are built from: blackballing, deliberately handing the turn to the opponent who can do least with it, and paying for a signal with a safe ask are all a guaranteed miss at a *chosen* seat. Such a move is available at 79.2% of decisions, and at two or more distinct chosen opponents at 50.1%.

§8.5 reports what happened when it was given back. Unbanned wholesale, it raises the win rate and destroys the policy. Rationed against the fitted linear score, it is never chosen — and the reason is structural rather than a matter of weights: the hit-probability term is zero on a deliberate miss by construction, and every other term of the score is a penalty, so no setting of the admission margin changes the output. Its value is the value of the position the donated turn produces, which is a forward-looking quantity that no static feature computes. Offered to the search, which is the only evaluator in the system that can price it, it produces the best search configuration measured, and the search as a whole is the one mechanism in this study that beats a static rule (§8).

So: a human who blackballs is doing something the bot structurally cannot express, and the bot is not obviously losing points by failing to. Both halves of that sentence are measured.

H.3 Deception

The manoeuvre the project owner reported — holding cards of a half-suit you were asked for, then declining to ask back in it — is implemented as the *withholder* archetype and is the one opponent against which v0.6 is decisively better. The gain replicates in sign and size on two disjoint seed banks and is the largest replicated effect in this study. It comes from the refit rather than from an opponent model: the fitting panel contains the archetype and the objective is minimax regret, so the optimiser buys robustness against it directly.

The complementary archetype, the *feint*, manufactures a misleading ask-legality certificate instead, and it remains the worst cell in the profile. v0.5's exposure there was created by its own fit and could not be tuned away at fixed weights; v0.6's fit includes it in the panel and improves it on one bank while giving a little back on the other.

H.4 Playing against it

Three things a person should know when sitting down against this program.

- **It never asks a question it can prove will fail.** That is a hard deduction from your own public record, so a question it does ask is always one it cannot rule out — which means an ask carries less information about its hand than a human's ask does.
- **It is deterministic.** Given the same public history and the same hand it plays the same move. A person who has played many games against it can, in principle, read it. Adding stochasticity is the outstanding item: the machinery for a partner-aware temperature is specified and the harness now supports the two partner regimes as separate columns, but the temperature itself is not fitted here.
- **Its declarations are conservative and accurate.** In mirror play it declares wrongly on roughly two percent of declarations, and the pre-gates that decide when to evaluate a declaration are exact — 0 false negatives in 14,449,770 rejections. If it declares, it is almost always right; the exploitable side is the timing, not the accuracy.

H.5 Where a human should expect to keep an edge

At more than half of its decisions the program's own score cannot separate its leading candidates, and §6.3 establishes that a correct posterior cannot separate them either. A human facing the same information is in the same position. This is worth stating plainly because it is the opposite of the usual intuition about card-game programs: the residual uncertainty here is not a modelling deficiency that better software will remove — it is the game.

Where a human should expect an edge is in the places this study found and could not close: choosing *which* opponent to hand the turn to when nothing good is available, paying a small material price for

a signal a partner will read, and varying behaviour so as not to be read. All three are moves in the class M1 deletes.

I Extended related work

This appendix records background that informed the design but that the main argument does not depend on. Nothing here supports a result in §11; a reader who wants only the load-bearing literature should read §3 and stop there. The organising convention is that each entry says what the work does and then says which of three things this study did with it: **uses** it as method, **cites** it as context, or **declines** it. §I.10 collects the declined set, because in a study whose thesis is that the binding constraint is information rather than machinery, the machinery left out is part of the argument.

I.1 Further informal sources on the game

Beyond Pagat [4] and Develin [5] the descriptive corpus is thin. Pagat's tactics list covers deliberate misses that inform a partner, locking out a dangerous opponent by never asking them anything, and exhausting one rank across all opponents before switching; it also records a partner-signalling convention and a "Forced Claims" rule, both attributed to named contributors. Develin's chapter contains the corpus's only quantitative endgame discussion, in which the alternative to a coin-flip declaration would need a success probability greater than one. The Wikipedia entry [8] names the "stalemate breaker", and secondary strategy pages [6, 7] restate blackballing, asking the asker, and an endgame delegation heuristic. These are used as evidence about how the game is played and talked about, not as sources of quantitative claims. The two substantial sources also disagree with each other on the point that matters most to a study of conventions: Develin's chapter forbids pre-agreed conventions outright, where Pagat records a partner-signalling convention as an accepted tactic. That disagreement is why the partner regime is a declared parameter in §7.6 rather than a fixed assumption.

On Somani's agent [9, 10]: it is a Python implementation with a complete rules engine and a sound deduction fixed point, and the four-player, 48-card variant it implements makes the declaration problem structurally easier than the one studied here. With a single teammate, naming the allocation of a six-card half-suit ranges over 2^6 possibilities and is frequently forced by the deduction fixed point; with two teammates it ranges over $3^6 = 729$, and the allocation question stops being a formality. No strength numbers have been published for it, so it is not usable as a baseline in any case. The v0.4 report records a reading of its training code that suggests the released model never received a win/loss gradient; that reading is repeated here without re-verification, because nothing in this study depends on it.

Valet [13] is the one piece of 2026 infrastructure that could change the situation. It encodes twenty-one traditional imperfect-information card games in a common description language and publishes measured branching factors and game durations, which is exactly the external calibration a claim like "this game is hard" needs. Whether any of the twenty-one is a team game is not stated on its abstract page, and the full text was not fetched, so no comparison is drawn anywhere in this report. Recorded so that the next study fetches it rather than re-deriving the question.

I.2 Determinization and perfect-information sampling

§3.2 states that averaging perfect-information values over deals sampled from the public history collapses to a hit-probability heuristic in this game. The collapse is the following. Almost every reachable position carries the same perfect-information value, because the team on turn takes everything it can

reach, so with t ranging over opponents, c over askable cards and h the public history,

$$\arg\max_{(t,c)} \mathbb{E}[V^{\mathrm{PI}}(t,c)] \;\approx\; \arg\max_{(t,c)} \Pr[\,t \text{ holds } c \mid h\,].$$

The right-hand side is one ply deep and blind to the information an ask leaks, the information a refusal supplies and the option value of postponing a declaration. This is a degeneracy of the perfect-information game itself, and it is why the determinized search of §8 scores candidates by rolling out under a belief-limited blueprint rather than by solving the sampled world.

Determinization for bridge was proposed well before it was made to work [14], and constrained deal generation in practice is done by rejection: deal against restrictions, then discard deals the program's own bidding module would not have produced [15]. Frank and Basin's diagnosis — strategy fusion and non-locality — is what makes it unsound however many worlds are drawn [16], and the repairs that work are those attacking non-locality as well as fusion [17]. Long, Sturtevant, Buro and Furtak's leaf correlation, bias and disambiguation factor explain when it nevertheless performs well [18]. Information Set Monte Carlo Tree Search replaces the independent trees with a single tree over information sets, redeterminizing each iteration [19], and has been extended by re-determinizing at non-root seats [20]. **Cited as context**; none is used as an engine here.

$\alpha\mu$ [21] deserves a separate note because it is the closest thing in this literature to a repair this study could have adopted. It backs up Pareto fronts of boolean world-vectors instead of scalar averages, which represents "this plan works in world set A, that plan works in world set B" without committing to a scalar mean — structurally the same move as the multi-valued leaf of §7.5, one ply down — and its published timings are an existence proof that vector-valued search over roughly twenty worlds is affordable in about a second per move on a CPU. Its reported gain in Bridge comes from differing with plain sampling on only a small fraction of decisions, which is the shape a good search should have and the calibration §8 uses when judging whether its own search is mis-scaled. **Cited as context, and its diagnostic used**: the algorithm is declined because it repairs unsoundness rather than degeneracy, and this game suffers the latter.

Knowledge-based paranoia search [22] is the closest methodological neighbour this study has: it runs on a CPU with no network, is built around deduced knowledge rather than sampled worlds, and reasons about the worst case over consistent worlds rather than the average. All three match this engine, whose deduction state carries hard exclusions, capacity constraints and ask-legality certificates, and whose count law already computes the probability of a named allocation. Worst-case-over-consistent-worlds is also the right criterion for the one irreversible action in the game. **Its criterion is used** in the declaration path of §7.4; its game-tree search is declined, because Skat's horizon and branching factor are not this game's. No compute cost is attributed to it anywhere, because its runtime is not stated on its abstract page.

I.3 Depth-limited and subgame solving

Brown, Sandholm and Amos [27] is the one result in this group that is **used as method**. Its content is a single observation with a large consequence: a search that stops at a depth limit and evaluates the leaf with one value function has silently assumed the opponent plays one fixed continuation, and that assumption is what makes depth-limited search in an imperfect-information game unsound. Letting the opponent choose among k continuations at the limit, each yielding its own leaf values, forces robustness to all of them, and the demonstration ran on a four-core CPU and 16 GB rather than a supercomputer. The prerequisite it identifies as expensive — a portfolio of distinct continuation strategies — is already paid for in this project, because a set of compiled baseline policies of different styles exists and is used for evaluation. The caveat is stated in §7.5 and repeated here: the soundness argument is two-player zero-sum, this game's leaf has three uncorrelated opponent seats, and the

construction is therefore adopted as a robustification heuristic. A second caveat is that the method assumes a blueprint approximating equilibrium, and this policy is a fitted heuristic, so what the leaves buy is robustness against a style portfolio and not robustness against equilibrium deviations.

Zhang and Sandholm's knowledge-limited subgame solving [28] is **cited as context and its warning honoured**. Their motivation is precisely this game's blocker: current techniques analyse the entire common-knowledge closure, which is acceptable when that closure is small and impossible when it is not. Their remedy is to work with low-order knowledge, pruning nodes no longer reachable on arrival at an information set, and their honest result is that this can increase exploitability, after which they develop the conditions that restore safety. This engine already ships a crude instance of the same move — a gate that restricts candidate asks to those with strictly positive hard-consistent probability — whose measured gains were large and whose exploitability cost has never been measured. §7.7 exists because of this paragraph.

Subgame solving in adversarial team games [29] builds a gadget from the team belief DAG and refines a blueprint by column generation, running in polynomial time given a best-response oracle when the blueprint is sparse. **Declined**, and recorded as a verified non-transfer: the sparsity condition is the escape hatch, and a policy with support over roughly seventy legal asks at each of roughly a hundred decisions per game is not sparse. It is named explicitly because it is the most natural thing a reader will ask about.

The counter-strategy line is **cited for one warning**. Continual depth-limited responses [34] state plainly that existing robust response methods do not compose with limited-look-ahead solving off the shelf, and supply a fix; adapting beyond the depth limit [35] extends the idea to counter-strategies in large games while staying robust against rational opponents. Any future version of this policy that carries both a depth-limited search and an online opponent model needs that composition argument, and this report does not have one, because it builds only the first. Look-ahead reasoning with a learned model [36] is the right direction and requires a learned abstraction, so it is deferred. Equilibrium refinements in gadget games [37] are not applicable here at all — no gadget game is solved anywhere in this study — but the finding transfers as a meta-lesson: when a subproblem has many optima that are theoretically equivalent, the choice among them is worth more than half of the exploitability, and that is a strong statement of the tie-breaking argument in §6.2.

The full-solver line is **cited as context and declined as implementation**. DeepStack [40] introduced continual re-solving from a range and a set of opponent counterfactual values; its re-solve gadget requires one value per opponent information set, which at three opponents holding nine cards each is not reachable on any CPU budget. ReBeL [38] treats the public belief state as the state and proves that running the same procedure at test time is safe, which is the licence under which §8 operates at all; its algorithm iterates over the belief support and is therefore blocked by cardinality. Student of Games [39] grows the public tree during solving and identifies Scotland Yard — a public-observation, hidden-position game — as the closest published analogue to this information structure; the transferable piece is architectural and cheap, namely that a candidate set should be grown where the search is uncertain rather than fixed at a constant, and even that is not implemented here.

I.4 Policy-aware inference in trick-taking games

The Skat line is the direct ancestor of this study's belief construction and is **used**: reweighting the deals consistent with the public record by a model of how each player would have acted, whether through a supervised per-card location model [24] or through reach probabilities taken from a policy [25], and reported as worth +217 tournament points per 36 games as a standalone module [23]. The policy prior of §5.3 is a two-parameter version of the same idea. What §5.4 contributes to this line is a case where the trade-off inverts far enough to be visible in win rate: exactness is affordable here, and the exact object still loses, because it is the one that cannot see behaviour.

History filtering — constructing a history consistent with a public state — is FNP-complete in the worst case, and enumeration is polynomial only for sparse public trees [26]. **Cited as context**, and this game escapes the problem entirely: its public record determines every card movement, so the only object to be reconstructed is the initial deal, which is what makes the exact count law of §5.2 possible.

Belief modelling in cooperative games is **declined**. The Bayesian Action Decoder made the public belief an explicit object of the policy [43], and its independent-marginals factorisation is the part that does not survive contact with fixed hand sizes and card conservation; the repair is the constrained scaling and block enumeration this engine already implements. Approximate policy iteration over common-payoff games [47] formalises factorised prescriptions and is a formalism rather than a tool here. Learned Belief Search [32] learns an approximate belief because Hanabi's exact belief is intractable; this game's is not, so learning one would be strictly worse than what exists. That is a real structural advantage of this domain and is stated plainly in §3.4: the belief half of the problem is solved, and only the search half is open.

I.5 Cooperative play, conventions, and deviations

Hanabi is the reference benchmark for coordination through observable actions [42], and is relevant because this game likewise has no side channel. The Simplified Action Decoder showed that letting teammates condition on the greedy rather than the exploratory action is a prerequisite for conventions to survive exploratory training [44]; Other-Play demonstrated how much of a self-play score is convention rather than skill, with the same agents scoring far worse when paired across independent training runs [45]; and Off-Belief Learning provides an explicit dial on convention depth together with a uniqueness result that makes independent runs interoperable [46]. Test-time search over a fixed blueprint provides a further improvement without any training at all [31]. Joint policy search, which scores simultaneous changes at several information sets — which is what inventing a convention requires — improved a bridge bidding system by +0.63 IMPs per board against a strong commercial program [50]. All **cited as context**; none is trained here.

Two structural asymmetries separate this game from Hanabi and are worth stating because they cut in opposite directions. The first is the audience. Hanabi has no adversary, so every bit of signal is pure profit; here a leaked card location is directly executable, because once it is public that a player holds a card, any opponent holding another card of that half-suit can take it and keep the turn. The Hanabi construction that most clearly fails to port is the modular hat-guessing code, which works because each receiver sees every other receiver's hand and can subtract it [49]; here no player sees any hand. The second asymmetry runs the other way. Off-Belief Learning's grounded level-one play, which refuses to read conventional meaning into a partner's action, is an approximation in Hanabi and is nearly exact here, because ask legality is a rules-level certificate: a legal ask proves its asker holds another card of the named half-suit, with no convention involved. The prediction that grounded play is much closer to optimal in this game than in Hanabi is testable with a single flag and no training, and it is the ablation of §11.8 that separates hard certificate reading from the soft prior weights.

Self-explaining deviations [48] are the most on-topic piece of this literature for the question of finding non-obvious plans, and are **declined for this version with the reason recorded**. A self-explaining deviation is an action that departs from what common understanding calls reasonable, taken so that a teammate reasoning by theory of mind concludes that circumstances must be abnormal; the associated algorithm produced the first algorithmic finesse plays in Hanabi. The analogue here is exact and currently unexploited: an ask certifies a holding, so an ask with low hit probability that certifies something a teammate could not otherwise deduce is precisely such a deviation, and the belief machinery to score one already exists. It is declined in this study for two reasons. The information features of the shipped policy are all negative — it charges for information rather than paying for it — so the mechanism would have to be built rather than tuned; and a deviation of this kind is a convention

by another name, which means it is the mechanism most likely to look excellent in self-play and fail against a human or an earlier version. Any future attempt must be reported with its cross-play cell and gated behind the partner regime of §7.6.

I.6 Human-compatible play and regularised search

piKL [51] regularises a search result toward a reference policy with a divergence bound that shrinks as the regularisation weight grows, and reports improved top-one agreement with human moves in chess and Go alongside strong play. **Cited as context, and its shape borrowed.** The framing that matters here is not human-likeness: it is that a KL anchor is a tunable, measurable interpolation between a reference policy and an unconstrained search, with the reference recovered exactly in the limit, so an ablation along that axis has a guaranteed floor. In a team game the anchor also does real strategic work, because a search that finds a clever line its own teammates will misread is a net loss.

Human-regularised search and learning [52] extends this to coordination with real partners in three steps, of which the third — regularised search at test time to absorb partner distribution shift — is the published architecture for exactly the requirement that this policy behave differently with a bot partner and a human one. **Cited as context**; there is no human corpus for this game, so it is not implemented. Cicero [53] is cited for the family rather than the architecture: a seven-player mixed cooperative-competitive game whose planning core is an anchored search over actions, and whose lesson is that in games with allies the planner's job is to act well against a model of what the other players will actually do rather than against an equilibrium. The ad-hoc human-AI coordination challenge [54] supplies the negative instruction this study follows: an agent trained with no human data at all outperformed a best-response-to-behavioural-cloning baseline against human proxies, so a small human corpus is not the way in.

I.7 Team games and common information

The common-information approach [55] reformulates a decentralised control problem as a single-agent problem for a coordinator who knows the common information and selects prescriptions — maps from each controller's local information to its action — and shows that the resulting problem is a POMDP. **Used as a correctness lens.** Two consequences are load-bearing in this report. First, it names the design object: the public deduction state is one shared thing, and what distinguishes seats is a refinement of it, which is the mechanism of §7.3 and the reason that mechanism is $O(1)$ rather than six-fold. Second, it explains why teammates must run identical procedures: a prescription is agreed in advance, and a seat that deviates by searching invalidates its teammates' belief updates. The coordinator's own problem is not solvable at this scale — a prescription maps every nine-card hand to an ask — so nothing here computes one.

The equilibrium-computation literature for teams is **cited to delimit scope and declined in full**. Celli and Gatti establish the complexity of the team-maxmin equilibrium with correlation and the worst-case cost of forgoing correlation [56]; Farina and coauthors connect ex-ante team collusion to extensive-form correlation [57]; the team belief DAG and the two-player conversion give a zero-sum game whose equilibria are realization-equivalent to a team-maxmin equilibrium with correlation [58, 59], at a representation cost exponential in how much private information distinguishes teammates inside a public state. That exponent is the blocker: three teammates hold unrestricted nine-card hands, and one seat's information set at deal time contains $45!/(9!)^5 \approx 1.9 \times 10^{28}$ deals. Hidden-role games [60] are cited only to delimit scope: teams here are public from the deal, so there is no role uncertainty and none of that machinery applies. Nothing in this report is an equilibrium result, and no claim in it should be read as one.

I.8 Counting and constrained sampling

Counting the deals consistent with a public history is a permanent computation. Ryser's inclusion–exclusion formula is exact in $O(2^n n)$ time [61], which is unusable at $n = 54$, and Glynn's sign-vector formula is the other exact method [62]; the Jerrum–Sinclair–Vigoda chain is a fully polynomial randomised approximation scheme [63] whose constants make it impractical at this scale. Matrix scaling is the usual practical approximation and carries a deterministic strongly polynomial guarantee [64]; a Sinkhorn iteration of this kind is what the deployed approximate path uses. Sequential importance sampling is the standard contingency-table alternative [65], and is not used, because it can underestimate badly while appearing to have converged. **Cited as context, with one used.**

The reason none of this is on the critical path is structural. The hidden state is exactly the initial deal, each seat's share of it is a known constant number of cards, and every ask-legality certificate is confined to a single half-suit, so the counting problem factorises into blocks that can be enumerated exactly within a half-suit and combined by a capacity dynamic programme. That is the construction of §5.2, and it is why this study can ask whether exactness helps rather than whether it is affordable.

I.9 Evaluation methodology and variance reduction

The v0.4 study declined variance reduction outright. This study takes the same decision about fitted control variates and a different one about pairing, and the distinction is worth stating because the two are often filed together. Advantage-sum estimators [67] and AIVAT [68] learn or hand-build a value function and subtract it, and their reported gains are large in two-player settings and smaller in six-player ones, which is the closer analogue here. They remain **declined**. AIVAT is unbiased for *any* value function, which is a constraint on its expectation rather than on any realised estimate, and a control variate fitted by the same author on the same agent using the same evaluation data is the pathology that literature itself warns about.

What this study adopts instead is pairing on common random numbers: the same deals played in both arms, with treatment and control alternated across seat rotations, and the difference taken per deal (§10.3, §9.2). It costs nothing to implement, introduces no fitted object, and removes the largest part of deal variance — the report quotes the realised width ratio against the unpaired estimator rather than asserting a benefit. It is also the standard remedy rather than a novel one: duplicate bridge replays each board with the same cards in the same seats, and the computer poker competition added common seeds across pairings for the same reason [66]. The cost of the alternative is that a fitted estimator would have to be validated, and validating it would consume more games than pairing saves.

One further piece of methodology is named because it is frequently applied here by analogy and should not be. Refining an abstraction does not monotonically reduce exploitability [69], which sounds like the general form of §5.4's finding that substituting a more accurate belief into a policy fitted around a less accurate one does not improve it. The analogy is suggestive and the result does not transfer: this policy is not an abstraction of a solved game, so neither the pathologies that literature describes nor its guarantees apply. The finding here is measured on its own terms and defended by a control, not inherited.

Rating and stopping are treated briefly. Bradley–Terry-style rating models [71, 72] are not invariant to redundancy in the rated population and cannot represent cyclic dominance, both of which are real failure modes when the rated population is one training run's own checkpoints; maximum-entropy Nash averaging over the antisymmetric logit matrix is the proposed alternative [70]. Ratings appear in this report only as a compact summary of a fixed round-robin, never as a claim about a strategy space. And monitoring an interval computed for a fixed sample size and stopping when it looks favourable is not a test, which is why sequential designs on normalised Elo exist [73] and why the seed

banks here are fixed in advance and reported at their planned size (§10.5).

I.10 Work this study deliberately does not use

The declined set, with the reason for each. These are design decisions, and stating them is the point of this appendix.

- **Full solvers over public belief states** [40, 38, 39]. Every one requires iterating or learning over a common-knowledge closure whose cardinality here is on the order of 10^{28}. The closure is used as a sufficient statistic for evaluation and never as an index set for iteration. ReBeL's safety result is retained as the licence for test-time search; its algorithm is not.
- **The team belief DAG and subgame solving over it** [58, 59, 29]. Representation cost is exponential in the private information distinguishing teammates inside a public state, and the polynomial escape hatch is conditional on blueprint sparsity that this policy does not have.
- **Learned belief models** [32, 43]. The exact belief is computed and oracle-validated here, so an approximation would be strictly worse. This is the one place where this domain is easier than Hanabi rather than harder.
- **Neural policies and decision-time gradient updates** [33]. There is no network training infrastructure in this project and none is warranted by the results: the two largest effects measured in this report are a tie-break and a statistical guard, neither of which is a capacity problem. The 2026 lightweight-agent study reaches the same conclusion independently [74].
- **Hidden-role machinery** [60]. Teams are public from the deal; there is no role uncertainty to model.
- **Gadget-game equilibrium refinement** [37] and **learned-model look-ahead** [36]. No gadget game is constructed here and no model is learned; both are recorded for their transferable lessons only.
- **Fitted control variates for evaluation** [67, 68]. Declined for the reason in §I.9; pairing on common random numbers is adopted instead.
- **Self-explaining deviations** [48]. Declined for this version rather than permanently, with the construction sketched in §I.5 so that the next study can build it against a cross-play measurement rather than a self-play one.

Two exclusions are not in this list because they are not choices. There is no equilibrium computation anywhere in this report, and no claim in it is an equilibrium claim. And there is no human dataset, so every statement about human play in §H is a statement about a small number of recorded games and is scoped as such.